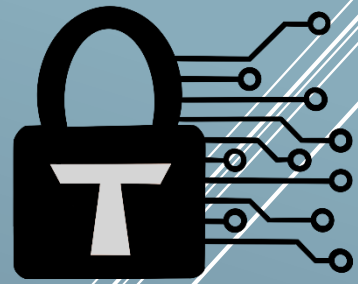


Trust Security

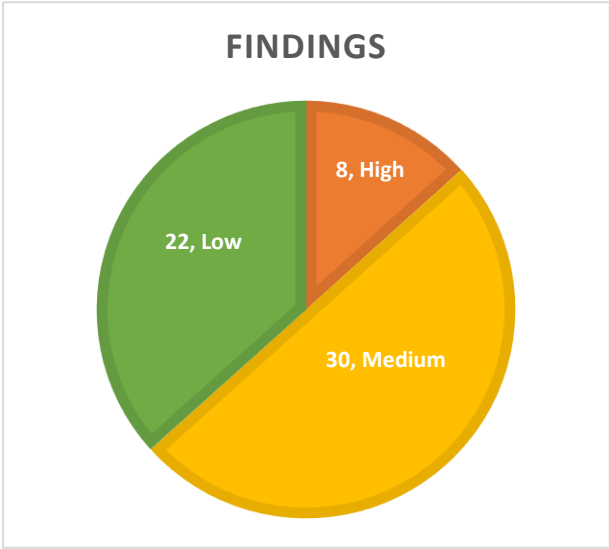


Smart Contract Audit

Curvance

23/11/2025

Executive summary



Category	Lending
Audited file count	41
Lines of Code	6364
Auditor	Trust HollaDieWaldfee ether_sky
Time period	25/08/2025 - 17/10/2025

Findings

Severity	Total	Fixed	Acknowledged
High	8	8	0
Medium	30	22	8
Low	22	15	7

Centralization score



Signature

EXECUTIVE SUMMARY	1
DOCUMENT PROPERTIES	6
Versioning	6
Contact	6
INTRODUCTION	7
Scope	7
Repository details	8
About Trust Security	8
About the Auditors	8
Disclaimer	9
Methodology	9
QUALITATIVE ANALYSIS	10
FINDINGS	11
High severity findings	11
TRST-H-1: Oracle deviation thresholds are not accounted for	11
TRST-H-2: Duplicate accounts in liquidation allow to liquidate beyond closeFactor	14
TRST-H-3: Unsafe Redstone default heartbeat	17
TRST-H-4: Stale interest accrual enables bad debt creation and incorrect liquidations	19
TRST-H-5: Liquidating healthy account with unhealthy account allows to remove all debts	20
TRST-H-6: The _checkTransfer() function uses wrong account for transferFrom()	23
TRST-H-7: The isCollateral flag is true on first borrows which leads to mispricing in OracleManager	24
TRST-H-8: Min/max price guards can override oracle prices	26
Medium severity findings	27
TRST-M-1: The redeemPaused flag only applies to collateral removals	27
TRST-M-2: Native token address inconsistency causes incorrect token equality checks	28
TRST-M-3: Accumulated debt rounding creates unrecoverable protocol value	29
TRST-M-4: Liquidation rounding allows debt manipulation and breaks market debt invariant	30
TRST-M-5: Missing interest rate accrual before interest rate parameter update	32
TRST-M-6: Changing collateral into borrow position is not possible due to 1 wei rounding difference	33
TRST-M-7: VaultAggregator doesn't support decimal offset	35
TRST-M-8: DynamicIRM doesn't cap the vertexMultiplier when vertexMultiplierMax is updated	36
TRST-M-9: Base reserve is ignored in the interest rate calculation	37
TRST-M-10: BasePositionManager.onRedeem() doesn't support swapping through native assets	38
TRST-M-11: Multiplying two token amounts should use full precision	39
TRST-M-12: Single feed is more permissive than dual-feed	40

TRST-M-13: DAO retains market permissions during timelock / emergency council transfer	41
TRST-M-14: DAOTimelock doesn't consider overlap in Emergency Council and DAO address	42
TRST-M-15: Rounding of marketDebtIndex breaks invariant	43
TRST-M-16: Memory corruption in Bytes32Helper can return wrong oracle prices	46
TRST-M-17: Debt repayment can be grieved in BasePositionManager and BaseZapper	47
TRST-M-18: Interest rate manipulation via uncollateralized deposits and withdrawals	48
TRST-M-19: Borrowers may not experience delayed multiplier and interest rate updates	50
TRST-M-20: Effective adjustment periods are not 10 minutes and accrual paths behave differently	50
TRST-M-21: Borrows below MIN_ACTIVE_LOAN_SIZE are possible due to repayments and exact liquidations	52
TRST-M-22: RedStoneCoreAdaptor.addAsset() cannot handle native tokens	53
TRST-M-23: Zappers lack validation to prevent callbacks	55
TRST-M-24: Non-atomic IRM upgrade can deny access to the protocol	55
TRST-M-25: Limited interest rate precision can cause significant rounding errors	56
TRST-M-26: Transient liquidation configuration is not enforced to be used in isolation	58
TRST-M-27: AuctionManager does not support dynamic close factor	59
TRST-M-28: AuctionManager should deny oracles and selectors by default	59
TRST-M-29: AuctionManager.setAtlasCloseFactor() as a governor protected function is incompatible with requirements	60
TRST-M-30: ERC4337 transactions can be denied due to callbacks, leading to DOS of the protocol	61
Low severity findings	64
TRST-L-1: CTokens breaks ERC4626 standard	64
TRST-L-2: BaseCToken transfers are not compatible with ERC20	64
TRST-L-3: OracleManager and downstream contracts revert instead of returning error codes	65
TRST-L-4: BaseCToken.supportInterface() breaks EIP165	66
TRST-L-5: The transferEmergencyCouncil() and transferTimelockPermissions() functions don't update the roles in DAOTimelock	67
TRST-L-6: DAO can transfer rounding protection shares from BaseCToken	67
TRST-L-7: The rescueToken() safety hatch can't be overridden to account for external vault shares	68
TRST-L-8: BasePositionManager.maxRemainingLeverage() is too restrictive to max out debt	69
TRST-L-9: BorrowableCToken miscalculates new debt in borrowForPositionManager()	70
TRST-L-10: Heartbeat grace period of 60 seconds is too short	71
TRST-L-11: RedstoneCoreAdaptor price guard must be applied when the price is read	72
TRST-L-12: ActionRegistry cooldown can be bypassed through borrow operations	74
TRST-L-13: Market permissions are lost when address becomes EmergencyCouncil or Timelock	74
TRST-L-14: Sanity check in depositAndLeverage() allows excessive slippage	76
TRST-L-15: Donated assets can cause totalAssets to underflow on bad debt realization	77
TRST-L-16: Missing accountPositions cleanup after liquidation can cause DoS	79
TRST-L-17: Bad debt can't be realized when the collateral token becomes worthless	82
TRST-L-18: Removing asset from adapter can revert if not configured as data feed	83
TRST-L-19: VaultZapper leaks malicious approval	85
TRST-L-20: BaseOracleAdaptor.removeAsset() does not clear price guards	86
TRST-L-21: AuctionManager verifies oracles and selectors separately	86
TRST-L-22: AuctionManager checkMetacallGasLimit=false setting introduces grieving risk	87
Additional recommendations	89
TRST-R-1: Heartbeat grace period is not applied to custom heartbeat values	89
TRST-R-2: In Zapper Interface, expectedShares can be ineffective and is not applied consistently	89
TRST-R-3: Collateral can be withdrawn with regular allowance	90

TRST-R-4: Flashloans enabled on uninitialized markets allow exchange rate manipulation	90
TRST-R-5: Slippage check prevents deleveraging for accounts in bad debt	90
TRST-R-6: Fee validation is bypassed when swap validation returns early	91
TRST-R-7: Inconsistent use of <code>_getTotalAssets()</code>	91
TRST-R-8: OracleManager doesn't round according to <code>getLower</code>	92
TRST-R-9: Redstone Core Adaptor allows zero prices when price guard is enabled	92
TRST-R-10: AuctionManager ETH transfer reentrancy	93
TRST-R-11: DAOTimelock duplicated code	93
TRST-R-12: BaseZapper should inherit Multicall	94
TRST-R-13: Misleading naming of delegation check	94
TRST-R-14: VaultAggregator should cast to ERC4626 Interface	94
TRST-R-15: Price guard configuration sanity checks	95
TRST-R-16: Inconsistent use of <code>fullMulDiv()</code> in BaseOracleAdaptor	96
TRST-R-17: Vault aggregators are incompatible with RedstoneClassicAdaptor	96
TRST-R-18: VaultZapper and NativeVaultZapper don't override <code>redeemSwapAndDeposit()</code>	96
TRST-R-19: <code>MAX_LIQUIDATION_INCENTIVE</code> should be applied to <code>liqIncHard</code>	97
TRST-R-20: Insufficient range for <code>minPrice</code> in BaseOracleAdaptor	97
TRST-R-21: Auction buffer unexpectedly applies to <code>cHard</code>	97
TRST-R-22: Approval not correctly reset in BasePositionManager	98
TRST-R-23: Chain removal should be protected by elevated permissions	98
TRST-R-24: Validation improvements	98
TRST-R-25: Redundant liquidation pause flags in MarketManagerIsolated	99
TRST-R-26: Set <code>_useVirtualShares()</code> to false	100
TRST-R-27: Incomplete price validation in token configuration checks	100
TRST-R-28: RedstoneCoreAdaptor. <code>getPrice()</code> should return price even if error is true	101
TRST-R-29: SwapperLib should round up slippage	101
TRST-R-30: SwapperLib underestimates <code>outAmount</code> when <code>inputToken</code> equals <code>outputToken</code>	101
TRST-R-31: Unnecessary swap action required when <code>debtAsset</code> equals <code>collateralAsset</code>	101
TRST-R-32: Document that slippage check in BasePositionManager is sanity check, not security guarantee	102
TRST-R-33: Cooldown reduction side effects should be documented	102
TRST-R-34: Unnecessary WAD multiplication and division in leverage calculation	103
TRST-R-35: Unify protocol interest fee config	103
TRST-R-36: Inconsistent <code>msg.value</code> validation and unnecessary <code>receive()</code> in BaseZapper	103
TRST-R-37: Dead code for <code>swapPermit2</code> in <code>OdosCallDataChecker</code>	104
TRST-R-38: Introduce lower max for multiplier	104
TRST-R-39: Insufficient precision in DynamicIRM threshold derivation	104
TRST-R-40: LiquidityManagerIsolated <code>_collateralValue()</code> should round like <code>_assetValue()</code>	105
TRST-R-41: BasePositionManager <code>leverage()</code> and <code>deleverage()</code> should check <code>cToken</code> is listed before interaction	105
TRST-R-42: <code>MAX_DEBT_CAP</code> is inconsistent with <code>uint136</code> <code>totalSupply</code> assumption	105
TRST-R-43: Use custom error instead of underflow revert when removing too much collateral	105
TRST-R-44: Inconsistent rounding in protocol fee share calculation	106
TRST-R-45: OracleManager should not be called with <code>address(0)</code> asset	106
TRST-R-46: BaseCToken should support <code>flashLoan()</code> as a source of revenue	107
TRST-R-47: Redundant overflow check for revenue accumulation in AuctionManager	107
TRST-R-48: CentralRegistry should support locking markets	107
TRST-R-49: AuctionManager UserOp can call unintended functions	107
TRST-R-50: AuctionManager destination setters should distribute revenue	108
TRST-R-51: <code>SwapUnsafe()</code> in Zappers	108
TRST-R-52: Redundant code	108
TRST-R-53: Coding style improvements	110

TRST-R-54: Documentation errors	112
Centralization risks	116
TRST-CR-1: Emergency Council	116
TRST-CR-2: DAO	116
TRST-CR-3: DAO Timelock	117
TRST-CR-4: Risk Manager contracts with market permissions	117
TRST-CR-5: Auction permissions	117
Systemic risks	119
TRST-SR-1: Token risks and assumptions	119
TRST-SR-2: Oracle risks	119
TRST-SR-3: Min/max price guards can override oracle prices and block operations	120
TRST-SR-4: Extreme interest risk and interest assumptions	120
TRST-SR-5: External protocol risk	120
TRST-SR-6: Dual borrowable market setup can disincentivize liquidators	121

Document properties

Versioning

Version	Date	Description
0.1	17/10/2025	Client Report
0.2	22/11/2025	Mitigation Review
0.3	23/11/2025	Second Mitigation Review

Contact

Trust

trust@trust-security.xyz

Introduction

Trust Security has conducted an audit at the customer's request. The audit is focused on uncovering security issues and additional bugs contained in the code defined in scope. Some additional recommendations have also been given when appropriate.

Scope

Curvance contracts repository:

- contracts/architecture/CentralRegistry.sol
- contracts/architecture/DAOTimelock.sol
- contracts/calldata-checker/BaseCalldataChecker.sol
- contracts/calldata-checker/multicall-checker/BaseMulticallChecker.sol
- contracts/calldata-checker/multicall-checker/RedstoneAdaptorMulticallChecker.sol
- contracts/calldata-checker/swap-checker/BaseSwapChecker.sol
- contracts/calldata-checker/swap-checker/OdosCalldataChecker.sol
- contracts/libraries/ActionRegistry.sol
- contracts/libraries/Bytes32Helper.sol
- contracts/libraries/CentralRegistryLib.sol
- contracts/libraries/CommonLib.sol
- contracts/libraries/ConstantsLib.sol
- contracts/libraries/LowLevelCallsHelper.sol
- contracts/libraries/Multicall.sol
- contracts/libraries/PluginDelegable.sol
- contracts/libraries/ReentrancyGuardTransient.sol
- contracts/libraries/RescueLib.sol
- contracts/libraries/SwapperLib.sol
- contracts/market/DynamicIRM.sol
- contracts/market/isolated/LiquidityManagerIsolated.sol
- contracts/market/isolated/MarketManagerIsolated.sol
- contracts/market/position-management/BasePositionManager.sol
- contracts/market/position-management/NativeVaultPositionManager.sol
- contracts/market/position-management/SimplePositionManager.sol
- contracts/market/position-management/VaultPositionManager.sol
- contracts/market/token/BaseCToken.sol
- contracts/market/token/BaseCTokenWithYield.sol
- contracts/market/token/BorrowableCToken.sol
- contracts/oracles/adaptors/BaseOracleAdaptor.sol
- contracts/oracles/adaptors/chainlink/ChainlinkAdaptor.sol
- contracts/oracles/adaptors/redstone/RedstoneClassicAdaptor.sol
- contracts/oracles/adaptors/redstone/RedstoneCoreAdaptor.sol
- contracts/oracles/adaptors/wrappedAggregators/BaseWrappedAggregator.sol

- contracts/oracles/adaptors/wrappedAggregators/VaultAggregator.sol
- contracts/oracles/adaptors/wrappedAggregators/WstETHAggregator.sol
- contracts/oracles/OracleManager.sol
- contracts/plugins/BaseZapper.sol
- contracts/plugins/market/NativeVaultZapper.sol
- contracts/plugins/market/SimpleZapper.sol
- contracts/plugins/market/VaultZapper.sol

Atlas integration repository:

- src/AuctionManager.sol

Repository details

- **Curvance contracts repository:** <https://github.com/curvance/curvance-contracts>
- **Curvance contracts commit hash:** ccc574e8cf60ad485768090f78363645aab1568c
- **Curvance contracts mitigation hash:** f10667cc05e3a57b2a63965fa20fc9953b521923
- **Curvance contracts second mitigation hash:**
193350535c7dcf1c1ff627f2f708be148ad0bf22
- **Atlas integration repository:** <https://github.com/curvance/Atlas-Integration>
- **Atlas integration commit hash:** 1cc1b9b1b6b448baa3372245e76df9c2aaf60aa5
- **Atlas integration mitigation hash:** 48e5a916d7392234f75999d601256d807f244d1a
- **Atlas integration second mitigation hash:**
98bb9383f80dcacc13c40a40884fbfe8d583d683

About Trust Security

Trust Security has been established by top-end blockchain security researcher Trust, in order to provide high quality auditing services. Trust is the leading auditor at competitive auditing service Code4rena, reported several critical issues to Immunefi bug bounty platform and is currently a Code4rena judge.

About the Auditors

Trust has established a dominating presence in the smart contract security ecosystem since 2022. He is a resident on the Immunefi, Sherlock and C4 leaderboards and is now focused in auditing and managing audit teams under Trust Security. When taking time off auditing & bug hunting, he enjoys assessing bounty contests in C4 as a Supreme Court judge.

HollaDieWaldfee is a distinguished security expert with a track record of multiple first places in competitive audits. He is a Lead Auditor at Trust Security and Senior Watson at Sherlock.

ether_sky is a security researcher with a focus on blockchain security. He specializes in algorithms and data structures and has a solid background in IT development. He has placed at the top of audit contests in Code4rena and Sherlock recently.

Disclaimer

Smart contracts are an experimental technology with many known and unknown risks. Trust Security assumes no responsibility for any misbehavior, bugs or exploits affecting the audited code or any part of the deployment phase.

Furthermore, it is known to all parties that changes to the audited code, including fixes of issues highlighted in this report, may introduce new issues and require further auditing.

Methodology

In general, the primary methodology used is manual auditing. The entire in-scope code has been deeply looked at and considered from different adversarial perspectives. Any additional dependencies on external code have also been reviewed.

Qualitative analysis

Metric	Rating	Comments
Code complexity	Good	Code is mostly simple and well structured, reducing attack risks.
Documentation	Excellent	Project is well documented.
Best practices	Good	Project generally follows best practices.
Centralization risks	Good	Project is managed by trusted roles, which can be set up with fine grained permissions.

Findings

High severity findings

TRST-H-1: Oracle deviation thresholds are not accounted for

- **Category:** Logical issues
- **Source:** OracleManager.sol
- **Status:** Fixed

Description

The protocol's oracle system fails to account for oracle deviation thresholds and oracle error accumulation when validating critical risk parameters, leading to multiple issues that can result in bad debt creation, DOS, and incorrect slippage calculations.

The *OracleManager* implements a dual feed divergence detection system with hardcoded global thresholds:

```
/// @notice The maximum allowed price feed divergence between prices
///     before `CAUTION` error code is returned, in `BPS`.
/// @dev 10050 = 0.5% = 50 basis point price feed deviation allowed.
uint128 public cautionPriceDivergence = 10050;
/// @notice The maximum allowed price feed divergence between prices
///     before `BAD_SOURCE` error code is returned, in `BPS`.
/// @dev 10100 = 1% = 100 basis point price feed deviation allowed.
uint128 public badSourcePriceDivergence = 10100;
```

Note that the system has a maximum configurable divergence of only **3%**:

```
/// @notice Maximum value that a price divergence flag can be set as
///     inside the protocol.
/// @dev 1.03e4 = 3.0%.
uint256 public constant MAX_DIVERGENCE_VALUE = 10300;
```

This discrepancy creates multiple issues. Firstly, it creates insufficient liquidation incentives. The protocol doesn't validate that liquidation incentives exceed combined oracle errors. When **IFactor = WAD**, liquidation incentives must exceed the combined oracle error of debt to collateral exchange rates to remain profitable. However, this might not be met due the thresholds set. For example, if the liquidation incentive is **1%** but the combined oracle errors for collateral and debt assets total 2%, liquidations can become unprofitable, leading to bad debt accumulation.

Another impact is that the divergence detection mechanism can lead to DOS. If the asset has a dual feed setup where feeds just have 0.5% deviation threshold (e.g. a Chainlink feed in a dual setup with a Redstone feed) then these deviations can add up to even cause a **BAD_SOURCE** error due to how `_checkBounds()` handles these feeds:

```
function _checkBounds(
    uint256 a,
```

```

uint256 b
) internal view returns (uint256) {
    if (a <= b) {
        // Check if both feeds are within `f.caution` of each other.
        if (((a * cautionPriceDivergence) / BPS) < b) {
            // Notify that the price is dangerous and to treat data as a
            // bad source because we are outside the accepted range of
            // divergence.
            if (((a * badSourcePriceDivergence) / BPS) < b) {
                return BAD_SOURCE;
            }

            // Notify that the price should be taken with caution because
            // we are outside the accepted range of divergence.
            return CAUTION;
        }

        return NO_ERROR;
    }

    // Check if both feeds are within `f.caution` of each other.
    if (((b * cautionPriceDivergence) / BPS) < a) {
        // Notify that the price is dangerous and to treat data as a
        // bad source because we are outside the accepted range of
        // divergence.
        if (((b * badSourcePriceDivergence) / BPS) < a) {
            return BAD_SOURCE;
        }

        // Notify that the price should be taken with caution because
        // we are outside the accepted range of divergence.
        return CAUTION;
    }

    return NO_ERROR;
}

```

For example:

If real price is **\$1.00**, Chainlink shows **\$0.995**, and Redstone shows **\$1.005**, the condition **\$0.995 * 1.01 < \$1.005** triggers **BAD_SOURCE**, causing DOS.

Another bad debt related risk is that the deviation thresholds are not considered in liquidation parameter validation. In *MarketManagerIsolated.updateTokenConfig()*, the validation only checks basic parameter relationships.

For example, if liquidation ratio is **1.01** (1% above bad debt) and Chainlink has a **2%** deviation threshold, an account can appear to meet the liquidation ratio at **1.01** while the actual ratio is **1.01 / (1 + 2%) ≈ 0.99**, creating bad debt.

It is also worth noting a specific behavior when multiple feeds are used for the oracle. When using combined feeds (e.g. native/asset × USD/native), errors multiply.

For example:

- Native/Asset feed: 2% deviation
- USD/Native feed: 2% deviation
- Combined error: ~4%

So, the combined deviation could be larger than 2% which enhances the oracle error.

Finally, with regards to slippage validation, *SwapperLib._swapSafe()* and *BasePositionManager.checkSlippage()* functions validate slippage against oracle prices without accounting for oracle errors. It means the estimates can be off by the price errors. Accordingly, oracle errors can make valid swaps appear to have excessive slippage or actual slippage can be masked by favorable oracle errors.

Recommended mitigation

It's recommended to define divergences per dual-feed rather than globally.

Team response

Fixed by commit [f10667c](#).

Mitigation review

The approach that has been implemented is to introduce adjustable deviation bounds in the *OracleManager* for each combination of **asset** and **inUSD**.

An issue has been identified with the implementation since *addAssetPricingAdaptor()* only sets the deviation bounds for the provided **inUSD** parameter. However, the price of the **asset** can be requested with the opposite **inUSD** flag in which case there are no deviation bounds configured. Deviation bounds equal to zero are used which cause **BAD_SOURCE** errors. It is recommended that the deviation bounds for both **inUSD=true** and **inUSD=false** can be provided at once.

Another issue is that **MAX_DEVIATION_BOUND** is set to **3%**. It has been established that Curvance wants to support price feeds with an error of up to **1%** and since *OracleManager* may multiply one price with a second to convert it from native to USD or vice versa, **3%** is insufficient to account for an error of **1%** in three different price feeds. The combined error is just above **3%**, and so it is impossible to configure the deviation bounds such that prices remain available through the maximum deviations in the feeds. It is recommended to increase **MAX_DEVIATION_BOUND** with the understanding that three rather than two combined oracle errors must be supported.

There have been no changes implemented inside the core protocol, i.e., the cTokens and the market manager. It is understood that protocol parameters must be set accordingly, e.g., liquidation incentives must be sufficient to offset the oracle errors such that liquidation remains profitable even if feeds deviate. There are no safety checks in the core protocol to ensure this. Furthermore, it has been acknowledged that price paths in *OracleManager* can be configured such that deviation bounds valid for one price path become invalid for another price path. This is a centralization risk since DAO Timelock and Emergency Council must be trusted to keep deviation bounds in sync with the price paths, i.e., with the oracle errors that are encountered along the path of pricing an asset.

Errors in slippage calculations in *SwapperLib._swapSafe()* and *BasePositionManager.checkSlippage()* have been acknowledged and it is accepted that these slippage checks can only be relied on as sanity checks.

Team response

The issues have been fixed by commit [6ea8779](#).

Mitigation review

Verified, the recommendations have been implemented.

TRST-H-2: Duplicate accounts in liquidation allow to liquidate beyond closeFactor

- **Category:** Logical issues
- **Source:** BorrowableCToken.sol, MarketManagerIsolated.sol
- **Status:** Fixed

Description

The *liquidateExact()* and *liquidate()* functions accept arrays of accounts to be liquidated without validating for duplicate entries. This allows liquidators to bypass the **closeFactor** restriction by including the same account multiple times in the liquidation call, effectively liquidating debt beyond designed limits.

```
function liquidateExact(
    uint256[] calldata debtAmounts,
    address[] calldata accounts,
    address collateralToken
) external nonReentrant {
    uint256 numAccounts = accounts.length;
    if (numAccounts != debtAmounts.length) {
        _revert(_INVALID_PARAMETER_SELECTOR);
    }

    _liquidate(
        debtAmounts,
        msg.sender,
        accounts,
        collateralToken,
        numAccounts,
        true
    );
}
```

The accounts are validated to be liquidatable by calling *MarketManagerIsolated.canLiquidate()* inside *BorrowableCToken._liquidate()*.

```
function _liquidate(
    uint256[] memory debtAmounts,
    address liquidator,
    address[] calldata accounts,
    address collateralToken,
    uint256 numAccounts,
    bool exactAmount
) internal {
    // Cannot have debt and collateral in the same token, so can revert
    // immediately if someone tries.
    if (collateralToken == address(this)) {
        _revert(_INVALID_PARAMETER_SELECTOR);
    }
}
```

```

// Accrue interest if needed.
_accrueIfNeeded();

IMarketManager.LiqResult memory result;

// Fails if liquidation not allowed, trying to repay too much debt
// will revert.
(result, debtAmounts) = marketManager.canLiquidate(
    debtAmounts,
    liquidator,
    accounts,
    IMarketManager.LiqAction({
        collateralToken: collateralToken,
        debtToken: address(this),
        numAccounts: numAccounts,
        liquidateExact: exactAmount,
        liquidatedShares: 0,
        debtRepaid: 0,
        badDebt: 0
    })
);

```

The `canLiquidate()` validation in `MarketManagerIsolated` processes each array entry independently without checking for duplicates:

```

for (uint256 i; i < action.numAccounts; ++i) {
    cachedAccount = accounts[i];

    // Do not let an account liquidate themselves.
    if (liquidator == cachedAccount) {
        _revert(_UNAUTHORIZED_SELECTOR);
    }

    (
        action.liquidatedShares,
        action.debtRepaid,
        action.badDebt
    ) = _canLiquidate(
        debtAmounts[i],
        cachedAccount,
        tData,
        aData,
        action.liquidateExact
    );

    // If the user is being liquidated update relevant values.
    if (action.liquidatedShares > 0) {
        result.debtRepaid += action.debtRepaid;
        result.liquidatedShares[i] = action.liquidatedShares;
        // ... process liquidation
    }
}

```

Each call to `_canLiquidate()` calculates the maximum liquidatable debt based on the `closeFactor`:


```

uint256 maxDebt = (aData.closeFactor * aData.debtBalance) / BPS;
// If they want to liquidate an exact amount, liquidate `debtAmount`,
// otherwise liquidate the maximum amount possible.
if (!liquidateExact) {
    debtAmount = maxDebt;
}

```

Therefore, an account to be liquidated can be used multiple times, allowing the **closeFactor** limit to be bypassed. As a result, users lose significantly more collateral than intended by the protocol design, creating additional profit incentives for liquidators. The POC is as follows:

```

// SPDX-License-Identifier: GPL-3.0
pragma solidity 0.8.28;

import { TestBaseBorrowableCToken } from "tests/market/token/BorrowableCToken/TestBaseBorrowableCToken.sol";
import { IMarketManager } from "contracts/interfaces/IMarketManager.sol";
import { FixedPointMathLib } from "contracts/libraries/external/FixedPointMathLib.sol";
import { WAD } from "contracts/libraries/ConstantsLib.sol";
import "forge-std/console2.sol";

// NOTE: Test also uses canLiquidate for extra accounting checks

contract LiquidateSameAccountTwiceTest is TestBaseBorrowableCToken {
    uint256 borrowableCTokenUnderlyingPrice = 1e18;

    function setUp() public override {
        super.setUp();

        _prepareLiquidationRethDrop();
    }

    // Test a single liquidation
    function test_liquidate_same_acc_twice() public {
        uint256 debtBeforeAccrual = borrowableCUSDC.debtBalance(user1);
        uint256 totalAssetsBeforeAccrual = borrowableCUSDC.totalAssets();

        borrowableCUSDC accrueIfNeeded();

        uint256 debtAfterAccrual = borrowableCUSDC.debtBalance(user1);
        uint256 totalAssetsAfterAccrual = borrowableCUSDC.totalAssets();

        assertGt(debtAfterAccrual, 1000e6);
        assertGt(debtAfterAccrual, debtBeforeAccrual);

        uint256 debtIncrease = debtAfterAccrual - debtBeforeAccrual;
        uint256 assetsIncrease = totalAssetsAfterAccrual - totalAssetsBeforeAccrual;
        assertEq(debtIncrease, assetsIncrease);

        address[] memory accounts = new address[](2);
        accounts[0] = user1;
        accounts[1] = user1;

        uint256[] memory debtAmounts = new uint256[](2);

        _prepareUSDC(user2, 1000e6);
        vm.startPrank(user2);
        usdc.approve(address(borrowableCUSDC), 1000e6);
    }
}

```

```

    debtAmounts[0] = 339807460;
    debtAmounts[1] = 339807460;
    borrowableCUSDC.liquidateExact(
        debtAmounts,
        accounts,
        address(strategyCBALRETH)
    );
    vm.stopPrank();
}

function _prepareLiquidationRethDrop() internal {
    address liquidityProvider = makeAddr("liquidityProvider");
    _prepareUSDC(liquidityProvider, 200000e6);
    _prepareBALRETH(liquidityProvider, 10e18);
    // Mint borrowable cUSDC.
    vm.startPrank(liquidityProvider);
    usdc.approve(address(borrowableCUSDC), 200000e6);
    borrowableCUSDC.deposit(200000e6, liquidityProvider);
    // Mint cBALETH.
    balRETH.approve(address(strategyCBALRETH), 10e18);
    strategyCBALRETH.deposit(10e18, liquidityProvider);
    vm.stopPrank();

    _prepareBALRETH(user1, 10 * _ONE);

    vm.startPrank(user1);
    balRETH.approve(address(strategyCBALRETH), 10 * _ONE);
    strategyCBALRETH.deposit(2 * _ONE, user1);
    strategyCBALRETH.postCollateral(14 * _ONE / 10 - 1);

    borrowableCUSDC.borrow(1000e6, user1);
    vm.stopPrank();

    // skip min hold period
    skip(20 minutes);
    _harvestAuraStrategyRewards(1 weeks);

    mockWethFeed.setMockAnswer(1000e8);
    mockRethFeed.setMockAnswer(1000e8);

    _prepareUSDC(user2, 250e6);
}

```

Recommended mitigation

It is recommended to add duplicate account validation in the liquidation functions.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-H-3: Unsafe Redstone default heartbeat

- **Category:** Oracle staleness issues

- **Source:** RedstoneCoreAdaptor.sol
- **Status:** Fixed

Description

The *RedstoneCoreAdaptor* sets the `config.heartbeat` to `DEFAULT_HEARTBEAT` if `type(uint256).max` is used in `addAsset()`:

```
config.heartbeat = uint16(heartbeat != type(uint256).max ?
    heartbeat : DEFAULT_HEARTBEAT);
```

The current implementation sets a 10-minute staleness threshold:

```
uint256 public constant DEFAULT_HEARTBEAT = 10 minutes;
```

Accordingly, `_getPrice()` accepts prices up to 10 minutes old, creating a significant staleness window that can be exploited:

```
function _getPrice(
    address asset,
    bool inUSD
) internal view override returns (PricingResult memory result) {
    // We can check if `asset` pricing in `inUSD` is supported by checking
    // `decimals` as we do not allow configuring `decimals` equal to 0.
    // If unsupported price in the other denomination and manually convert
    // in Oracle Manager.
    if (assetConfig[asset][inUSD].decimals == 0) {
        inUSD = !inUSD;
    }
    AssetConfig memory config = assetConfig[asset][inUSD];
    result.inUSD = inUSD;

    // Validate the price returned is not stale.
    uint256 timestampInSeconds = config.redstoneTimestamp / 1000;
    if (
        timestampInSeconds < block.timestamp &&
        block.timestamp - timestampInSeconds > config.heartbeat
    ) {
        result.hadError = true;
        return result;
    }
    result.price = config.price;
}
```

During periods of high volatility, adversaries can exploit the gap between stale oracle prices and current market prices to extract value through undercollateralized loans. In the event of flash crashes, which historically move prices 50%-60% in only 5 minutes, this threshold will organically lead the protocol to be undercollateralized.

Recommended mitigation

The issue can be addressed by setting a much smaller allowed staleness period and additionally operating a bot that updates prices in case the market price has still deviated too much despite the shorter staleness period.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented. The maximum staleness period is now 1 minute on Ethereum and may be configured lower, but not higher, on other chains.

TRST-H-4: Stale interest accrual enables bad debt creation and incorrect liquidations

- **Category:** Logical issues
- **Source:** BorrowableCToken.sol, MarketManagerIsolated.sol
- **Status:** Fixed

Description

The protocol uses outdated views of *totalAssets()* and debt balances when performing critical market operations, leading to incorrect health calculations that enable bad debt creation and improper liquidations. The root cause is that view functions and market operations don't accrue interest before returning values, creating discrepancies between actual and reported states.

BorrowableCToken.getSnapshot() returns debt balance without accruing interest:

```
function getSnapshot(
    address account
) external view override returns (AccountSnapshot memory result) {
    uint256 outstandingDebt = debtBalance(account);

    result.asset = address(this);
    result.decimals = decimals();
    result.isCollateral = outstandingDebt > 0 ? false : true;
    result.collateralPosted = collateralPosted[account];
    result.debtBalance = outstandingDebt;
}
```

Similarly, view functions don't account for interest accrual periods, returning outdated *totalAssets()*:

```
function totalAssets() public view nonReentrant override returns (
    uint256
) {
    return _getTotalAssets();
}
//...//
function _getTotalAssets() internal view virtual returns (uint256 result) {
    result = _totalAssets;
}
```

This issue arises when a user deposits collateral in **Token 1** and borrows **Token 2**. Because interest in **Token 2** does not accrue over time, health checks rely on an outdated and artificially low debt balance. As a result, the user can withdraw more collateral from **Token 1** than they should, creating bad debt. Since the protocol uses stale values of *totalAssets()* and

debt balances, this leads both to incorrect liquidations of healthy accounts and to unauthorized collateral withdrawals, ultimately resulting in bad debt.

The POC is as follows:

```
function test_debtbalance_update_audit1() public {
    mockUsdcFeed.setMockAnswer(1e8);
    mockDaiFeed.setMockAnswer(1e8);
    vm.startPrank(user);
    deal(address(usdc), user, 1000e6 + 1000);
    usdc.approve(address(borrowableCUSDC), 1000e6 + 1000);
    assertGt(borrowableCUSDC.deposit(1000e6 + 1000, user), 0);
    borrowableCUSDC.postCollateral(1000e6 + 1000);
    borrowableCDAI.borrow(700 ether, user);

    (uint256 collateral, uint256 maxDebt, uint256 debt) = marketManagerIsolated.statusOf(user);
    console2.log("### TEST - maxDebt=%d, debt=%d", maxDebt, debt);

    skip(1 weeks);

    mockUsdcFeed.setMockUpdatedAt(block.timestamp);
    mockDaiFeed.setMockUpdatedAt(block.timestamp);

    // @audit without this line, the test succeeds
    borrowableCDAI accrueIfNeeded();

    (collateral, maxDebt, debt) = marketManagerIsolated.statusOf(user);
    console2.log("### TEST - maxDebt=%d, debt=%d", maxDebt, debt);

    borrowableCUSDC.removeCollateral(1000);

    (collateral, maxDebt, debt) = marketManagerIsolated.statusOf(user);
    console2.log("### TEST - maxDebt=%d, debt=%d", maxDebt, debt);
    vm.stopPrank();
}
```

Recommended mitigation

Before performing any market interaction, the protocol must first update its accounting by accruing interest. In practice, this means both **cTokens** involved in the market need to be accrued so that *totalAssets()* and debt balances reflect the latest state.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-H-5: Liquidating healthy account with unhealthy account allows to remove all debts

- **Category:** Logical issues
- **Source:** MarketManagerIsolated.sol
- **Status:** Fixed

Description

Due to a logical issue in *MarketManagerIsolated._canLiquidate()*, an action that doesn't liquidate any shares continues with the liquidator-provided **debtAmount**. The conditional logic only updates accounting when **liquidatedShares > 0**:

```
if (action.liquidatedShares > 0) {
    result.debtRepaid += action.debtRepaid;
    result.liquidatedShares[i] = action.liquidatedShares;

    if (action.badDebt > 0) {
        result.badDebtRealized += action.badDebt;
        // Add the bad debt to debt to remove from the liquidated
        // account.
        action.debtRepaid += action.badDebt;
    }

    // If its an exact liquidation this will be a redundant setter
    // but anticipation is majority of liquidators will use
    // non-exact so checking for liquidateExact each time is a
    // waste.
    debtAmounts[i] = action.debtRepaid;
}
```

The attack works by including both a healthy account (which will have **liquidatedShares = 0** but keeps its **debtAmount**) and an unhealthy account (which provides non-zero **result.debtRepaid** to pass the validation). This allows reducing the debt of the healthy account by the provided **debtAmount** without seizing any collateral.

For healthy accounts the *_canLiquidate()* function returns early with **(0, 0, 0)**:

```
if (aData.lFactor == 0) {
    return (0, 0, 0);
}
```

This early return means **liquidatedShares = 0** for healthy accounts, but the conditional logic in *_canLiquidate()* only updates **debtAmounts[i]** when **liquidatedShares > 0**, allowing the original liquidator-provided **debtAmount** to persist and be used for debt reduction without any collateral seizure.

This creates a vector to steal protocol funds where an attacker can borrow assets, reduce their debt through this exploit without losing collateral, and repeat the process.

The POC is as follows:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity 0.8.28;

import { TestBaseBorrowableCToken } from "tests/market/token/BorrowableCToken/TestBaseBorrowableCToken.sol";
import { IMarketManager } from "contracts/interfaces/IMarketManager.sol";
import { FixedPointMathLib } from "contracts/libraries/external/FixedPointMathLib.sol";
import { WAD } from "contracts/libraries/ConstantsLib.sol";
import "forge-std/console2.sol";

// NOTE: Test also uses canLiquidate for extra accounting checks
```

```

contract LiquidateTwoTest is TestBaseBorrowableCToken {
    uint256 borrowableCTokenUnderlyingPrice = 1e18;

    function setUp() public override {
        super.setUp();

        _prepareLiquidationRethDrop();
    }

    // Test a single liquidation
    function test_liquidate_two_success() public {

        address[] memory accounts = new address[](2);
        accounts[0] = user1;
        accounts[1] = user3;

        uint256[] memory debtAmounts = new uint256[](2);

        _prepareUSDC(user2, 1000e6);
        vm.startPrank(user2);
        usdc.approve(address(borrowableCUSDC), 1000e6);

        // Hard liquidation, should have lost all collateral
        //assertEq(result.liquidatedShares[0], _ONE - 1, "Liquidated amount mismatch");
        uint256 user3DebtBalanceBeforeLiq = borrowableCUSDC.debtBalanceUpdated(user3);
        uint256 user3CollBeforeLiq = strategyCBALRETH.collateralPosted(user3);
        debtAmounts[0] = 858046862 / 2;
        debtAmounts[1] = 500e6;
        borrowableCUSDC.liquidateExact(
            debtAmounts,
            accounts,
            address(strategyCBALRETH)
        );
        vm.stopPrank();
        uint256 user3DebtBalanceAfterLiq = borrowableCUSDC.debtBalanceUpdated(user3);
        uint256 user3CollAfterLiq = strategyCBALRETH.collateralPosted(user3);
        console2.log("user3DebtBalanceBeforeLiq: ", user3DebtBalanceBeforeLiq);
        console2.log("user3DebtBalanceAfterLiq: ", user3DebtBalanceAfterLiq);
        console2.log("user3CollBeforeLiq: ", user3CollBeforeLiq);
        console2.log("user3CollAfterLiq: ", user3CollAfterLiq);

        skip(20 minutes);
        vm.startPrank(user3);
        strategyCBALRETH.withdraw(_ONE-1e16,user3,user3);
        vm.stopPrank();
    }

    function _prepareLiquidationRethDrop() internal {
        address liquidityProvider = makeAddr("liquidityProvider");
        _prepareUSDC(liquidityProvider, 200000e6);
        _prepareBALRETH(liquidityProvider, 10e18);
        // Mint borrowable cUSDC.
        vm.startPrank(liquidityProvider);
        usdc.approve(address(borrowableCUSDC), 200000e6);
        borrowableCUSDC.deposit(200000e6, liquidityProvider);
        // Mint cBALRETH.
        balRETH.approve(address(strategyCBALRETH), 10e18);
        strategyCBALRETH.deposit(10e18, liquidityProvider);
        vm.stopPrank();
    }

```

```
_prepareBALRETH(user1, _ONE);

vm.startPrank(user1);
balRETH.approve(address(strategyCBALRETH), _ONE);
strategyCBALRETH.deposit(_ONE, user1);
strategyCBALRETH.postCollateral(_ONE - 1);

borrowableCUSDC.borrow(1000e6, user1);
vm.stopPrank();

// skip min hold period
skip(20 minutes);
_harvestAuraStrategyRewards(1 weeks);

mockWethFeed.setMockAnswer(1000e8);
mockRethFeed.setMockAnswer(1000e8);

_prepareUSDC(user2, 250e6);

_prepareBALRETH(user3, _ONE);

vm.startPrank(user3);
balRETH.approve(address(strategyCBALRETH), _ONE);
strategyCBALRETH.deposit(_ONE, user3);
strategyCBALRETH.postCollateral(_ONE-1e16);

borrowableCUSDC.borrow(500e6, user3);
vm.stopPrank();
}
}
```

Recommended mitigation

Remove the conditional check if (**action.liquidatedShares > 0**) so that the debt accounting logic executes unconditionally. This ensures that when no shares are liquidated, the **debtAmounts[i]** is properly set to the actual debt that should be repaid (which would be zero when no liquidation occurs), maintaining consistency between debt reduction and collateral seizure.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-H-6: The `_checkTransfer()` function uses wrong account for `transferFrom()`

- **Category:** Logic issues
- **Source:** BaseCToken.sol
- **Status:** Fixed

Description

The health check in `BaseCToken._checkTransfer()` uses **msg.sender** instead of **owner** when calling `MarketManagerIsolated.canTransfer()`, which means the wrong account is checked for health in the case of `transferFrom()` when **msg.sender != owner**:


```
uint256 collateralRedeemed = marketManager.canTransfer(
    address(this),
    shares,
    msg.sender,
    balanceOf(owner),
    collateralOf,
    collateralOf > 0 ? true : false
);
```

The `_checkTransfer()` function is called from both `transfer()` and `transferFrom()`. However, the health check inside `_checkTransfer()` always uses `msg.sender` as the account parameter for `canTransfer()`, even when the shares are being transferred from the `owner` account in a `transferFrom()` call.

The health check can be fully bypassed by triggering the following case in `MarketManagerIsolated._canRedeemWithCollateralRemoval()`:

```
if (accountPositions[cToken][account] != 2) {
    bool[] memory emptyPositions;
    return (0, emptyPositions);
}
```

This allows transferring shares away from the sender of the transfer and creating bad debt. The shares can then be withdrawn from the receiver, effectively allowing borrowed funds to be stolen from the protocol. An attacker can use a healthy account as `msg.sender` to transfer collateral shares from an unhealthy owner account, bypassing the liquidity checks that should prevent such transfers.

Recommended mitigation

Use `owner` instead of `msg.sender` in the `canTransfer()` call.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-H-7: The `isCollateral` flag is true on first borrows which leads to mispricing in `OracleManager`

- **Category:** Pricing issues
- **Source:** `BorrowableCToken.sol`, `OracleManager.sol`
- **Status:** Fixed

Description

When a user makes their first borrow in a `BorrowableCToken` (i.e., their previous debt balance is zero), the `getSnapshot()` function incorrectly returns `isCollateral = true`:

```
function getSnapshot(
    address account
```

```
) external view virtual returns (AccountSnapshot memory result) {
    result.asset = address(this);
    result.decimals = decimals();
    // Can only be true for non-BorrowableCTokens.
    result.isCollateral = true;
    result.collateralPosted = collateralPosted[account];
    // result.debtBalance is 0 for non-BorrowableCTokens, no need to set.
}
```

This causes *OracleManager.getPricesForMarket()* to request the price for the **cToken** itself rather than for the cToken's **underlying asset**:

```
// If the asset is being used as collateral the users liquidity is
// priced in shares, if they're borrowing the outstanding debt is
// measured in assets (underlying).
(prices[i], errorCode) = getPrice(
    snapshots[i].isCollateral ? asset : cTokens[asset],
    true,
    snapshots[i].isCollateral
);
```

When **isCollateral = true**, the **asset** (cToken address) is passed to *getPrice()*, which then multiplies the underlying asset price by the **cToken**'s exchange rate:

```
if (cToken != address(0)) {
    price = (price * IToken(cToken).exchangeRate()) / WAD;
}
```

Most of the time when **exchangeRate > WAD**, this overestimates the debt value, making health checks more restrictive. However, if there has been bad debt causing the exchange rate to drop below **WAD**, this allows users to take on more debt than they can handle. This incorrect pricing is used in *MarketManagerIsolated._hypotheticalLiquidityOf()* for debt and collateral valuation, potentially leading to protocol insolvency. Even minor bad debt that causes the exchange rate to drop below **WAD** will trigger this vulnerability.

Recommended mitigation

The *getPricesForMarket()* function needs to capture the intention of how each asset will be used. For normal **cTokens** it's always as **collateral**, for *BorrowableCTokens* it's ambiguous. All uses of *_assetDataOf()*, which is the only caller of *getPricesForMarket()*, are correct, except for *_hypotheticalLiquidityOf()*, which is incorrect only when **action.borrowAssets > 0** and **outstandingDebt** of *BorrowableCToken* is **0**.

It is recommended to introduce a **forceBorrow** flag that *_hypotheticalLiquidityOf()* passes as **true** depending on its input. Then, in *getPricesForMarket()*, get the normal snapshot and override the **isCollateral** flag to **false** when **forceBorrow = true**.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the issue has been mitigated by checking for this specific case inside [LiquidityManagerIsolated._hypotheticalLiquidityOf\(\)](#) and adjusting the wrong snapshot accordingly.

TRST-H-8: Min/max price guards can override oracle prices

- **Category:** Access control issues
- **Source:** MarketManagerIsolated.sol
- **Status:** Fixed

Description

The protocol's price guard mechanism returns a successful max price when the upper guard is hit, rather than reverting. The price guard mechanism returns an invalid zero price when the lower guard is hit which bubbles up as a **BAD_SOURCE** error. This asymmetric behavior is a design choice. The lower guard acts as a circuit breaker, blocking all protocol operations that rely on the token's price. The upper guard aims to protect against inflated prices. The risk of capping a price is that the price might genuinely deviate to the upside, which e.g., could allow borrowers to take on excessive debt that has a higher market value than the internal valuation calculated by the protocol.

Recommended mitigation

If the desired behavior is to not respect a depegged value at all cost, consider reverting in *getPrice()* if a price outside the boundaries is found.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Issue has been addressed as recommended. The supplementary risk has been disclosed under TRST-SR-3.

Medium severity findings

TRST-M-1: The `redeemPaused` flag only applies to collateral removals

- **Category:** Access control issues
- **Source:** `MarketManagerIsolated.sol`
- **Status:** Fixed

Description

The `redeemPaused` flag is documented as controlling token redemptions:

```
/// @notice Admin function to set market-wide redemption status.
/// @dev Requires timelock authority if unpasing.
/// Emits an {ActionPaused} event.
/// @param state Whether the desired action is pausing or unpasing.
function setRedeemPaused(bool state) external {
    _checkAuthorizedPermissions(state);

    redeemPaused = state ? 2 : 1;
    emit ActionPaused("Redeem Paused", state);
}
```

However, it is only checked in `_canRedeem()`:

```
function _canRedeem(
    address cToken,
    uint256 shares,
    address account
) internal view returns (uint256, bool[] memory) {
    if (redeemPaused == 2) {
        revert MarketManager__Paused();
    }
}
```

The issue arises in `_canRedeemWithCollateralRemoval()` where the pause check is bypassed for uncollateralized redemptions:

```
if (collateralRedeemed > 0) {
    (
        uint256 positionClosureNeeded,
        bool[] memory positionsToClose
    ) = _canRedeem(cToken, collateralRedeemed, account);
    _closePositionsIfNeeded(
        positionClosureNeeded,
        account,
        positionsToClose
    );
} else {
    _checkIsListedToken(cToken);
    _checkTransfersAllowed(account);
}
```

When `collateralRedeemed == 0` (i.e., no collateral is withdrawn), the function skips `_canRedeem()` and only runs basic checks. As a result, the `redeemPaused` status is never enforced. This lets users bypass redemption restrictions and withdraw uncollateralized shares even when redemptions are supposed to be halted during emergencies or maintenance. In short, the paused state is not respected for redemptions involving zero collateral.

Recommended mitigation

It is recommended to apply the **redeemPaused** flag to all share redemptions, including uncollateralized ones.

Additionally, document that collateral removals that do not perform withdrawals are also affected by the **redeemPaused** flag, clarifying that while this may seem restrictive, it ensures consistent pause behavior across all collateral-related operations.

Team response

Fixed by commit [f10667c](#). Streamlined various `_canRedeem()` paths. The **redeemPaused** check is now always implemented inside `canRedeemRedeemWithCollateralRemoval()`. Transfer actions are no longer blocked by **redeemPaused**, only **transferPaused**.

Mitigation review

The recommendation to apply **redeemPaused** to all redemptions has been implemented, and removing the flag from transfers is an intended change.

TRST-M-2: Native token address inconsistency causes incorrect token equality checks

- **Category:** Logic issues
- **Source:** CommonLib.sol, SimpleZapper.sol, SwapperLib.sol
- **Status:** Fixed

Description

The protocol uses two different addresses to identify native tokens, causing simple equality checks between **inputToken** and **outputToken** to fail when one uses **address(0)** and the other uses **0xEeeeeEeeeEeEeeEeEeEEEEEEEEEEEEEEEEEE**.

The `CommonLib._isNative()` function recognizes both addresses as representing the same native token:

```
function _isNative(address token) internal pure returns (bool result) {
    result = token == address(0) ||
        token == 0xEeeeeEeeeEeEeeEeEeEEEEEEEEEEEEEEEEEE;
}
```

However, throughout the codebase, direct equality comparisons are used instead of the `_isNative()` function, leading to incorrect behavior when the same native token is represented by different addresses. This affects swap operations where the expected behavior is to skip the swap for tokens that are semantically the same. For example, in *SimpleZapper*:

```
if (swapAction.inputToken == swapAction.outputToken) {
    outAmount = swapAction.inputAmount;
} else {
    // Execute swap into cToken asset.
    outAmount = SwapperLib._swapUnsafe(centralRegistry, swapAction);
}
```

The *SwapperLib._swapUnsafe()* function does not check slippage properly in this case, and any future position manager and external calldata checker for external swapper logic could be affected by this inconsistency.

Recommended mitigation

It is recommended to create an additional function in *CommonLib* to define equality for token addresses with the special case that **address(0)** and **0xEeeeeEeeeEeEeeEeEeEEeeeeEEEEEEEEEEEE** return **true**.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-M-3: Accumulated debt rounding creates unrecoverable protocol value

- **Category:** Rounding issues
- **Source:** BorrowableCToken.sol
- **Status:** Fixed

Description

The protocol rounds debt calculations in favor of the protocol to prevent exchange rate manipulation, but the accumulated rounding errors from multiple users and transactions create unrecoverable value that is not earned by anyone.

The debt rounding mechanism is explicitly documented in the code:

```
// We round user debt in favor of the protocol to prevent exchange
// rate manipulation, as a result in some cases the last user cannot
// fully repay their debt.
if (marketOutstandingDebt < assets) {
    marketOutstandingDebt = 0;
} else {
    marketOutstandingDebt = uint240(marketOutstandingDebt - assets);
}
```

For high value tokens like **WBTC**, even a **1 wei** rounding difference is significant (currently around **\$0.001**, and potentially up to **\$0.1** based on protocol assumptions). When scaled across **10,000** users, each accruing debt **100 times**, these small discrepancies can add up to around **1 million wei** in rounding errors.

The unrecoverable portion can be expressed as **marketOutstandingDebt + underlyingBalance – totalAssets**. This value represents assets that remain in the protocol but cannot be withdrawn by users or collected by the treasury. As a result, the protocol suffers a permanent value leak, where funds are effectively locked and unusable. This reduces economic efficiency, and the impact grows as user activity and token value increase.

Recommended mitigation

It is recommended to implement a permissioned function to recover accumulated rounding losses rather than allowing them to remain permanently locked. The function must not add the excess assets to the total assets to avoid donation attacks where malicious actors could manipulate the exchange rate by directly transferring tokens to the contract.

Team response

Fixed by commit [f10667c](#). Introduced a *skim()* function to remove unrecoverable value.

Mitigation review

Verified, the recommendation has been implemented.

TRST-M-4: Liquidation rounding allows debt manipulation and breaks market debt invariant

- **Category:** Rounding issues
- **Source:** MarketManagerIsolated.sol
- **Status:** Fixed

Description

The liquidation calculation rounds down **liquidatedShares**, which can result in zero shares being liquidated while still removing debt from the liquidated account:

```
liquidatedShares = (debtAmount * debtToCollateral) / WAD_SQUARED;
```

This means there is up to **1 wei** less shares liquidated than what is needed to match **debtAmount**. Based on the protocol's token assumption of up to **\$0.1 per wei** of collateral underlying, and interest accrual which makes shares even more valuable, a rounding error of at least **\$0.1** can occur.

The core insight is that **liquidatedShares** can round down to **zero**, in which case a small **debtAmount** worth **> \$0.1** can be removed from the liquidated account's debt without any collateral being seized. Given that the protocol is expected to be deployed on chains where gas costs are lower than this amount, the owner of a liquidatable account can intentionally use another account as a liquidator to remove their debt at a cost lower than actually having shares liquidated.

The reason debt is removed from the account without paying for it is that **result.debtRepaid** is only incremented when **action.liquidatedShares > 0**:

```
if (action.liquidatedShares > 0) {
    result.debtRepaid += action.debtRepaid;
    result.liquidatedShares[i] = action.liquidatedShares;

    if (action.badDebt > 0) {
        result.badDebtRealized += action.badDebt;
        // Add the bad debt to debt to remove from the liquidated
        // account.
        action.debtRepaid += action.badDebt;
```

```

    }

    // If its an exact liquidation this will be a redundant setter
    // but anticipation is majority of liquidators will use
    // non-exact so checking for liquidateExact each time is a
    // waste.
    debtAmounts[i] = action.debtRepaid;
}

```

Thus the debt is not transferred from the liquidator. As a side effect, this breaks the invariant of **sum of user debts >= marketOutstandingDebt**. The attack can be repeated multiple times to amplify the difference, and the divergence can grow over time due to compounded interest. Additionally, bad debt realization is skipped when **liquidatedShares** equals **zero**, preventing proper accounting of bad debt.

Similar rounding issues exist in other liquidation calculations:

```

uint256 maxDebt = (aData.closeFactor * aData.debtBalance) / BPS;

```

```

uint256 sharesNeeded = _mulDiv(
    aData.debtBalance,
    debtToCollateral,
    WAD_SQUARED
);

```

```

badDebt = FixedPointMathLib.fullMulDiv(
    _mulDiv(debtAmount, sharesNeeded, sharesPosted),
    WAD_SQUARED - _mulDiv(WAD_SQUARED, sharesPosted, sharesNeeded),
    WAD_SQUARED
);

```

Recommended mitigation

It's recommended to **round up** the following liquidation calculations to make rounding directions consistent with the goal of rounding against the liquidated account and avoiding small bad debts:

- [MarketManagerIsolated.sol#L1440](#)
- [MarketManagerIsolated.sol#L1432](#)
- [MarketManagerIsolated.sol#L1470](#)
- [MarketManagerIsolated.sol - L1480-L1484](#)

The fourth calculation can be simplified as such:

```

// debtAmount * sharesNeeded / sharesPosted * (1 - sharesPosted / sharesNeeded)

// = debtAmount * (sharesNeeded / sharesPosted - 1)

// = debtAmount * sharesNeeded / sharesPosted - debtAmount

badDebt = FixedPointMathLib.fullMulDivUp(debtAmount, sharesNeeded, sharesPosted) - debtAmount;

```

Part of the recommendation overlaps with TRST-H-5, which is to remove the **if (action.liquidatedShares > 0)** condition to ensure debt accounting remains consistent even when no shares are liquidated.

Even though the adjusted rounding direction presents a solution for liquidatable accounts exploiting the protocol, the balance of power is now shifted towards liquidators. They benefit from the opposite rounding direction and are able to liquidate small debts and receive outsized amounts of collateral. The fundamental limitation is the atomic nature of 1 wei collateral shares, and the protocol should not be deployed with a setup where the benefit of 1 wei is greater than the cost of 1 transaction. To mitigate the impact of malicious liquidators, a minimum liquidation debt amount can be required when the debt balance of the account is sufficient.

Team response

Fixed by commit [f10667c](#).

Mitigation review

The **if (action.liquidatedShares > 0)** condition has been removed as recommended. For the rounding directions it has been decided that liquidated shares should be rounded down and debt amounts should be rounded up, against the liquidator and in favor of users and the protocol. Bad debt should be rounded up to mitigate the risk of small bad debts that can't be liquidated. The intended rounding policy is followed consistently except for one instance where [sharesNeeded](#) is rounded down but should be rounded up.

Team response

Fixed by commit [6ea8779](#).

Mitigation review

Verified, the recommendation has been implemented and **sharesNeeded** is rounded up.

TRST-M-5: Missing interest rate accrual before interest rate parameter update

- **Category:** Logical issues
- **Source:** DynamicIRM.sol
- **Status:** Fixed

Description

The *updateDynamicIRM()* function fails to accrue interest in the linked **cToken** before changing interest rate parameters, causing new parameters to be applied retroactively. The function directly updates interest rate model parameters without ensuring pending interest is calculated using the previous parameters:

```
function updateDynamicIRM(  
    uint256 baseRatePerYear,  
    uint256 vertexRatePerYear,  
    uint256 vertexStart,  
    uint256 adjustmentVelocity,  
    uint256 decayPerAdjustment,  
    uint256 vertexMultiplierMax,  
    bool vertexReset  
) external {  
    _checkMarketPermissions();  
  
    _updateDynamicIRM(  
        baseRatePerYear,
```

```
vertexRatePerYear,  
vertexStart,  
adjustmentVelocity,  
decayPerAdjustment,  
vertexMultiplierMax,  
vertexReset  
);  
}
```

The *DynamicIRM* contract maintains a reference to its linked **cToken** through the **ratesConfig.linkedToken** field:

```
function linkedToken() external view returns(address result) {  
    result = ratesConfig.linkedToken;  
}
```

When interest rate parameters are updated without prior accrual, the new rates are applied to calculate interest for the entire period since the last accrual, including time periods that should have used the old parameters. This creates a scenario where rate changes can be applied retroactively, potentially causing significant discrepancies in interest calculations.

The worst-case impact occurs when rates increase significantly (**R1** to **R2** where **R2 > R1**) after a period without accrual.

Users who should not be liquidatable under the old rate **R1** may suddenly become liquidatable when interest accrues using the higher rate **R2** for the entire period, potentially crossing into bad debt territory and causing protocol losses.

Recommended mitigation

It's recommended to accrue the interest in the linked **cToken** before changing interest rate.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-M-6: Changing collateral into borrow position is not possible due to 1 wei rounding difference

- **Category:** Rounding issues
- **Source:** MarketManagerIsolated.sol, LiquidityManagerIsolated.sol
- **Status:** Fixed

Description

Users cannot remove all their collateral due to a rounding inconsistency in the `_hypotheticalLiquidityOf()` function that calculates **maxDebt** and **newDebt** differently. The protocol rounds down **maxDebt** but rounds up **newDebt**, causing the collateral being removed to appear more valuable than the collateral available for removal, leading to an artificial liquidity deficit of **1 wei**.

When calculating **maxDebt** from existing collateral, the function uses **increasesCollateral = true**, resulting in rounding down:

```
maxDebt += _collateralValue(
    snap.collateralPosted,
    prices[i],
    10 ** snap.decimals,
    _tokenConfig[snap.asset].collRatio,
    true
);
```

However, when calculating the hypothetical debt impact of collateral removal, it uses **increasesCollateral = false**, resulting in **rounding up**:

```
newDebt += _assetValue(
    snap.debtBalance,
    prices[i],
    10 ** snap.decimals,
    false
);
```

This issue causes collateral removal to appear as though it reduces borrowing capacity by more than the collateral's actual value. As a result, the health check fails even when a user tries to fully remove their deposited collateral.

This breaks a core expectation: a user who deposits **X** collateral should be able to withdraw **X**. It becomes especially problematic when users want to switch from a collateral position to a borrow position, since they must set their collateral balance to zero before borrowing. The impact includes broken functionality and temporary DOS until users realize they need to withdraw 1 wei less than they should be able to. Even so, they won't be able to take on a borrow position.

The POC is as follows:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity 0.8.28;

import { TestBaseBorrowableCToken } from "../TestBaseBorrowableCToken.sol";
import { BorrowableCToken } from "contracts/market/token/BorrowableCToken.sol";
import { MarketManagerIsolated } from "contracts/market/isolated/MarketManagerIsolated.sol";

contract RemoveCollateralOneWeiTest is TestBaseBorrowableCToken {
    function test_cant_remove_all_collateral_due_to_rounding() public {
        borrowableCUSDC.deposit(200e6, address(this));
        strategyCBALRETH.postCollateral(1e18 - 1);

        skip(20 minutes);

        strategyCBALRETH.removeCollateral(1e18 - 1);
    }
}
```

Recommended mitigation

It's recommended to refactor the token evaluation logic to avoid calculating **debt** and **collateral** twice for the same asset. Instead, subtract them in **asset** units and run through the evaluation, which would zero out the calculation as intended.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-M-7: VaultAggregator doesn't support decimal offset

- **Category:** Decimals issues
- **Source:** VaultAggregator.sol, BaseWrappedAggregator.sol
- **Status:** Fixed

Description

The *VaultAggregator* incorrectly assumes that vault shares and underlying assets have the same decimal precision, but common ERC4626 implementations like **OpenZeppelin** and **Solady** allow configurable decimal offsets that break this assumption.

The exchange rate calculation in *getExchangeRate()* uses **convertToAssets(WAD)** and expects the result to be in **WAD** format:

```
function getExchangeRate() public view virtual override returns (
    uint256 result
){
    // Return exchange rate in `WAD` format directly.
    // We can use ICToken since its an ERC4626 vault itself.
    result = ICToken(vault).convertToAssets(WAD);
}
```

This result is then used in *BaseWrappedAggregator.latestRoundData()* where it's multiplied with the underlying asset price and divided by **WAD**:

```
answer = (answer * _toInt256(getExchangeRate())) / _toInt256(WAD);
```

The multiplication and division by **WAD** only cancel out correctly when vault shares and underlying assets have the same decimal precision. However, when there's a decimal offset, this calculation produces incorrect results.

For example, if vault shares have **19 decimals** and the underlying asset has **18 decimals (offset=1)**, then for **WAD** shares with a 1:1 exchange rate, **convertToAssets(WAD)** returns **WAD/10 = 1e17** assets instead of the expected **WAD**. This causes the final answer to be divided by **10**, producing a price that's **10x** lower than the correct value.

Recommended mitigation

The correct calculation is **answer * convertToAssets(10^(shares decimals) / 10^(asset decimals))**

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-M-8: DynamicIRM doesn't cap the vertexMultiplier when vertexMultiplierMax is updated

- **Category:** Logical issues
- **Source:** DynamicIRM.sol
- **Status:** Fixed

Description

The *DynamicIRM* contains flawed logic when **vertexMultiplierMax** is updated to a lower value while **vertexReset=false**, leading to undefined behavior.

When updating the configuration, **vertexMultiplierMax** and **vertexReset** are independent variables that can be provided in any combination:

```
function updateDynamicIRM(
    uint256 baseRatePerYear,
    uint256 vertexRatePerYear,
    uint256 vertexStart,
    uint256 adjustmentVelocity,
    uint256 decayPerAdjustment,
    uint256 vertexMultiplierMax,
    bool vertexReset
) external {
    _checkMarketPermissions();

    _updateDynamicIRM(
        baseRatePerYear,
        vertexRatePerYear,
        vertexStart,
        adjustmentVelocity,
        decayPerAdjustment,
        vertexMultiplierMax,
        vertexReset
    );
}
```

The problematic scenario occurs when **vertexReset=false** and **vertexMultiplierMax** is updated to a value lower than the current **vertexMultiplier**. In *_updateDynamicIRM()*, the vertex multiplier is only reset when **vertexReset=true**:

```
vertexMultiplier = vertexReset ? WAD : vertexMultiplier;
```

This enables a situation where the current **vertexMultiplier** exceeds the new **vertexMultiplierMax**, and the multiplier update logic behaves inconsistently. The early return case for low utilization does not clamp the multiplier:

```
return newMultiplier < WAD ? WAD : newMultiplier;
```

While the full calculation path does apply clamping:

```
// Make sure `newMultiplier` is not above `vertexMultiplierMax`.
newMultiplier = newMultiplier < c.vertexMultiplierMax
? newMultiplier
: c.vertexMultiplierMax;
```

This inconsistency breaks a key expectation that **multiplier1** <= **multiplier2** for **util1** < **util2**.

Recommended mitigation

It's recommended to clamp the **vertexMultiplier** to the new **vertexMultiplierMax** in **updateDynamicIRM()** even when **vertexReset=false**.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-M-9: Base reserve is ignored in the interest rate calculation

- **Category:** Logical issues
- **Source:** DynamicIRM.sol, BorrowableCToken.sol
- **Status:** Fixed

Description

The **utilizationRate()** function in the IRM counts the entire balance as **assetsHeld**, even though the **BASE_UNDERLYING_RESERVE** of **77,777 wei** cannot be borrowed. This skews the calculation because it treats funds that are unavailable as if they could be lent out. As a result, the market can never reach true 100% utilization, and the reported utilization rate may differ significantly from the real value. This undermines the accuracy of the interest rate model and leads to distorted outcomes.

```
/// @notice Calculates the borrow utilization rate of the market.
/// @param assetsHeld The amount of underlying assets held in the pool.
/// @param debt The amount of outstanding debt in the pool.
/// @return r The utilization rate between [0, WAD].
function utilizationRate(
    uint256 assetsHeld,
    uint256 debt
) public pure returns (uint256 r) {
    // Utilization rate is 0 when there are no outstanding debt.
    r = debt == 0 ? 0 : _mulDiv(debt, WAD, assetsHeld + debt);
}
```

In the WBTC case, if all other deposits totaled **77,777 wei** and they were all borrowed, the utilization rate would show **50%** when it should logically be **100%**. The market clearly shows interest in borrowing more WBTC, but the utilization rate doesn't reflect this demand. Since the rate can't exceed **50%**, there would be no increase in borrow rates to motivate additional WBTC deposits to fill the demand, resulting in an inefficient market that fails to accrue TVL.

Additionally, while this is not a security issue by itself, if the protocol wants to support tokens with larger values than WBTC, the reserve could become too significant an amount to burn. The protocol's assumption of at most **0.1\$ per wei** would make this reserve worth **\$7,777**.

Recommended mitigation

It's recommended to reduce **assetsHeld** by **BASE_UNDERLYING_RESERVE** in the utilization calculation. The reserve is meant to protect from share manipulation and shouldn't interfere with interest rate calculations.

Team response

Fixed by commit [f10667c](#). Added base reserve to interest rate calculation

Mitigation review

Verified, the recommendation has been implemented.

TRST-M-10: `BasePositionManager.onRedeem()` doesn't support swapping through native assets

- **Category:** Logical issues
- **Source:** `BasePositionManager.sol`
- **Status:** Fixed

Description

The `BasePositionManager.onRedeem()` function takes a **DeleverageAction** struct containing multiple **swapActions** and loops through all output tokens to send remaining balances to the owner. The `balanceOf()` call and transfer operations do not work for native tokens, causing transaction reverts when one of the output tokens is the native token.

```
// Transfer remaining swap dust back to the user.
if (action.swapActions.length > 0) {
    for (uint256 i; i < action.swapActions.length; ++i) {
        remaining = IERC20(action.swapActions[i].outputToken)
            .balanceOf(address(this));
        if (remaining > 0) {
            SafeTransferLib.safeTransfer(
                action.swapActions[i].outputToken,
                owner,
                remaining
            );
        }
    }
}
```

The impact is DoS in future implementations when multiple swap actions are supported and one of the output tokens is native token. `BasePositionManager` is **abstract** and `SimplePositionManager` does not allow for more than one swap action, so the issue would only surface in future implementations.

There's an unused `_transferToRecipient()` function that wraps native tokens, but it only solves the problem partially since the `balanceOf()` call would still fail.

```
function _transferToRecipient(
```

```
    address token,
    address recipient,
    uint256 amount
) internal {
    // If the token to refund is the chains' native gas token we wrap
    // then transfer it to prevent callback attack vectors.
    if (CommonLib._isNative(token)) {
        IWETH(wrappedNative).deposit{ value: amount }();
        token = wrappedNative;
    }

    SafeTransferLib.safeTransfer(token, recipient, amount);
}
```

Additionally, the pattern of supporting multiple swap actions is unsafe as each input token and output token must be validated to prevent callbacks.

Recommended mitigation

It's recommended to make the refund logic compatible with native tokens by using *CommonLib._balanceOf()* instead of *ERC20.balanceOf()* and wrapping native tokens before the transfer. Furthermore, consider the need for validating each token against a whitelist to prevent callbacks.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented. Callbacks from missing token validation have been acknowledged as part of TRST-H-8.

TRST-M-11: Multiplying two token amounts should use full precision

- **Category:** Overflow issues
- **Source:** BorrowableCToken.sol, MarketManagerIsolated.sol
- **Status:** Fixed

Description

According to the protocol's token and value range assumptions, some calculations can overflow due to multiplying two token amounts that can each reach up to **type(uint136).max**. When two such variables are multiplied, the necessary range grows to **136x2=272 bits**, which exceeds the **256 bit** limit and causes overflows.

One instance is the protocol fee calculation in *BorrowableCToken*:

```
uint256 protocolFeeShares = _mulDiv(
    protocolFee,
    totalSupply(),
    cachedTa + assetsToVest - protocolFee
);
```

An overflow here is highly unlikely given that protocol fee is only a percentage of newly accrued assets.

Another instance is in the liquidation calculation in *MarketManagerIsolated._canLiquidate()*:

```
debtAmount = FixedPointMathLib.mulDivUp(
    debtAmount,
    sharesPosted,
    liquidatedShares
);
```

This line is conditional on providing a **debtAmount** too large to liquidate due to insufficient **sharesPosted**. Both **debtAmount** and **sharesPosted** can be as high as **type(uint136).max**, so their product doesn't fit into **uint256**. A liquidator can circumvent this revert by decreasing **debtAmount**.

The last instance is in the bad debt calculation in *MarketManagerIsolated._canLiquidate()*:

```
badDebt = FixedPointMathLib.fullMulDiv(
    _mulDiv(debtAmount, sharesNeeded, sharesPosted),
    WAD_SQUARED - _mulDiv(WAD_SQUARED, sharesPosted, sharesNeeded),
    WAD_SQUARED
);
```

In this instance, a liquidator cannot vary their inputs to avoid the overflow, making it a DoS condition. The inner **_mulDiv(debtAmount, sharesNeeded, sharesPosted)** calculation suffers from the same overflow issue as the previous instances.

Recommended mitigation

The first and second instance should use *FixedPointMathLib.fullMulDivUp()*. In the third instance, the calculation should be refactored, as described in TRST-M-4.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-M-12: Single feed is more permissive than dual-feed

- **Category:** Oracle issues
- **Source:** OracleManager.sol
- **Status:** Acknowledged

Description

The *_getPriceDualFeed()* function has inconsistent error code behavior that breaks the rule that worse problems should result in worse error codes.

When there's only one oracle source, it always returns **CAUTION** for any price. But with two sources, it can return **BAD_SOURCE** if the prices are too different from each other. This creates an anomaly where the system first returns **BAD_SOURCE** because prices deviate too much but then switches to **CAUTION** just because one feed stops working.

Recommended mitigation

It's recommended to consider returning **BAD_SOURCE** as soon as one of the dual feeds has an error to avoid the **BAD_SOURCE** → **CAUTION** transition.

Team response

It is by design that a dual oracle setup is more trusted than a single oracle setup.

TRST-M-13: DAO retains market permissions during timelock / emergency council transfer

- **Category:** Access control issues
- **Source:** CentralRegistry.sol
- **Status:** Fixed

Description

When transferring timelock or emergency council permissions, if the previous address is the same as the DAO address, market permissions are not removed from the old address. This creates a privilege escalation since the DAO should only be able to access market permissions through the timelock, not directly.

There are two instances of this issue. The first one is in *transferTimelockPermissions()*:

```
if (previousTimelock != daoAddress) {  
  delete hasDaoPermissions[previousTimelock];  
  delete hasMarketPermissions[previousTimelock];  
  emit PermissionsUpdated("Market", previousTimelock, false);  
}
```

The second one is in *transferEmergencyCouncil()*:

```
if (previousEmergencyCouncil != daoAddress) {  
  delete hasDaoPermissions[previousEmergencyCouncil];  
  delete hasMarketPermissions[previousEmergencyCouncil];  
  emit PermissionsUpdated(  
    "Market",  
    previousEmergencyCouncil,  
    false  
  );  
}
```

Recommended mitigation

It's recommended to move the deletion of **hasMarketPermissions** one layer up into the outer if-block for both functions.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-M-14: DAOTimelock doesn't consider overlap in Emergency Council and DAO address

- **Category:** Access control issues
- **Source:** DAOTimelock.sol
- **Status:** Fixed

Description

In *CentralRegistry*, when the DAO address or emergency council address is changed, there is logic to not revoke permissions if the two roles were held by the same address:

```

if (previousDaoAddress != timelock) {
    delete hasDaoPermissions[previousDaoAddress];
}
//..//
if (previousTimelock != daoAddress) {
    delete hasDaoPermissions[previousTimelock];
    delete hasMarketPermissions[previousTimelock];
    emit PermissionsUpdated("Market", previousTimelock, false);
}

```

This logic is missing from *DAOTimelock.updateRoles()*:

```

function updateRoles() external {
    address registryDaoAddress = centralRegistry.daoAddress();
    address timelockDaoAddress = _DAO_ADDRESS;

    if (registryDaoAddress != timelockDaoAddress) {
        _revokeRole(PROPOSER_ROLE, timelockDaoAddress);
        _revokeRole(EXECUTOR_ROLE, timelockDaoAddress);
        _revokeRole(CANCELLER_ROLE, timelockDaoAddress);

        _grantRole(PROPOSER_ROLE, registryDaoAddress);
        _grantRole(EXECUTOR_ROLE, registryDaoAddress);
        _grantRole(CANCELLER_ROLE, registryDaoAddress);
        _DAO_ADDRESS = registryDaoAddress;
        _grantRole(PROPOSER_ROLE, registryDaoAddress);
        _grantRole(EXECUTOR_ROLE, registryDaoAddress);
        _grantRole(CANCELLER_ROLE, registryDaoAddress);
        _DAO_ADDRESS = registryDaoAddress;
    }

    address registryEC = centralRegistry.emergencyCouncil();
    address timelockEC = _EMERGENCY_COUNCIL;
    if (registryEC != timelockEC) {
        _revokeRole(CANCELLER_ROLE, timelockEC);

        _grantRole(CANCELLER_ROLE, registryEC);
        _EMERGENCY_COUNCIL = registryEC;
    }
}

```

Accordingly, when the DAO address changes, it's possible to end up with **CANCELLER_ROLE** revoked from the emergency council, and when the emergency council changes, the **CANCELLER_ROLE** can be revoked from the DAO. This breaks an important security line by removing cancellation abilities from critical roles.

Recommended mitigation

It's recommended to add the missing overlap checks to prevent revoking roles from addresses that still need them.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-M-15: Rounding of marketDebtIndex breaks invariant

- **Category:** Rounding issues
- **Source:** BorrowableCToken.sol
- **Status:** Fixed

Description

The **_marketDebtIndex** is rounded down during interest accrual, which breaks the critical invariant that **sum of users' debt >= marketOutstandingDebt**. This can lead to a situation where, after all users have repaid their debt, there are slightly insufficient assets for everyone to redeem their shares.

```
marketDebtIndex =  
    _mulDiv(assetsToVest, marketDebtIndex, outstandingDebt)  
    + marketDebtIndex;
```

The `_mulDiv()` function rounds down and causes the **marketDebtIndex** to be slightly lower than it should be. Since individual user debt calculations use this index, borrowers end up paying slightly less debt than the protocol expects. The protocol expects that the entire **assetsToVest** has accrued to borrowers and updates **marketOutstandingDebt** accordingly:

```
// Update marketOutstandingDebt invariant with vested assets.  
marketOutstandingDebt = uint240(outstandingDebt + assetsToVest);
```

The impact is theoretical protocol insolvency and breaking the important debt invariant.

The POC is as follows:

```
// SPDX-License-Identifier: GPL-3.0  
pragma solidity 0.8.28;  
  
import { SwapperLib } from "contracts/libraries/SwapperLib.sol";  
import { SimplePositionManager } from "contracts/market/position-management/SimplePositionManager.sol";  
import { SimpleCToken } from "contracts/market/token/SimpleCToken.sol";  
import { MockCalldataChecker } from "contracts/mocks/MockCalldataChecker.sol";  
import { ICentralRegistry } from "contracts/interfaces/ICentralRegistry.sol";  
import { IBorrowableCToken } from "contracts/interfaces/IBorrowableCToken.sol";  
import { ICToken, AccountSnapshot } from "contracts/interfaces/ICToken.sol";  
import { IERC20 } from "contracts/interfaces/IERC20.sol";  
import { TestBaseMarketIsolated } from "tests/market/TestBaseMarketIsolated.sol";  
import { console2 } from "forge-std/console2.sol";
```

```
contract TestSimplePositionManager is TestBaseMarketIsolated {
    address public owner;
    address public user;

    SimplePositionManager public positionManager;

    receive() external payable {}

    fallback() external payable {}

    function setUp() public override {
        super.setUp();

        owner = address(this);
        user = user4;

        centralRegistry.setExternalCalldataChecker(
            _UNISWAP_V2_ROUTER,
            address(new MockCalldataChecker(_UNISWAP_V2_ROUTER))
        );

        // Setup borrowable cDAI.
        {
            // Add cToken support on Oracle Manager.

            _prepareDAI(owner, 200000e18);
            dai.approve(address(borrowableCDAI), 200000e18);
        }

        // Setup borrowable cUSDC.
        {
            _deployBorrowableCUSDC();
            _prepareUSDC(owner, 100e6);
            usdc.approve(address(borrowableCUSDC), 100e6);
            oracleManager.addCTokenSupport(address(borrowableCUSDC));
        }

        marketManagerIsolated.listTokens(address(borrowableCUSDC), address(borrowableCDAI));

        _setCTokenConfigBasic(address(borrowableCUSDC), 100_000e18, 100_000e18);
        _setCTokenConfigBasic(address(borrowableCDAI), 100_000e18, 100_000e18);

        positionManager = new SimplePositionManager(
            ICentralRegistry(address(centralRegistry)),
            address(marketManagerIsolated),
            _WETH_ADDRESS
        );

        marketManagerIsolated.addPositionManager(address(positionManager));

        _provideEnoughLiquidityForLeverage();
    }

    function _provideEnoughLiquidityForLeverage() internal {
        address liquidityProvider = makeAddr("liquidityProvider");

        _prepareUSDC(liquidityProvider, 100e6);
        _prepareDAI(liquidityProvider, 20000000e18 * 1000);

        vm.startPrank(liquidityProvider);
    }
}
```

```

// Mint borrowable cDAI.
dai.approve(address(borrowableCDAI), 20000000 ether * 1000);
borrowableCDAI.deposit(20000000 ether * 1000, liquidityProvider);

// Mint borrowable cUSDC.
usdc.approve(address(borrowableCUSDC), 100e6);
borrowableCUSDC.deposit(100e6, liquidityProvider);

vm.stopPrank();
}

function test_borrow_cdai_audit2() public {
    borrowableCDAI.setInterestFee(0);
    address liquidityProvider = makeAddr("liquidityProvider");
    /*
    vm.startPrank(user1);
    deal(address(dai), user1, 1000e18);
    dai.approve(address(borrowableCDAI), 1000e18);
    assertGt(borrowableCDAI.deposit(1000e18, user1), 0);
    vm.stopPrank();
    */

    vm.startPrank(user2);
    deal(address(usdc), user2, 1000e6);
    usdc.approve(address(borrowableCUSDC), 1000e6);
    assertGt(borrowableCUSDC.deposit(1000e6, user2), 0);
    borrowableCUSDC.postCollateral(1000e6);
    borrowableCDAI.borrow(100 ether - 1, user2);
    vm.stopPrank();

    uint256 totalAssetsAvailable;
    console2.log(totalAssetsAvailable = borrowableCDAI.debtBalance(user2) + dai.balanceOf(address(borrowableCDAI)));
    uint256 assetsOfLiquidityProvider;
    console2.log(assetsOfLiquidityProvider = borrowableCDAI.convertToAssets(borrowableCDAI.balanceOf(liquidityProvider)));
    console2.log(borrowableCDAI.marketOutstandingDebt());
    console2.log("Diff totalAssetsAvailable - assetsOfLiquidityProvider=%d", totalAssetsAvailable - assetsOfLiquidityProvider);
    console2.log("Diff debtBalance - marketOutstandingDebt=%d", borrowableCDAI.debtBalance(user2) -
    borrowableCDAI.marketOutstandingDebt());

    for (uint i=0;i<10000;i++) {
        skip(10 minutes);
        borrowableCDAI.accrueIfNeeded();
        console2.log(totalAssetsAvailable = borrowableCDAI.debtBalance(user2) + dai.balanceOf(address(borrowableCDAI)));
        console2.log(assetsOfLiquidityProvider = borrowableCDAI.convertToAssets(borrowableCDAI.balanceOf(liquidityProvider)));
        console2.log(borrowableCDAI.marketOutstandingDebt());
        console2.log(borrowableCDAI.debtBalance(user2));
        //console2.log("Diff debtBalance - marketOutstandingDebt=%d", borrowableCDAI.debtBalance(user2) -
        borrowableCDAI.marketOutstandingDebt());
        console2.log("Diff marketOutstandingDebt-debtBalance=%d", borrowableCDAI.marketOutstandingDebt() -
        borrowableCDAI.debtBalance(user2));
        if (totalAssetsAvailable < assetsOfLiquidityProvider) {
            console2.log("Returning at i=%d", i);
            console2.log("Diff assetsOfLiquidityProvider - totalAssetsAvailable=%d", assetsOfLiquidityProvider - totalAssetsAvailable);
            break;
        } else {
            console2.log("Diff totalAssetsAvailable - assetsOfLiquidityProvider=%d", totalAssetsAvailable - assetsOfLiquidityProvider);
        }
    }
}

```

Recommended mitigation

It's recommended to use the `FixedPointMathLib.mulDivUp()` for the `marketDebtIndex` calculation to round in favor of the protocol.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-M-16: Memory corruption in Bytes32Helper can return wrong oracle prices

- **Category:** Memory safety issues
- **Source:** Bytes32Helper.sol
- **Status:** Fixed

Description

According to the [Solidity documentation](#), there are operations in Solidity that need temporary memory areas larger than **64 bytes** and will not fit into the scratch space. These operations are placed where the free memory pointer points to, but given their short lifetime, the pointer is not updated.

Critically, *"The memory may or may not be zeroed out. Because of this, one should not expect the free memory to point to zeroed out memory."*

The `Bytes32Helper.toBytes32()` function can load corrupted memory when data has a size less than **32 bytes**, as Solidity does not guarantee that the remaining bytes are zero:

```
function toBytes32(string memory data) public pure returns (bytes32 r) {
    /// @solidity memory-safe-assembly
    assembly {
        r := mload(add(data, 32))
    }
}
```

This function directly loads **32 bytes** from memory starting at `add(data, 32)` without checking if the string data is actually **32 bytes** long. If data is shorter than **32 bytes**, the function will read beyond the string's actual content into uninitialized or previously used memory, which may contain arbitrary values.

The `RedstoneCoreAdaptor` uses this function to create symbol hashes for price feeds:

```
if (inUSD) {
    // Redstone Core does not append anything at the end of USD
    // denominated feeds, so we use toBytes32 here.
    symbolHash = Bytes32Helper._toBytes32(asset);
} else {
    // Redstone Core appends "/" + the native chain token's symbol at
    // the end of native denominated feeds, so we can compute the
    // output with `toBytes32Symbol`.
    symbolHash = Bytes32Helper._toBytes32Symbol(asset, _nativeSymbol);
}
```

If a token symbol is shorter than **32 bytes**, the corrupted memory can result in incorrect symbol hashes. This leads to wrong price data being retrieved when new prices are written.

```
uint256 price = getOracleNumericValueFromTxMsg(config.symbolHash);
```

The worst-case impact is an asset like **ETH** storing the price for **ETHX** as the result of the wrong **symbolHash**. This directly impacts the *RedstoneCoreAdaptor* returning wrong prices, which can lead to significant protocol failures including incorrect liquidations, borrowing limits, and collateral valuations.

While the likelihood is very low (depending on Curvance using a new compiler version that changes the existing behavior), the impact is high as incorrect oracle prices can cause severe protocol malfunctions, and it could be very hard to detect when a new compiler exposes the attack.

Recommended mitigation

It's recommended to properly handle strings shorter than **32 bytes** by ensuring the remaining bytes are zeroed.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-M-17: Debt repayment can be grieved in BasePositionManager and BaseZapper

- **Category:** Griefing issues
- **Source:** BasePositionManager.sol, BaseZapper.sol
- **Status:** Fixed

Description

BasePositionManager.onRedeem() takes the amount of debt to repay from the **action.repayAssets** parameter that is supplied by the user:

```
uint256 repayAssets = action.repayAssets;
```

In *BorrowableCToken.repayFor()*, which can be called by anyone for any account, a repayment greater than the account's debt reverts:

```
// Validate repayment amount is not excessive.  
if (assets > debtOf) {  
    _revert(_INVALID_PARAMETER_SELECTOR);  
}
```

It is possible to repay debt fully, indicated by **assets=0**:

```
// If assets == 0, repay max; assets = debtOf.  
assets = assets == 0 ? debtOf : assets;
```



```
_checkZeroAmount/assets);
```

However, *BasePositionManager* does not support that flow. It is common that a user would like to deleverage and repay all debt, and this is grievable with even just **1 wei** repayment by an attacker.

The same issue applies to *BaseZapper._repayDebt()*:

```
function _repayDebt(
    address borrowableCToken,
    address debtAsset,
    uint256 assetsHeld,
    uint256 repayAssets,
    address receiver
) internal returns (uint256) {
    // Revert if the swap experienced too much slippage.
    if (assetsHeld < repayAssets) {
        revert BaseZapper__InsufficientToRepay();
    }

    // Approve `debtAsset` transfer to cToken contract, if needed.
    SwapperLib._approveIfNeeded(debtAsset, borrowableCToken, repayAssets);

    // Execute repayment of outstanding debt.
    IBorrowableCToken(borrowableCToken).repayFor(repayAssets, receiver);
}
```

Recommended mitigation

It's recommended to implement a mechanism to handle saturating debt repayments gracefully within the affected functions.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-M-18: Interest rate manipulation via uncollateralized deposits and withdrawals

- **Category:** Interest manipulation issues
- **Source:** BorrowableCToken.sol, DynamicIRM.sol
- **Status:** Acknowledged

Description

A depositor with uncollateralized shares can manipulate interest rates at zero cost except gas fees. The attack works by temporarily withdrawing funds to decrease *assetsHeld()* and manipulate the utilization rate, which affects interest rates for a fixed 10-minute period, then redepositing the funds.

The interest rate calculation directly uses *assetsHeld()*:

```
(rate, adjustmentRate)
    = IRM.adjustedBorrowRate(assetsHeld(), outstandingDebt);
```

```
function assetsHeld() public view returns (uint256) {  
    return _asset.balanceOf(address(this));  
}
```

The utilization rate calculation in the *DynamicIRM* uses both **assetsHeld** and **debt**:

```
function utilizationRate(  
    uint256 assetsHeld,  
    uint256 debt  
) public pure returns (uint256 r) {  
    // Utilization rate is 0 when there are no outstanding debt.  
    r = debt == 0 ? 0 : _mulDiv(debt, WAD, assetsHeld + debt);  
}
```

An attacker can withdraw their uncollateralized deposits to decrease *assetsHeld()*, which increases the utilization rate and causes higher interest rates to be locked for 10 minutes, then immediately redeposit the funds to benefit from the higher rates.

The manipulation is profitable when the lost 1 second interest is worth less than the gain from manipulated rates. Unlike standard lending protocols like Aave/Compound, Curvance's fixed 10-minute interest rate periods and vertex multiplier mechanism make this manipulation more attractive and profitable. The protocol attempts to prevent such manipulation through cooldown periods, but these only apply to borrows and collateralized positions:

```
uint256 public constant MIN_HOLD_PERIOD = 20 minutes;
```

However, uncollateralized deposits and withdrawals are not subject to these cooldown restrictions.

A separate issue arising from the same lack of a cooldown for uncollateralized deposits and withdrawals is that flash loan fees can be sandwiched by depositing before the flash loan fee is earned and withdrawing immediately after. The flash loan fees don't accrue over time, and so without a cooldown there is an incentive to capture part of the fee.

Recommended mitigation

To establish a consistent incentive mechanism against interest manipulation, any funds moving into (and therefore out of) the protocol must have contributed to two interest rate adjustments. This extends the invariant from borrows to deposits (and therefore withdrawals).

Another possible mitigation to disincentivize manipulation of interest rates is to only make users earn interest after a certain period, e.g., 10 minutes. Currently the only incentive against performing the withdraw -> deposit loop is the foregone interest of one block. By increasing the duration for which no interest is earned, the opportunity cost of withdrawing can be made more meaningful.

Team response

Acknowledged.

TRST-M-19: Borrowers may not experience delayed multiplier and interest rate updates

- **Category:** Logical issues
- **Source:** DynamicIRM.sol
- **Status:** Acknowledged

Description

DynamicIRM documents the invariant that each borrower must [experience at least 2 adjustments](#).

DynamicIRM.adjustedBorrowRate() returns the **ratePerSecond** calculated with the old multiplier and updates the multiplier after that, becoming effective only in the next call to *adjustedBorrowRate()*.

When a borrower borrows at time **t**, the period with the fixed rate can last up to 10 minutes. At **t+600** the rate is updated but it takes another 10 minutes for the new multiplier to become effective at **t+1200**. Since the borrower can repay at **t+1200**, they have only experienced 1 rate adjustment and 0 multiplier adjustments based on their borrow action.

In each adjustment step, the multiplier can be adjusted by at most 20%, so for the multiplier, the impact is capped at paying 20% too small of an interest rate for 10 minutes.

Recommended mitigation

It's recommended to calculate **ratePerSecond** with the new multiplier instead of the old one. Additionally, to ensure borrowers experience the minimum 2 interest rate adjustments according to spec, the cooldown period must be increased from 20 minutes to 30 minutes. The requirement is that **MIN_HOLD_PERIOD / ADJUSTMENT_RATE >= 3** to guarantee at least 2 full rate adjustments with multiplier effects.

Team response

Acknowledged, we updated documentation to indicate that users experience 2 updates on average.

TRST-M-20: Effective adjustment periods are not 10 minutes and accrual paths behave differently

- **Category:** Logical issues
- **Source:** DynamicIRM.sol, BorrowableCToken.sol
- **Status:** Acknowledged

Description

The multiplier in *DynamicIRM* is updated with every call to *adjustedBorrowRate()* and *DynamicIRM* specifies the **ADJUSTMENT_RATE** as **10 minutes**. However, when jumping over intervals in *_accrueIfNeeded()*, there is no call to *adjustedBorrowRate()* and no multiplier adjustment occurs.

Multiple intervals that are accrued at once don't compound their interest. This lack of compounding and dependence on the accrual path is accepted within an adjustment period but may be unintentional across adjustment periods. Furthermore, when **block.timestamp >= vestingEnd**, *adjustedBorrowRate()* is calculated based on **outstandingDebt**, and the accrued debt from the previous period, which is equal to **assetsToVest**, is not added to **outstandingDebt**, causing another lack of compounding on the boundary between adjustment periods. Not only does this lack of compounding affect the past, but has implications for the next 10 minutes, since *adjustedBorrowRate()* is slightly underestimated.

Also the new **vestingEnd** can be as little as **1 second** into the future, meaning adjustments can occur at **t** and **t + 1**, violating the 10-minute specification.

The interval jumping logic in *_accrueIfNeeded()* is the following:

```
// The multiplication logic here is to round down to
// discrete `adjustmentRate` cycles, e.g. if block.timestamp is 3
// `adjustmentRate`'s ahead then begin vesting all of them for
// users at once.
adjustmentRate = (((block.timestamp - vestingEnd) /
  adjustmentRate) * adjustmentRate) + adjustmentRate;
vestingEnd = vestingEnd + adjustmentRate;
```

Consider an elevated interest rate that should increase the multiplier. When the *BorrowableCToken* is not interacted with for extended periods, there is only 1 adjustment at the time of the next interaction, and all the periods that passed without interaction have not contributed to the new multiplier.

Additionally, the jumped-over periods are accrued with the new rate instead of the old rate. The correct behavior would be to use the old rate for historical periods.

The evidence that jumped over periods must not be accrued with the new rate is provided by the effect of donations. For example, at **99% utilization** and **multiplier = 10 WAD**, interest is around **165% per month**. By lowering utilization to **98%**, interest drops to **162%**. If a large borrower has **50%** of all debt and debt hasn't been accrued for **1 month**, they can donate **2% * debt** to achieve a **3% * debt** cost reduction by changing the utilization calculation from **99/100** to **99/101**.

Recommended mitigation

With the goal of preserving existing interest rate mechanics and mitigating the above issues, the following two properties guide the recommendation:

1. Adjustment periods accrued one-at-a-time vs. multiple-at-a-time vs. partially-accrued result in the same outcomes, only with the exception of compounding vs. non-compounding interest within a given adjustment period.

2. Effective adjustment periods have constant length of 10 minutes (i.e., the multiplier is updated exactly every 10 minutes)

The fix is complicated by the observation that the multiplier needs to be applied in a loop. Each 10-minute interval must apply increase or decrease, and decay, to the accumulated multiplier according to the interest rate configuration. However, a loop can't be implemented for gas reasons.

Thus a valid approach could be to either find a closed form calculation to compress the loop into a constant gas cost, even simplifying if necessary, or implement a loop approach for a reasonable number of iterations to maintain equivalence. For the second option, fallback to one-interval in case it is too long to avoid DOS and implement bots to accrue interest and prevent staleness.

Team response

Acknowledged. We don't want to apply multiplier many times if nobody has called contracts as it means interest rates are in an equilibrium (no people leaving/entering loans).

Mitigation review

Even though the overall issue has not been addressed, one specific change has been made, by adding [assetsToVest](#) from the previous vesting period to the **outstandingDebt** when a new vesting period is started.

TRST-M-21: Borrows below `MIN_ACTIVE_LOAN_SIZE` are possible due to repayments and exact liquidations

- **Category:** Logical issues
- **Source:** `BorrowableCToken.sol`, `MarketManagerIsolated.sol`
- **Status:** Fixed

Description

The `MIN_ACTIVE_LOAN_SIZE` constant exists to minimize the potential of debt positions being created that cannot be profitably closed. This constant is properly enforced during borrow operations in *LiquidityManagerIsolated*:

```
if (newDebt < MIN_ACTIVE_LOAN_SIZE) {  
    revert LiquidityManager__InsufficientLoanSize();  
}
```

However, the protection can be bypassed through two attack vectors.

The first is that, after the 20-minute hold period, users can repay partial amounts to reduce their outstanding debt below `MIN_ACTIVE_LOAN_SIZE`. The `canRepay()` function in *MarketManagerIsolated* only checks the hold period and token listing status but does not validate the resulting debt size.

```
function canRepay(address cToken, address account) external view {  
    _checkIsListedToken(cToken);
```

```
_checkHoldPeriod(account);  
}
```

The second is that the *liquidateExact()* function allows liquidators to specify exact debt amounts to repay, which can leave the borrower with a remaining debt smaller than **MIN_ACTIVE_LOAN_SIZE**. The liquidation logic does not enforce a minimum remaining debt requirement.

These bypasses allow attackers to create loans that are not profitable to liquidate, potentially leading to bad debt accumulation. Attackers can split positions across many small loans to disincentivize liquidation.

Recommended mitigation

It's recommended to prevent repayments and liquidations that create small debts by adding validation in both the repayment and liquidation flows to ensure the remaining debt after the operation is either **zero** or at least **MIN_ACTIVE_LOAN_SIZE**. Then the only remaining way for debt to reach below **MIN_ACTIVE_LOAN_SIZE** is through value loss of the borrowed token, which is a systemic risk.

Team response

Fixed by commit [f10667c](#).

Mitigation review

A new [canRepayWithReview\(\)](#) function has been implemented which checks that the remaining debt can't be less than **MIN_LOAN_SIZE**, unless it is zero. To calculate the debt value, the price is queried with **getLower=false** and *_assetValue()* is rounded down. This is incorrect since in this case debt should be underestimated, not overestimated. It is recommended to set **getLower=true** and to round down the *_assetValue()* by passing **increasesCollateral=true**. Not checking remaining debt in the liquidation flow is a design choice.

Team response

Fixed by commit [6ea8779](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-M-22: RedStoneCoreAdaptor.addAsset() cannot handle native tokens

- **Category:** Logical issues
- **Source:** RedStoneCoreAdaptor.sol
- **Status:** Fixed

Description

In *RedstoneCoreAdaptor*, **config.symbolHash** is used as the data feed ID:

```
function writePrice(  
    address asset,  
    bool inUSD,  
    uint48 redstoneTimestamp
```

```

) external {
    AssetConfig storage config = assetConfig[asset][inUSD];

    // We can check if `asset` is supposed by checking `decimals` as we
    // do not allow configuring `decimals` equal to 0.
    if (config.decimals == 0) {
        revert RedstoneCoreAdaptor__AssetIsNotSupported();
    }

    if (config.redstoneTimestamp >= redstoneTimestamp) {
        return; // Can skip storing the data since the data is stale.
    }

    _validateTimestamp(redstoneTimestamp);
    /// @solidity memory-safe-assembly
    assembly {
        tstore(_TRANSIENT_REDSTONE_TIMESTAMP_KEY, redstoneTimestamp)
    }

    uint256 price = getOracleNumericValueFromTxMsg(config.symbolHash);
    // Adjust price pulled if necessary.
    price = _adjustPrice(asset, inUSD, price, config.decimals);

```

This value is set in `addAsset()`, which calls `_toBytes32()` or `_toBytes32Symbol()` from `Bytes32Helper`:

```

function addAsset(
    address asset,
    bool inUSD,
    uint8 decimals,
    uint256 heartbeat
) external {
    bytes32 symbolHash;
    if (inUSD) {
        symbolHash = Bytes32Helper._toBytes32(asset);
    } else {
        symbolHash = Bytes32Helper._toBytes32Symbol(asset, _nativeSymbol);
    }
}

```

These library functions call `_symbol()` to fetch the token symbol:

```

function _symbol(address token) internal view returns (string memory r) {
    r = IERC20(token).symbol();
}

```

This fails for native tokens represented as `address(0)` or `0xEeeeeEeeeE...eeeeEeeeeeeeeEEeE` in the protocol, preventing `RedstoneCoreAdaptor` from pricing native tokens.

Recommended mitigation

It's recommended that `addAsset()` accepts `symbolHash` (data feed ID) as a direct parameter from authorized callers instead of calculating it within the function, allowing native token support.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-M-23: Zappers lack validation to prevent callbacks

- **Category:** Reentrancy issues
- **Source:** BaseZapper.sol
- **Status:** Acknowledged

Description

The protocol aims to prevent all callbacks as a potential attack vector. While all exploitable callbacks have been reported separately, due to insufficient input validation, zappers contain such arbitrary callbacks, which weren't directly exploited. Following agreement with Curvance, these are reported at medium severity as it is difficult to be sure of their ultimate potential impact when accounting for third-party integrations and other unidentified threats.

Zappers don't validate cToken, input token and output token addresses before interacting with them. Without validation, malicious tokens can implement callback mechanisms during transfers, approvals, or other operations.

The protocol's intent to prevent callbacks is evident by the comment in *BaseZapper._transferToRecipient()*:

```
// If the token to refund is the chains' native gas token we wrap  
// then transfer it to prevent callback attack vectors.
```

Recommended mitigation

It's recommended to maintain a consistent callback prevention policy across all zapper functions by validating external contract addresses before interaction. Specifically, validation that cToken addresses are registered in the protocol's market manager, and that input/output tokens are either recognized protocol assets or explicitly whitelisted, is necessary.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Validation has been implemented for cToken addresses, however validation for input/output tokens is still missing. It is recommended to implement the missing validation or to assess the reentrancy attack surface that is enabled by interactions with arbitrary input/output tokens. If no risks are found, the missing validation may be accepted.

Team response

Acknowledged callbacks from input/output tokens.

TRST-M-24: Non-atomic IRM upgrade can deny access to the protocol

- **Category:** DOS issues
- **Source:** DynamicIRM.sol, BorrowableCToken.sol

- **Status:** Acknowledged

Description

Elevated permissions can upgrade the IRM via *setIRM()*. The next *accrueIfNeeded()* will call the IRM's *adjustedBorrowRate()*, which checks the caller is the **linkedToken**:

```
if (msg.sender != c.linkedToken) {
    revert DynamicIRM__Unauthorized();
}
```

To set **linkedToken**, the *setLinkedToken()* setter requires the **CToken** to already point to this IRM:

```
if (address(IBorrowableCToken(cTokenAddress).IRM()) != address(this)) {
    revert DynamicIRM__InvalidToken();
}
```

During initial CToken setup, the flow is:

- Create the IRM.
- Create the CToken and set the IRM.
- Call *setLinkedToken()* to connect the CToken to the IRM.
- At this point *accrueIfNeeded()* succeeds as they are paired.

If these actions aren't performed atomically, between steps (2) and (3) the **CToken** cannot perform any actions (*accrueIfNeeded()* will fail). This isn't problematic initially as only the initial reserve holds CTokens.

However, when upgrading IRM on a live system with active users, if steps (2) and (3) aren't atomic, there will be a period where all CToken interactions are frozen, users cannot borrow, repay, or accrue interest until *setLinkedToken()* is called. At worst case, the market could fall victim to bad debt during this gap, as liquidations and repays won't function.

Recommended mitigation

It's recommended that *setLinkedToken()* does not require the CToken to already point to it. Instead, it should be required in *setIRM()* that the new IRM has **linkedToken == address(this)**. Then the upgrade path becomes:

- Call *setLinkedToken()* on the new IRM.
- Call *setIRM()* on the CToken.

At no point does the BorrowableCToken point to an inactive IRM.

Team response

Acknowledged.

TRST-M-25: Limited interest rate precision can cause significant rounding errors

- **Category:** Math issues
- **Source:** BorrowableCToken.sol
- **Status:** Fixed

Description

Due to limited precision in the interest rate accrual mechanism, users can incur significant deviations of the interest they have to pay in comparison to the interest they expect to pay.

Suppose the following sample rates:

- 2% interest base rate, 70% vertex start, 10% util → 90,599,405 rate $\approx 9e7$

Modifying the above parameters to establish a reasonable range:

- 1% util → 9,059,540 rate $\approx 9e6$
- 10% base rate, 70% vertex start, 10% util → 452,997,028 rate $\approx 5e8$

It follows that an interest rate in the range of $1e7$ is supported.

BorrowableCToken._assetsToVest() then calculates the assets that this rate corresponds to and *BorrowableCToken._accrueIfNeeded()* increases the **marketDebtIndex** accordingly (it is already reported in another finding that **marketDebtIndex** is incorrectly rounded, but that is not the point here). Users are then charged interest in *BorrowableCToken.debtBalance()* based on the **marketDebtIndex** and their own user index.

Three equations can be extracted that lead from the rate to the user debt:

- **assetsToVest = rate * timePassed * outstandingDebt / WAD**
- **marketDebtIndex = assetsToVest * marketDebtIndex / outstandingDebt + marketDebtIndex**
- **userDebtNew = userDebtOld * marketDebtIndex / userDebtIndex**

The bottleneck is in the calculation of **assetsToVest**. For **rate = $1e7$** , **timePassed = 1**, and **outstandingDebt = 100 WBTC = $100 * 1e8 = 1e10$ wei**, **assetsToVest** rounds to zero: **floor($1e7 * 1e10 / 1e18$) = 0**.

Rounding up solve the problem but introduces another - the **assetsToVest** will be overestimated by a factor of 10 - as if util was 10 times higher. The root issue is about having the intermediate step where rate is converted to assets and assets to the index.

In addition to the bottleneck in the intermediate step, the final step where the user's debt is accrued can be performed once per second, and thus encounter a rounding error of up to **1 wei** per second against the user. If interest were accrued for a user each second for 1 year, then this amounts to: **1 wei per second * 31,536,000 seconds / $1e8$ wei per WBTC * \$100k per WBTC = \$31,535**, which is a substantial error.

Recommended mitigation

The intermediate rounding bottleneck could be removed by translating **rate** into **marketDebtIndex** immediately. Then there are roughly 7 decimals precision in the **rate**, and 18 decimals precision in **marketDebtIndex**. But in the current design it is also necessary to increment **totalAssets**, and that is why a solution to address the intermediate rounding must include a refactoring.

To address the rounding in the final accrual step, user debt can be calculated with additional precision in fractions of **1 wei**. Then, if only the users themselves can make the final accrual step into token amount, they are protected from unreasonably high cumulative rounding errors.

In practice, higher token amounts than for WBTC are expected, and the chosen example rates have intentionally been on the low end. Nonetheless, numerical precision along the interest accrual flow is limited for long-tail, yet still supported, market configurations.

Team response

Fixed by commit [f10667c](#). Now **marketDebtIndex** rounds up, and added comment to clarify very low utilization could result in users paying higher interest rates than estimated.

Mitigation review

The issue is considered fixed based on the updated [documentation](#). Rounding up **marketDebtIndex** is the mitigation for TRST-M-15 and not a direct fix for the limited precision in the interest rate accrual flow, and the above description of the issue and calculations still apply.

TRST-M-26: Transient liquidation configuration is not enforced to be used in isolation

- **Category:** Transient storage issues
- **Source:** AuctionManager.sol, CentralRegistry.sol, MarketManagerIsolated.sol
- **Status:** Acknowledged

Description

AuctionManager sets transient liquidation configuration in its pre-solver hook. Thereby each solver operation is executed with its own liquidation parameters. There are two related issues with this approach.

The first is that transient storage remains set for the duration of the entire transaction, and the context that is executed after the Atlas *metacall()* returns can use the same liquidation parameters as the last solver. As a result, a malicious bundler can break out of its trust assumptions and perform liquidations on one account that are meant to be used on another account.

Secondly, a potentially untrusted call from within one of the solver operations could grief the solver by calling a second *AuctionManager* (multiple *AuctionManager*'s will be supported in the future). The second *AuctionManager* would be qualified to change the liquidation parameters, and those will persist back in the original *AuctionManager*.

Recommended mitigation

Implement a post-solver hook that resets transient liquidation configuration and locks the market. This mitigates the first vector. Furthermore, revert when trying to change transient liquidation configuration from a non-zero state to a different non-zero state. This mitigates the second vector.

Team response

First point was acknowledged by Fastlane, second was confirmed "not an issue" from Fastlane as well.

TRST-M-27: AuctionManager does not support dynamic close factor

- **Category:** Logical issues
- **Source:** AuctionManager.sol, MarketManagerIsolated.sol
- **Status:** Fixed

Description

According to the *AuctionManager* requirements discussed with Fastlane Labs, the *AuctionManager* is expected to support a setting to enable a dynamic close factor. Fastlane Labs expects this setting to be enabled by setting **atlasCloseFactor** in *AuctionManager* to zero. However, it is not allowed to be set to zero:

```
if (atlasCloseFactor_ == 0 || atlasCloseFactor_ > BPS) revert AuctionManager__InvalidCloseFactor();
```

Furthermore, Fastlane Labs expects that by using a dynamic close factor, they are still able to determine the liquidation incentive. But by setting **atlasCloseFactor=0**, even if it were allowed in *AuctionManager*, *MarketManagerIsolated* defaults to the internally calculated incentive:

```
if (aData.closeFactor == 0 || aData.liqInc == 0) {  
    aData.closeFactorBase = c.closeFactorBase;  
    aData.closeFactorCurve = c.closeFactorCurve;  
    aData.liqIncBase = c.liqIncBase;  
    aData.liqIncCurve = c.liqIncCurve;  
}
```

This means the expected behavior for dynamic close factors with custom liquidation incentives is not implemented and Fastlane Labs can't operate the *AuctionManager* as required.

Recommended mitigation

It's recommended to clearly define and implement the integration between Fastlane Labs, *AuctionManager*, and *MarketManagerIsolated* such that all but no more than the necessary permissions are made available.

Team response

The current design is always passing 0/0 for dynamic values via auction manager.

Mitigation review

Dynamic close factors have been implemented by commit [48e5a91](#) / [f10667c](#) and the issue is fixed.

TRST-M-28: AuctionManager should deny oracles and selectors by default

- **Category:** Access control issues
- **Source:** AuctionManager.sol
- **Status:** Fixed

Description

In *AuctionManager*, if there are no selectors or oracles whitelisted, all selectors and oracles are allowed:

```
function verifyOracleWhitelist(address oracle) external view {
    if (whitelistedOraclesCount > 0 && !oracleWhitelist[oracle]) revert AuctionManager__InvalidOracle();
}
function verifyAllowedSelector(bytes4 selector) external view {
    if (allowedSelectorsCount > 0 && !allowedSelectors[selector]) revert AuctionManager__InvalidSelector();
}
```

This "allow by default" design introduces an issue when there are no oracles whitelisted. In fact, the constructor only whitelists specific selectors, so this issue exists at the outset.

```
allowedSelectors[IRedstoneProxy.updateDataFeedsValues.selector] = true;
allowedSelectors[IRedstoneProxy.updateDataFeedsValuesPartial.selector] = true;
```

Generally, the action of removing an oracle can result in the opposite, i.e., inadvertently whitelisting all oracles.

Recommended mitigation

There are specific oracles and selectors that should be called, and these should have to be whitelisted explicitly. Rather than "allow by default", the policy must be "deny by default".

Team response

Fixed by commit [48e5a91](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-M-29: AuctionManager.setAtlasCloseFactor() as a governor protected function is incompatible with requirements

- **Category:** Logical issues
- **Source:** AuctionManager.sol
- **Status:** Fixed

Description

AuctionManager exposes *setAtlasCloseFactor()* as a governor-protected function:

```
function setAtlasCloseFactor(uint256 atlasCloseFactor_) external {
    _checkIsGovernor();

    if (atlasCloseFactor_ == 0 || atlasCloseFactor_ > BPS) revert AuctionManager__InvalidCloseFactor();
    uint256 old = atlasCloseFactor;
    atlasCloseFactor = uint16(atlasCloseFactor_);
    emit AtlasCloseFactorSet(old, atlasCloseFactor_);
}
```

The governor is a multisig with signers from Fastlane Labs and Curvance. This design, as the only way of setting the close factor, is incompatible with the requirement that the close factor must depend on account health.

Account health can change rapidly while the multisig cannot act quickly. In fact, the faster the multisig can act, the less protected it is against compromise. This signifies the incompatibility

of multisig action and close factor setting, which should respond dynamically to market conditions and thus account health.

Recommended mitigation

Determining the **atlasCloseFactor** must be implemented via a mechanism that does not rely on a high-permissioned multisig role to provide updates.

Team response

The current design is always passing 0/0 for dynamic values via auction manager.

Mitigation review

Dynamic close factors have been implemented by commit [48e5a91](#) / [f10667c](#) and the issue is fixed.

TRST-M-30: ERC4337 transactions can be denied due to callbacks, leading to DOS of the protocol

- **Category:** Reentrancy issues
- **Source:** BasePositionManager.sol, OdosCalldataChecker.sol, BaseCToken.sol, BorrowableCToken.sol
- **Status:** Acknowledged

Description

The protocol fails to protect against callbacks in multiple different instances. In this finding, all the exploitable instances are covered. The non-exploitable instances, which all originate from zappers, are separated into a Medium-severity finding.

This finding covers the following sources for callbacks:

Firstly, *BasePositionManager.onRedeem()* can call an untrusted **outputToken** address. This happens because validation of the **outputToken** should occur inside the call to *_swapCollateralAssetToDebtAsset()*, but it is only applied conditionally due to the early return. If the position manager is used with a Curvance market that hits the early return, a callback from the token transfer inside *onRedeem()* can be achieved.

Secondly, the pattern of multiple swaps in the position manager **DeleverageAction** struct is unsafe. Each input token and output token must be validated so that it does not allow callbacks. It does not seem necessary to support multiple swap actions since one swap action can use a router that also combines multiple hops (assuming it can be ensured that there cannot be a callback from within the router).

Thirdly, the *OdosCalldataChecker* does not verify **pathDefinition**, **executor**, and **referralCode**, which can be the source of a callback and must be validated.

Lastly, in *BasePositionManager.onBorrow()*, the **action.cToken** address is not validated. In *BasePositionManager.onRedeem()*, the **action.borrowableCToken** address is also not validated.

All the above instances have in common that the callback is gained from within a context where the **cToken** contract's reentrancy guard is enabled; hence, a future **nonReentrant** call would revert.

For example, `BasePositionManager.onRedeem()` is called from within `BaseCToken._processPositionManagerRedemption()`, where the **cToken**'s reentrancy guard is enabled. Therefore, trying to transfer **cToken** shares or interacting with the **cToken** in a different way reverts, if done within this context.

The enabled reentrancy guard leads to an issue in combination with Account Abstract (AA) wallets that operate by trustless delivery models (Gnosis Safe, ERC4337 wallets etc). Below is a breakdown of the issue with respect to ERC-4337 wallets, which can be applied somewhat similarly to other wallets.

[ERC-4337](#) enables **Account Abstraction** by introducing a **UserOperation** pseudo-transaction object. User operations are sent to a separate mempool where they are picked up by bundlers and submitted as regular Ethereum transactions by calling `handleOps()` on a singleton [EntryPoint](#) contract.

The `handleOps()` function is permissionless, and **UserOperations** are propagated in a public mempool (as described in ERC-4337):

“Decentralization

- *Allow any bundler (think: block builder) to participate in the process of including account-abstracted `UserOperations`.*
- *Work with all activity happening over a public mempool; users do not need to know the direct communication addresses (e.g., IP, onion) of any specific actors.*
- *Avoid trust assumptions on bundlers.”*

Since `handleOps()` is permissionless, anyone can act as a [bundler](#) or observe the bundler mempools and submit a **UserOperation** on-chain (by self-including, or by frontrunning the future bundler and submitting to the general mempool / mev-boost directly). Therefore, an attacker can submit a transaction from their EOA to their custom contract, which will internally call the `handleOps()` function. In this call pattern, an attacker can modify the protocol state before a **UserOperation** is executed.

The most common mechanism by which protocol state can be manipulated to affect subsequent user operations is via a callback from a function with a reentrancy guard. In the context of the callback, the reentrancy guard is enabled, and so any interaction with the protocol that enforces a reentrancy guard will revert. This can be abused in a griefing attack where an attacker intentionally executes user operations in this context to make them revert.

To prevent a user from transferring their **cToken** shares, the following steps are performed by the attacker:

1. A user signs a `BaseCToken.transfer()` transaction and shares it with an alt-mempool.

2. An attacker observes the **UserOperation** and creates a malicious wrapper transaction.
3. The attacker EOA calls their own malicious contract:
 - The contract interacts with Curvance to enable the reentrancy guard. Any of the previously mentioned instances of callbacks may be used.
 - In the callback, the attacker calls *EntryPoint.handleOps()* and inserts the single victim **UserOperation**. The *EntryPoint* will deliver the **UserOperation** after successful validation.
 - The **UserAccount** will call *BaseCToken.transfer()*, which will revert.
 - The *EntryPoint* will charge the **used gas + penalty** and credit the attacker.
 - The attacker will return gracefully from the callback, finalizing execution.

Not only are users denied access to the protocol and lose the gas that their operation has already spent, but they are also likely to pay a penalty directly to the malicious bundler when the execution flow is interrupted early, from the [specs](#):

“A penalty of 10% (`UNUSED\_GAS\_PENALTY\_PERCENT`) is applied on the amounts of `callGasLimit` and `paymasterPostOpGasLimit` gas that remains unused. This penalty is only applied if the amount of the remaining unused gas is greater than or equal to 40,000 (`PENALTY\_GAS\_THRESHOLD`). This penalty is necessary to prevent the `UserOperations` from reserving large parts of the gas space in the bundle but leaving it unused and preventing the bundler from including other UserOperations.”

Essentially, the malicious bundler makes the victim **UserOperation** appear as if it were a malicious operation that intentionally tries to occupy gas space. Since the revert is likely to happen early (due to the reentrancy check), it is very likely to leave a sufficiently large amount of unused gas and trigger the penalty.

BaseCToken and *BorrowableCToken* guard all user interactions with a reentrancy guard. Therefore, the attack leads to complete denial of service (DoS) and freezing of funds. Since users need to actively manage their positions (e.g., to avoid liquidation), the freezing of funds can result in direct loss of funds.

Recommended mitigation

Since Curvance intends to disallow any callbacks, the solution is straightforward: add explicit validation in all of the vulnerable instances described above.

Team response

Acknowledged the callback from Odos swaps (the **path** is not validated) and intermediate swap tokens in the **DeleverageAction** struct. Fixed the remaining instances by commit [f10667c](#).

Mitigation review

Verified, all instances that are not acknowledged have been fixed correctly.

Low severity findings

TRST-L-1: CTokens breaks ERC4626 standard

- **Category:** Compliance issues
- **Source:** BaseCToken.sol
- **Status:** Acknowledged

Description

The protocol's cToken contracts implement ERC4626 but violate key specification requirements in multiple functions.

The `convertToShares()` and `convertToAssets()` functions use the **nonReentrant** modifier which can cause reverts during reentrancy. ERC4626 specification states these functions *"MUST NOT revert unless due to integer overflow caused by an unreasonably large input."*

Additionally, the `maxWithdraw()` and `maxRedeem()` functions don't account for withdrawal limits. Here, the ERC4626 specification states these functions *"MUST factor in both global and user-specific limits, like if withdrawals are entirely disabled (even temporarily) it MUST return 0."* The protocol has withdrawal limits based on available underlying assets and borrowing activity, but these are not reflected in the max functions.

Recommended mitigation

It's recommended to implement all ERC4626 functions according to specification or to document that ERC4626 compliance is not provided. Compliance with a standard can either be in full or not at all. Specifying a partial compliance is inconsistent.

Team response

Acknowledged, updated [documentation](#).

TRST-L-2: BaseCToken transfers are not compatible with ERC20

- **Category:** Compliance issues
- **Source:** BaseCToken.sol
- **Status:** Acknowledged

Description

The `BaseCToken` inherits from ERC4626, which inherits from ERC20. The ERC20 specification states: *"Note: Transfers of 0 values MUST be treated as normal transfers and fire the Transfer event."* However, the `_checkTransfer()` function calls `_checkZeroAmount()` which reverts on zero amounts:

```
function _checkZeroAmount(uint256 assets) internal pure {  
    /// @solidity memory-safe-assembly  
    assembly {  
        if iszero(assets) {  
            mstore(0x00, 0xc0883a55) // Revert BaseCToken __ZeroAmount().  
            // Return bytes 29-32 for the selector.  
            revert(0x1c, 0x04)  
        }  
    }  
}
```

```

    }
  }
}

```

Both *transfer()* and *transferFrom()* functions call *_checkTransfer()*.

This could cause integrations which perform zero value transfers as part of their logic to revert the entire call, breaking compatibility with external protocols that expect standard ERC20 behavior.

Recommended mitigation

It's recommended to remove the ***_checkZeroAmount(shares)*** call from *_checkTransfer()* to allow zero value transfers to proceed normally as required by the ERC20 specification.

Team response

Acknowledged, updated [documentation](#).

TRST-L-3: OracleManager and downstream contracts revert instead of returning error codes

- **Category:** Integration issues
- **Source:** OracleManager.sol, BaseOracleAdaptor.sol
- **Status:** Acknowledged

Description

OracleManager and downstream contracts may revert rather than returning an error code that can be handled by integrators. The *getPrice()* function documentation states it returns error codes (**0 = no error, 1 = caution, 2 = bad source**), but several functions within the pricing flow can revert instead of gracefully returning error codes. Examples of reverts that are not returned as error codes are as follows:

_checkSupportedAsset() in *BaseOracleAdapter*:

```

function _checkSupportedAsset(address asset) internal view {
    // Validate we support pricing `asset`.
    if (!isSupportedAsset[asset]) {
        revert BaseOracleAdaptor__AssetIsNotSupported();
    }
}

```

_checkIsApprovedAdaptor() in *OracleManager*:

```

function _checkIsApprovedAdaptor(address adaptor) internal view {
    if (!isApprovedAdaptor[adaptor]) {
        revert OracleManager__AdaptorIsNotApproved();
    }
}

```

Decimal adjustment in *adjustPrice()* can overflow:

```

function _adjustPrice(
    address asset,

```

```
bool inUSD,
uint256 price,
uint256 decimals
) internal view returns (uint256) {
    // Normalize price to 18 decimals (WAD).
    if (decimals != 18) {
        price = FixedPointMathLib.fullMulDiv(price, WAD, 10 ** decimals); // Can overflow
    }
    // ...
}
```

Even with *fullMulDiv()*, the result can overflow, and cause reverts instead of returning a **BAD_SOURCE** error code. As a result of the missing error handling, integrators may suffer unexpected reverts and DOS.

Recommended mitigation

All sources of reverts in *OracleManager* and downstream contracts must be detected and returned as an error code. For conditions such as an unsupported asset, it's recommended to consider adding a new error code more severe than **BAD_SOURCE**.

Team response

Acknowledged.

TRST-L-4: BaseCToken.supportInterface() breaks EIP165

- **Category:** Compliance issues
- **Source:** BaseCToken.sol
- **Status:** Fixed

Description

The *BaseCToken* breaks ERC165 specification compliance by not checking for the ERC165 interface ID in its *supportsInterface()* function:

```
function supportsInterface(
    bytes4 interfaceId
) public pure virtual returns (bool result) {
    result = interfaceId == type(ICToken).interfaceId ||
        interfaceId == type(ERC4626).interfaceId;
}
```

The function only checks for **ICToken** and **ERC4626** interface IDs but never checks the ERC165 **interfaceId** itself, breaking the ERC165 specification.

Recommended mitigation

It's recommended to add the ERC165 interface check to the *supportsInterface()* function.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-L-5: The `transferEmergencyCouncil()` and `transferTimelockPermissions()` functions don't update the roles in `DAOTimelock`

- **Category:** Logical issues
- **Source:** `CentralRegistry.sol`
- **Status:** Fixed

Description

The `transferEmergencyCouncil()` function should call `timelock.updateRoles()` just like `transferDaoPermissions()` does since `DAOTimelock` needs to give **CANCELLER_ROLE** to the new emergency council.

The `updateRoles()` function in `DAOTimelock` is permissionless so the state can be updated, but there's a short window where the old emergency council keeps its **CANCELLER_ROLE**, unless the `updateRoles()` function is called atomically.

Furthermore, to avoid a scenario where a new timelock contract is created, and either the DAO or emergency council address is changed before the new timelock is set in the `CentralRegistry`, `transferTimelockPermissions()` should also call `updateRoles()`.

Recommended mitigation

It's recommended to add the same `updateRoles()` call to `transferEmergencyCouncil()` and `transferTimelockPermissions()` that exists in `transferDaoPermissions()`.

Team response

Fixed by commit [f10667c](#).

Mitigation review

The call to `ITimelock(newTimelock).updateRoles()` is still missing from `transferTimerlockPermissions()`.

Team response

Fixed by commit [6ea8779](#).

Mitigation review

Verified, the recommendation has been implemented. Another improvement to implement is to remove the interface check for the **timelock** from `transferEmergencyCouncil()` and `transferDaoPermissions()` since whenever a **timelock** is set, it is already checked that it implements the `ITimelock` interface.

TRST-L-6: DAO can transfer rounding protection shares from `BaseCToken`

- **Category:** Logical issues
- **Source:** `BaseCToken.sol`
- **Status:** Fixed

Description

BaseCToken.initializeDeposits() mints **_BASE_UNDERLYING_RESERVE** shares to itself as a layer of protection against rounding exploits. However, the DAO can call *rescueToken()* and transfer away these rounding protection shares without going through the 5-day timelock, effectively nullifying the rounding protection.

The *rescueToken()* function only prevents rescuing the underlying asset but allows rescuing the shares:

```
function rescueToken(address token, uint256 amount) external {
    _checkDaoPermissions();

    if (token == asset()) {
        _revert(_UNAUTHORIZED_SELECTOR);
    }

    RescueLib._rescueToken(centralRegistry, token, amount);
}
```

The DAO can act maliciously by calling **rescueToken(address(this), amount)**. These shares are rescued to the DAO itself, allowing the DAO to redeem them and remove the rounding protection. This can be considered a privilege escalation, as actions that risk the entire safety of the protocol should be gated behind elevated privileges.

Recommended mitigation

It's recommended to revert when **token == asset() || token == address(this)** to prevent rescuing both the underlying asset and the rounding protection shares.

Team response

Fixed by commit [f10667c](#). The *rescueToken()* function has been removed.

Mitigation review

Verified, the vulnerable function has been removed.

TRST-L-7: The *rescueToken()* safety hatch can't be overridden to account for external vault shares

- **Category:** Logical issues
- **Source:** BaseCToken.sol
- **Status:** Fixed

Description

It is possible to implement cTokens that inherit *BaseCToken* and that deposit minted shares in an external vault. The *BaseCToken* ends up owning the external vault shares instead of the original cToken shares for its **_BASE_UNDERLYING_RESERVE**. However, *rescueToken()* cannot be overridden since it's not marked as **virtual**, meaning any future implementation that uses an external vault inherits the wrong behavior without being able to change it.

This creates a privilege escalation issue for future cToken implementations since the DAO could move the external vault shares that represent the actual cToken shares and nullify the rounding protection, similar to the previous finding TRST-L-6.

Recommended mitigation

It's recommended to make *rescueToken()* a **virtual** function and document the need to override it depending on which address represents the cToken shares.

Team response

Fixed by commit [f10667c](#). The *rescueToken()* function has been removed.

Mitigation review

Verified, the vulnerable function has been removed.

TRST-L-8: *BasePositionManager.maxRemainingLeverage()* is too restrictive to max out debt

- **Category:** Logical issues
- **Source:** *BasePositionManager.sol*
- **Status:** Fixed

Description

In *BasePositionManager.maxRemainingLeverageOf()*, it is documented that the result can be overestimated. This would mean that *_leverage()* passes the remaining leverage sanity check but then fails the health check that is executed post-leverage.

However, the opposite case, where *maxRemainingLeverageOf()* returns a value that is too small, is also possible given calculations that involve oracle errors.

Consider the following example without oracle errors:

- **price(collateralToken) = 1**
- **price(debtToken) = 1**
- **collRatio = 98%**
- **sumCollateral = 100**
- **maxDebt = 98**
- **sumDebt = 0**

=> **maxLeverage = 98 * 100 / 2 = 4900**

This results in:

- **sumCollateral = 100 + 4900 = 5000**
- **maxDebt = 5000 * 98% = 4900**
- **sumDebt = 4900**

Thus, the position size has been increased **50x**, from **100** to **5000**, and indeed **maxLeverage = sumDebt**.

Now suppose there is an oracle error, and **debtToken** is actually worth **1.02** collateral tokens, i.e. the swap from debt to collateral is executed at **1.02**. All else remains unchanged. In particular, the debt and collateral price is still reported as **1**. Then at most **\$4900** debt can be borrowed due to the result of *maxRemainingLeverageOf()*, but it is swapped into **\$4998** collateral tokens.

Therefore **maxDebt** becomes $(100 + 4998) * 98\% = 4996$, and **sumDebt** is still **\$4900**. The allowed debt has not been maxed out, even though the maximum amount allowed from *maxRemainingLeverageOf()* has been borrowed.

Recommended mitigation

It's recommended to remove the *maxRemainingLeverageOf()* check from *_leverage()* since it is not needed. There is a regular health check anyway in the borrowing process. Instead, *maxRemainingLeverageOf()* should only be made available as an external function for frontend estimation purposes.

Team response

Fixed by commit [f10667c](#). The *maxRemainingLeverageOf()* function has been removed.

Mitigation review

Verified, the vulnerable function has been removed.

TRST-L-9: BorrowableCToken miscalculates new debt in *borrowForPositionManager()*

- **Category:** Logical issues
- **Source:** BorrowableCToken.sol
- **Status:** Fixed

Description

In *borrowForPositionManager()*, it is checked with the market manager if the borrow is allowed:

```
marketManager.canBorrow(  
    address(this),  
    0,  
    owner,  
    marketOutstandingDebt + assets  
);
```

The passed new net debt is **marketOutstandingDebt + assets**, to account for the new assets borrowed. However, this call happens after *_borrow()*, which already updates the **marketOutstandingDebt**. So, the passed value is too large.

The impact is that borrowing actions can incorrectly revert, if the borrowed amount makes the total exceed **debtCap**. This is very unlikely, and at worst would cause a revert.

Recommended mitigation

It's recommended to use **marketOutstandingDebt** as a parameter instead of **marketOutstandingDebt + assets**.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-L-10: Heartbeat grace period of 60 seconds is too short

- **Category:** Oracle issues
- **Source:** ConstantsLib.sol
- **Status:** Fixed

Description

HEARTBEAT_GRACE_PERIOD is defined as **60 seconds**:

```
/// @dev Extra time added to top end Oracle feed heartbeat incase of
/// transaction congestion delaying an update.
uint256 constant HEARTBEAT_GRACE_PERIOD = 60;
```

This is the buffer that is added on top of the formal heartbeat to account for update delays. For example, the heartbeat for a feed may be **1 hour (3600s)** but prices as old as **(3600s + 60s)** are accepted to account for network delays in the update.

The grace period is used across multiple oracle adaptors:

```
// ChainlinkAdaptor.sol
uint256 public constant DEFAULT_HEARTBEAT =
    1 days + HEARTBEAT_GRACE_PERIOD;

// RedstoneClassicAdaptor.sol
uint256 public constant DEFAULT_HEARTBEAT =
    1 days + HEARTBEAT_GRACE_PERIOD;

// Api3Adaptor.sol
uint256 public constant DEFAULT_HEARTBEAT =
    1 days + HEARTBEAT_GRACE_PERIOD;
```

Internal analysis, which has been conducted for a previous audit, has shown that in roughly **1%** of cases a **60s** buffer was too short. In all of those cases, **120s** would have been sufficient. Due to network delays, oracle updates may take a few seconds longer than expected, and prices inside Curvance can be stale. This results in a DOS for a small number of blocks.

Recommended mitigation

It's recommended to use **120s** instead **60s**, which is a sweet spot between allowing sufficient grace period, but also detecting staleness as early as possible in case the feed is really stale.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-L-11: RedstoneCoreAdaptor price guard must be applied when the price is read

- **Category:** Oracle issues
- **Source:** RedstoneCoreAdaptor.sol
- **Status:** Fixed

Description

When the price guard is set or changed for *RedstoneCoreAdaptor*, this does not affect the latest price that has been stored with *writePrice()*. This is since price adjustment happens on the input flow, not the output flow. So, if there is not a new input price atomically after *setGuardedPriceConfig()*, there is a window where a price can be read with an old / no price guard.

This is different from Chainlink and Redstone Classic where the price guard goes into effect immediately. RedStone Classic and Chainlink adaptors apply price guards in their *_getPrice()* function, meaning every time a price is requested, the current price guard configuration is applied to the raw feed data:

```
// ChainlinkAdaptor._getPrice()
function _getPrice(
    address asset,
    bool inUSD
) internal view override returns (PricingResult memory result) {
    // Get fresh price data from Chainlink feed
    (, int256 price,, uint256 updatedAt, ) = IChainlink(c.aggregatorProxy)
        .latestRoundData();

    // Apply current price guard configuration on every read
    uint256 adjustedPrice = _adjustPrice(
        asset,
        inUSD,
        uint256(price),
        c.decimals
    );

    result.hadError = _verifyData(adjustedPrice, updatedAt, c.heartbeat);
    result.price = adjustedPrice; // Returns price with current guard applied
}
```

```
// RedstoneClassicAdaptor._getPrice()
function _getPrice(
    address asset,
    bool inUSD
) internal view override returns (PricingResult memory result) {
    // Get fresh price data from Redstone Classic feed
    (, int256 price,, uint256 updatedAt, ) = IRedstone(c.feedProxy)
        .latestRoundData();

    // Apply current price guard configuration on every read
    uint256 adjustedPrice = _adjustPrice(
        asset,
        inUSD,
        uint256(price),
        c.decimals
    );
};
```

```

result.hadError = _verifyData(adjustedPrice, updatedAt, c.heartbeat);
result.price = adjustedPrice; // Returns price with current guard applied
}

```

The *RedstoneCoreAdaptor* applies price guards when storing prices via *writePrice()*, but not when reading them via *_getPrice()*:

```

// RedstoneCoreAdaptor.writePrice() - Price guard applied during storage
function writePrice(
  address asset,
  bool inUSD,
  uint256 redstoneTimestamp
) external {
  // ... validation ...

  uint256 price = getOracleNumericValueFromTxMsg(config.symbolHash);

  // Price guard applied only during write operation
  price = _adjustPrice(asset, inUSD, price, config.decimals);

  // Store the price with guard already applied
  config.price = uint184(price);
  config.redstoneTimestamp = redstoneTimestamp;
}

// RedstoneCoreAdaptor._getPrice() - No price guard re-applied on read
function _getPrice(
  address asset,
  bool inUSD
) internal view override returns (PricingResult memory result) {
  AssetConfig memory config = assetConfig[asset][inUSD];

  // ... staleness validation ...

  // Simply return the stored price (no current price guard applied)
  result.price = config.price;
}

```

An additional consequence is that *_guardedPrice()* is calculated based on **block.timestamp** when the price is written and hence can be slightly outdated by the time the price is read.

Recommended mitigation

It's recommended to apply the price guard in the output flow in order for guard updates to take effect immediately and to call *_guardedPrice()* with the correct timestamp. This involves calling *_adjustPrice()* in the *_getPrice()* function similar to other adaptors, though this costs additional gas for every read.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-L-12: ActionRegistry cooldown can be bypassed through borrow operations

- **Category:** Logical issues
- **Source:** ActionRegistry.sol
- **Status:** Acknowledged

Description

The *ActionRegistry* documentation states the following:

```
/// Second, 'ActionRegistry' enables the locking system,  
/// which operates as an optional 2FA setting to reduce the potential of  
/// a successful phishing attempt on a user. A cooldown can be set for  
/// token transfers and plugin delegation that activates after an action  
/// lock is enabled.
```

However, the cooldown mechanism can be bypassed through borrow operations. While transfers are disabled during the cooldown period, users can still call *borrow()* with the **receiver** parameter set to any address, including an attacker's address. This allows an attacker who has phished a user to borrow against the victim's collateral locked in Curvance, set themselves as the receiver of the borrowed fund and extract value immediately without waiting for the cooldown period.

This bypasses the intended protection mechanism, as the attacker can still extract a significant portion of the victim's funds (limited only by the collateralization ratio) despite transfers being locked.

Additionally, similar bypass vectors exist for the *deposit()* and *repay()* functions where an attacker could trick a victim into depositing funds for the attacker's benefit or repaying the attacker's debt using the victim's external funds.

In these last two cases, the funds are sourced from outside the domain of Curvance, whereas in the first case, the funds inside Curvance are lost.

Recommended mitigation

It's recommended to specify a precise definition of *ActionRegistry*'s cooldown mechanism, and the security guarantees it provides. With this definition, consistent protections should be implemented throughout the protocol by extending the cooldown restrictions to cover all of its functionality.

Team response

Acknowledged, updated the [documentation](#).

TRST-L-13: Market permissions are lost when address becomes EmergencyCouncil or Timelock

- **Category:** Access control issues
- **Source:** CentralRegistry.sol
- **Status:** Acknowledged

Description

Market permissions can be lost unexpectedly when they have been granted separately to an address that later becomes emergency council or timelock. Consider the following scenario:

- Contract **A** receives market permissions through `addMarketPermissions()`.
- **A** is nominated as Emergency Council.
- **B** is nominated as Emergency Council.
- At this point, **A** will lose market permissions.

Since **A** was granted market permission explicitly, it should not be removed implicitly. There is no reason to think **A** should no longer have market permissions in this flow.

The issue occurs in `transferEmergencyCouncil()`:

```
if (previousEmergencyCouncil != timelock) {
    delete hasElevatedPermissions[previousEmergencyCouncil];

    // If the previous Emergency Council also has DAO permissions
    // for some reason, do not remove their permissioning.
    if (previousEmergencyCouncil != daoAddress) {
        delete hasDaoPermissions[previousEmergencyCouncil];
        delete hasMarketPermissions[previousEmergencyCouncil];
        emit PermissionsUpdated(
            "Market",
            previousEmergencyCouncil,
            false
        );
    }
}
```

Similarly, in `transferTimelockPermissions()`:

```
if (previousTimelock != emergencyCouncil) {
    delete hasElevatedPermissions[previousTimelock];

    // If the previous Timelock also has DAO permissions
    // for some reason, do not remove their permissioning.
    if (previousTimelock != daoAddress) {
        delete hasDaoPermissions[previousTimelock];
        delete hasMarketPermissions[previousTimelock];
        emit PermissionsUpdated("Market", previousTimelock, false);
    }
}
```

Both functions unconditionally execute `delete hasMarketPermissions[previousAddress]` without checking whether the address had received market permissions explicitly through `addMarketPermissions()`.

Recommended mitigation

Two mitigation approaches are possible. For the first approach, it's recommended to separate the "explicit" market permission and the "implicit" market permission. For implicitly granted permissions, it is known that the current **timelock** and **EC** have it. So, tracking them in `hasMarketPermissions` is not necessary, which will only track explicitly granted permissions. Then a `_checkMarketPermissions()` function is needed, like `_checkDaoPermissions()`, which checks both the mapping, and the current **timelock/EC**.

-+

As a second approach, a context bit can be added to the mapping (which denotes implicitly/explicitly granted permissions), so that the implicit flows may not remove explicitly granted permissions.

Team response

Acknowledged.

TRST-L-14: Sanity check in `depositAndLeverage()` allows excessive slippage

- **Category:** Slippage issues
- **Source:** `BasePositionManager.sol`
- **Status:** Fixed

Description

In a regular `leverage()` operation, the expected behavior is **additionalDebt = additionalCollateral**, so checking for equality is correct. However, when there's a deposit involved in `depositAndLeverage()`, this equation doesn't hold.

The issue occurs in the `checkSlippage()` modifier:

```
// Scoping to avoid stack too deep.
{
    address[] memory assets = marketManager.assetsOf(account);
    uint256 numAssets = assets.length;
    IBorrowableCToken asset;

    for (uint256 i; i < numAssets; ++i) {
        asset = IBorrowableCToken(assets[i]);
        if (asset.isBorrowable()) {
            asset accrueIfNeeded();
        }
    }
}

(uint256 collateralBefore, , uint256 debtBefore) = marketManager
    .statusOf(account);
uint256 valueIn = collateralBefore - debtBefore;

_;

(uint256 collateralAfter, , uint256 debtAfter) = marketManager
    .statusOf(account);
uint256 valueOut = collateralAfter - debtAfter;

// If there was slippage, make sure its within slippage tolerance.
if (valueIn > valueOut) {
    if ((valueIn - valueOut) > _mulDiv(valueIn, slippage, WAD)) {
        revert BasePositionManager__InvalidSlippage();
    }
}
}
```

The modifier captures **collateralBefore** and **debtBefore** before the function body executes. In `depositAndLeverage()`, the deposit happens inside the function body:

```
function depositAndLeverage(
```

```
uint256 assets,
LeverageAction calldata action,
uint256 slippage
) external checkSlippage(msg.sender, slippage) nonReentrant {
    ICToken cToken = action.cToken;
    address collateralAsset = cToken.asset();

    // Transfer `collateralAsset` to deposit.
    SafeTransferLib.safeTransferFrom(
        collateralAsset,
        msg.sender,
        address(this),
        assets
    );

    // Approve cToken to process a deposit.
    SwapperLib._approveIfNeeded(collateralAsset, address(cToken), assets);

    // Deposit and collateralize `collateralAsset` in cToken contract.
    cToken.depositAsCollateralFor(assets, msg.sender);

    // Execute leverage operation.
    _leverage(action, msg.sender);
}
```

This means the **collateralBefore** value doesn't include the deposited amount, while **collateralAfter** does. The allowed slippage is therefore too large since the "after" value is increased by the deposited value while the "before" value doesn't include it. The issue is reported as Low severity, as this slippage check is only a sanity check. However, the sanity check is not effective in this case.

Recommended mitigation

It's recommended to update the accounting to include deposited collateral in the "before" value or to document that the "sanity check" is not effective in this case.

Team response

Fixed by commit [f10667c](#). Moved deposit action to a modifier which occurs before slippage check, properly including the collateral value in slippage calculations.

Mitigation review

Verified, the recommendation has been implemented.

TRST-L-15: Donated assets can cause totalAssets to underflow on bad debt realization

- **Category:** Underflow issues
- **Source:** BorrowableCToken.sol
- **Status:** Fixed

Description

A donation to the *BorrowableCToken* creates an accounting issue where **totalAssets** can underflow during bad debt realization. The donated underlying does not count towards **totalAssets** yet can be borrowed. When bad debt is later realized, the subtraction can cause an underflow.

The issue occurs because `assetsHeld()` returns the full balance including donations:

```
function assetsHeld() public view returns (uint256) {
    return _asset.balanceOf(address(this));
}
```

This allows borrowing of donated assets since `_checkAssetsHeld()` passes:

```
// Check if we have enough underlying held to support the withdrawal.
// We add _BASE_UNDERLYING_RESERVE to the calculation to ensure that
// the market never actually runs out of assets and may introduce
// invariant manipulation.
// This also acts as a protective mechanism against trying to
// manipulate marketOutstandingDebt above total underlying assets
// inside the system since there will always be at least
// _BASE_UNDERLYING_RESERVE excess inside the market.
if (assetsHeld() < assets + _BASE_UNDERLYING_RESERVE) {
    revert BorrowableCToken__InsufficientAssetsHeld();
}
```

However, when bad debt is realized during liquidation, it subtracts from `_totalAssets` without ensuring that the result is `>=0`.

```
// Update total assets to recognize that lenders wont be getting
// `result.badDebtRealized` back due to realized bad debt.
// Emit corresponding event recognizing bad debt.
if (result.badDebtRealized > 0) {
    _totalAssets = _totalAssets - result.badDebtRealized;
    emit BadDebtRecognized(result.badDebtRealized, liquidator);
}
```

Additionally, the ability to make `totalAssets` very small while just being enough to not underflow also allows to make it less than `_BASE_UNDERLYING_RESERVE` and `totalAssets >= _BASE_UNDERLYING_RESERVE` should help to protect against invariant manipulation:

```
// Check if we have enough underlying held to support the withdrawal.
// We add _BASE_UNDERLYING_RESERVE to the calculation to ensure that
// the market never actually runs out of assets and may introduce
// invariant manipulation.
```

Furthermore, realizing any amount of bad debt makes it so that `_BASE_UNDERLYING_RESERVE` shares can correspond to less than this amount of assets.

E.g. if 50% of assets are realized as bad debt, and everyone withdraws, then there are only `_BASE_UNDERLYING_RESERVE / 2` of assets left. Since this is fundamental to bad debt realization, and the scenario of low assets and high shares does not allow for the typical donation attack, this is considered noteworthy but not worthy of a mitigation or change in logic.

Recommended mitigation

It's recommended to address the donation issue by not counting donations toward *assetsHeld()*. Additionally, the **_totalAssets < result.badDebtRealized** case should be handled gracefully to prevent underflow (rounding can make total debt greater than total assets).

It is still possible for bad debt realization to make **totalAssets** significantly lower than **_BASE_UNDERLYING_RESERVE**, but this is an extreme case and may be acknowledged as a known risk that extreme bad debt can break cToken numerics (e.g., *_convertToShares()* can revert due to div-by-zero and rounding errors can accumulate due to a low exchange rate).

Team response

Fixed by commit [f10667c](#).

Mitigation review

The donation issue has been mitigated as recommended by not including them in *assetsHeld()*. However, the case of **_totalAssets < result.badDebtRealized** from rounding up debts has not been handled. In order to preserve **_BASE_UNDERLYING_RESERVE** assets, and to avoid the underflow in **_totalAssets**, the following change is recommended:

```
if (result.badDebtRealized > 0) {  
+   require(_totalAssets >= result.badDebtRealized + _BASE_UNDERLYING_RESERVE);  
   _totalAssets = _totalAssets - result.badDebtRealized;  
   emit BadDebtRecognized(result.badDebtRealized, liquidator);  
}
```

By making this change, bad debt can only be realized if sufficient assets remain, and additional bad debt must wait until more assets are deposited. Liquidators can adjust how much debt they liquidate to avoid reverts.

Team response

Fixed by commit [1933505](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-L-16: Missing accountPositions cleanup after liquidation can cause DoS

- **Category:** Logical issues
- **Source:** BorrowableCToken.sol, MarketManagerIsolated.sol
- **Status:** Acknowledged

Description

The **accountPositions** mapping of the market manager is updated in borrow and redeem flows, but it is untouched in liquidation flows. After liquidation, one, both, or none of the **borrow/collateral** tokens could have a **debt balance/collateral balance** of zero, which means

they should not be present in the mapping anymore. For example, when collateralizing, the position is added:

```
// On collateral posting:
// We need to flip their cooldown flag to prevent flashloan attacks.
accountAssets[account].cooldownTimestamp = block.timestamp;
// If `account` does not have a position in `collateralToken`,
// open one.
if (accountPositions[collateralToken][account] != 2) {
    accountPositions[collateralToken][account] = 2;
    accountAssets[account].assets.push(collateralToken);

    emit PositionUpdated(collateralToken, account, true);
}
```

Similarly, when borrowing:

```
// Check if the user already has an outstanding debt position in
// `debtToken`.
if (accountPositions[debtToken][account] != 2) {
    // The account does not have outstanding debt position in
    // `debtToken`, so add `debtToken` as an active position for
    // `account`.
    accountPositions[debtToken][account] = 2;
    accountAssets[account].assets.push(debtToken);

    emit PositionUpdated(debtToken, account, true);
}
```

However, during liquidation, the debt is reduced, and collateral is seized, but **accountPositions** is never updated.

The `_hypotheticalLiquidityOf()` function does mark positions for closure when balances reach zero:

```
if (snap.isCollateral) {
    // If there is no collateral posted and its not a
    // position to be modified, clean up the position
    // entry as the user was liquidated.
    if (snap.collateralPosted == 0) {
        // If there is no collateral posted and its not a
        // position to be modified, clean up the position
        // entry as the user was liquidated.
        if (action.cTokenModified != snap.asset) {
            positionsToClose[i] = true;
            if (result.positionClosureNeeded == 0) {
                result.positionClosureNeeded = 2;
            }
        }
    }
} else {
    // There is collateral posted in this cToken,
    // increasing collateral, or more simply the user
    // can take on more debt.
    maxDebt += _collateralValue(
        snap.collateralPosted,
        prices[i],
        10 ** snap.decimals,
        _tokenConfig[snap.asset].collRatio,
        true
    );
}
```

```

    }
  } else {
    // If they have a debt balance, increment their debt.
    if (snap.debtBalance > 0) {
      newDebt += _assetValue(
        snap.debtBalance,
        prices[i],
        10 ** snap.decimals,
        false
      );
    } else {
      // If there is no debt and its not a position
      // to be modified, clean up the position entry as the
      // user was liquidated (bad debt insolvency).
      if (action.cTokenModified != snap.asset) {
        positionsToClose[i] = true;
        if (result.positionClosureNeeded == 0) {
          result.positionClosureNeeded = 2;
        }
      }
    }
  }
}
}
}

```

However, this cleanup only occurs on the next borrow or redeem operation, not immediately after liquidation or repayment.

If a user has no borrows after liquidation, redeeming collateral should succeed regardless of whether the borrow token pricing fails. However, due to the missing **accountPositions** cleanup, there can be a wrong pricing dependency that leads to DOS if the error level is **CAUTION** or above. The system will attempt to price a token that is no longer active in the user's position, causing unnecessary failures.

Recommended mitigation

It's recommended to update **accountPositions** such as to maintain the invariant that **token is in accountPositions** $\Leftrightarrow$ **debt or collateral balance is greater than zero**.

As part of this, functions like `_closePositionsIfNeeded()` can be simplified. Given the restriction to **2 cTokens**, this function is not needed and **accountPositions** can be maintained without such general logic that supports **n>2 cTokens**.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Repayments that remove all of an account's debt now clean up the **accountPositions** mapping. Liquidations still don't perform the cleanup step, and this is intended to save gas costs.

Therefore, the invariant that was recommended has not been implemented, and `_closePositionsIfNeeded()` is still more verbose than necessary for a market with two tokens. Since the issue has been addressed in the repayment flow, and this is the main flow by which a user should expect their **accountPositions** to get cleaned up, the issue is considered fixed.

Update

After discussion between the team and the auditors, it was determined that pruning should be removed in all code paths, thus the finding can be considered acknowledged.

TRST-L-17: Bad debt can't be realized when the collateral token becomes worthless

- **Category:** Overflow issues
- **Source:** MarketManagerIsolated.sol
- **Status:** Fixed

Description

The **debtToCollateral** calculation can become a bottleneck when the collateral token loses its value, causing overflows that prevent liquidations and bad debt realization. The same issue affects the **sharesNeeded** calculation.

The vulnerable calculation for **debtToCollateral**:

```
uint256 debtToCollateral =
  (((aData.liqInc * tData.debtUnderlyingPrice * WAD_SQUARED_BPS_OFFSET) /
    tData.collateralSharesPrice) * tData.collateralDecimals) /
    tData.debtDecimals;
```

As an example of how this can overflow, consider the first part of the formula:

```
uint256 debtToCollateral =
  (((aData.liqInc * tData.debtUnderlyingPrice * WAD_SQUARED_BPS_OFFSET) /
    tData.collateralSharesPrice) * tData.collateralDecimals)
```

Suppose:

- **liqInc = 1e4**
- **debtUnderlyingPrice = 1e22**
- **collateralSharesPrice = 10000**
- **collateralDecimals = 1e24**

The result of the partial formula is **1e78**, causing a revert due to overflow. If *fullMulDiv()* were used, the subsequent division by **debtDecimals** would avoid the overflow and allow for a bigger value range to be supported.

Similarly, the **sharesNeeded** calculation is vulnerable:

```
uint256 sharesNeeded = _mulDiv(
  aData.debtBalance,
  debtToCollateral,
  WAD_SQUARED
);
```

Here is the **badDebt** calculation:

```
badDebt = FixedPointMathLib.fullMulDiv(
  _mulDiv(debtAmount, sharesNeeded, sharesPosted),
  WAD_SQUARED - _mulDiv(WAD_SQUARED, sharesPosted, sharesNeeded),
  WAD_SQUARED
);
```

A revert in the liquidation flow can create a situation where the debt token is healthy, in the sense that it maintains its value, but the collateral has become worthless. The system should be able to realize all outstanding debt as bad debt and let lenders withdraw what is left. However, due to overflows, bad debt cannot be realized, and lenders are stuck in a bank run scenario where liquidations are impossible.

Recommended mitigation

It's recommended to use `fullMulDiv()` / `fullMulDivUp()` for all calculations in the liquidation flow. Liquidations should be gas-optimized, but this is an area where a numeric issue in one **cToken** can spread to another **cToken**, so calculations should support as big a value range as possible. This approach does not guarantee that overflows can't occur (the result of calculations can still exceed the max value of a **uint256**), but it is likely a sufficiently robust solution for practical purposes.

As part of the refactoring to `fullMulDiv()` / `fullMulDivUp()`, the **badDebt** calculation can be simplified as such:

```
// debtAmount * sharesNeeded / sharesPosted * (1 - sharesPosted / sharesNeeded)
// = debtAmount * (sharesNeeded / sharesPosted - 1)
// = debtAmount * sharesNeeded / sharesPosted - debtAmount
badDebt = FixedPointMathLib.fullMulDivUp(debtAmount, sharesNeeded, sharesPosted) - debtAmount;
```

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-L-18: Removing asset from adaptor can revert if not configured as data feed

- **Category:** Configuration issues
- **Source:** BaseOracleAdaptor.sol, OracleManager.sol
- **Status:** Fixed

Description

In adaptors, assets can be added by elevated users through the `addAsset()` function. In the *OracleManager*, adaptors can be added as data feeds for assets. However, there is no guarantee that an adaptor must be a data feed of an asset, even if that asset is allowed in the adaptor.

This creates an issue when removing an asset from an adaptor. The `removeAsset()` function unconditionally calls `notifyFeedRemoval()` on the *OracleManager*:

```
function removeAsset(address asset) external {
    _checkElevatedPermissions();
    _checkSupportedAsset(asset);

    // Notify the adaptor to stop supporting the asset.
    delete isSupportedAsset[asset];
    _wipeAssetConfigs(asset);

    // Notify the Oracle Manager that we are going to stop supporting
```

```
// the asset.
CommonLib._oracleManager(centralRegistry).notifyFeedRemoval(asset);

emit AssetRemoved(asset);
}
```

The `notifyFeedRemoval()` function requires the adapter to be a data feed for the asset:

```
function notifyFeedRemoval(address asset) external {
    _checkIsApprovedAdaptor(msg.sender);
    _removeFeed(asset, msg.sender);
}
```

All the while, `_removeFeed()` will revert if the adapter is not actually configured as a feed for the asset:

```
function _removeFeed(address asset, address feed) internal {
    uint256 numFeeds = _checkFeeds(asset);

    // If theres two feeds, figure out which to remove,
    // otherwise we know the feed to remove is the first entry.
    if (numFeeds > 1) {
        // Check whether `feed` is a currently supported feed for `asset`.
        if (
            assetPriceFeeds[asset][0] != feed &&
            assetPriceFeeds[asset][1] != feed
        ) {
            revert OracleManager__NotSupported();
        }

        // We want to remove the first feed of two,
        // so move the second feed to slot one.
        if (assetPriceFeeds[asset][0] == feed) {
            assetPriceFeeds[asset][0] = assetPriceFeeds[asset][1];
        }
    } else {
        if (assetPriceFeeds[asset][0] != feed) {
            revert OracleManager__NotSupported();
        }
    }

    // We know the feed exists, but we cannot use `isApprovedAdaptor` as
    // we could have removed it as an approved adaptor prior.

    assetPriceFeeds[asset].pop();
}
```

If the adapter is not a data feed for the asset when `removeAsset()` is called, the transaction will revert with **OracleManager__NotSupported()**.

As a result, the configuration functionality is broken. However, the admin can still work around it by switching the *OracleManager* in the *CentralRegistry*, though this is a disruptive workaround.

Recommended mitigation

It's recommended to wrap the `notifyFeedRemoval()` call in a *try/catch* block and ignore the revert if the reason is `OracleManager__NotSupported()`.

Alternatively, manage adaptors through calls to *OracleManager* so that the dependency goes in one direction and the adaptors don't need to make any calls into *OracleManager*.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Since *OracleManager.notifyFeedRemoval()* still checks that **msg.sender** is an approved adaptor, the function call can still revert. It is recommended to **return** when **msg.sender** is not an approved adaptor.

Team response

Fixed by commit [6ea8779](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-L-19: VaultZapper leaks malicious approval

- **Category:** Approval issues
- **Source:** SimpleZapper.sol, BaseZapper.sol
- **Status:** Fixed

Description

The *swapAndDeposit()* function approves **outputToken** to a vault but never resets the approval. A malicious **cToken** could provide a malicious vault with a legitimate ERC20 as the underlying asset:

```
IVault vault = IVault(ICToken(cToken).asset());
address asset = address(vault.asset());

if (asset != swapAction.outputToken) {
    revert BaseZapper__UnderlyingTokenIsNotInputToken();
}

if (swapAction.inputToken == swapAction.outputToken) {
    outAmount = swapAction.inputAmount;
} else {
    // Execute swap into cToken asset.
    outAmount = SwapperLib._swapUnsafe(centralRegistry, swapAction);
}

SwapperLib._approveIfNeeded(
    swapAction.outputToken,
    address(vault),
    outAmount
);

// Deposit into vault.
outAmount = vault.deposit(outAmount, address(this));
```

The malicious vault could virtualize the deposit without consuming the approval, leaving the *VaultZapper* with an approval leak. While the zapper typically holds no funds between transactions, a malicious callback in flows like *redeemAndSwap()* could exploit the approval to transfer out victim tokens if **slippage** parameters still pass.

Recommended mitigation

It's recommended to validate **cTokens** and token addresses in zappers. In *swapAndDeposit()*, validate the **cToken** to ensure only trusted vaults are interacted with, and reset approvals after deposit.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Addresses of **cTokens** are now validated and approvals are reset correctly, thus the specific issue has been mitigated. The broader recommendation of validating token addresses overlaps with TRST-M-23 and is reviewed there. It has only been mentioned in the above recommendation to provide a complete list of missing validations, not because fixing TRST-L-19 requires it.

TRST-L-20: BaseOracleAdaptor.removeAsset() does not clear price guards

- **Category:** Logical issues
- **Source:** BaseOracleAdaptor.sol
- **Status:** Fixed

Description

The *removeAsset()* function removes the asset from both **inUSD=true** and **inUSD=false** configurations but does not remove the set price guard. If an asset is removed and later reintroduced, the old price guard remains active with potentially outdated configuration. Partial removal of configuration data creates inconsistent state.

Recommended mitigation

It's recommended that *BaseOracleAdaptor.removeAsset()* would call **disableGuardedPriceConfig(asset, true)** and **disableGuardedPriceConfig(asset, false)** to fully clear price guard configuration, while *disableGuardedPriceConfig()* should be made public to support the fix.

Team response

Fixed by commit [f10667c](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-L-21: AuctionManager verifies oracles and selectors separately

- **Category:** Privilege escalation issues

- **Source:** AuctionManager.sol
- **Status:** Fixed

Description

Oracle and selector are verified separately in `_preOpsCall()`:

```
// The called oracle must be whitelisted.  
AuctionManager(CONTROL).verifyOracleWhitelist(_oracle);  
// The update call data must be a valid function call.  
AuctionManager(CONTROL).verifyAllowedSelector(bytes4(_updateCallData));
```

A selector is only valid within the context of a specific oracle. `func()` of **oracle1** may be intended for whitelisting, but the same function in **oracle2** could be dangerous. The admin cannot express this granularity, allowing Fastlane Labs to potentially call restricted oracle functions.

Recommended mitigation

It's recommended to whitelist selectors per oracle address rather than globally, so a two-key mapping is advised.

Team response

Fixed by commit [48e5a91](#).

Mitigation review

Verified, the recommendation has been implemented.

TRST-L-22: AuctionManager `checkMetacallGasLimit=false` setting introduces griefing risk

- **Category:** Configuration issues
- **Source:** AuctionManager.sol
- **Status:** Acknowledged

Description

The **checkMetacallGasLimit=false** setting introduces additional trust assumptions for the bundler. This config is checked in *AtlasVerification*.

Per Atlas [comments](#):

If gas limit is too low, the bonded balance threshold checked may not cover all gas reimbursements.

If gas limit is too high, the bonded balance threshold checked could unexpectedly price out solvers.

From the Atlas [PR](#) where this flag was implemented:

NOTE: *If checkMetacallGasLimit is disabled, there is a risk that solvers should be aware of - that the bundler could set an unreasonably high gas limit which may prevent solverOps from executing if the solver doesn't have enough bonded at1ETH to cover the full gas limit + surcharges should they win.*

This allows the bundler to adversarially interfere with metacall execution by manipulating gas limits, even though metacalls should be guarded by the auctioneer's signature.

Recommended mitigation

It's recommended to assess whether the bundler's ability to adversarially determine gas limits presents unacceptable risk. Given that the bundler can interfere with metacall execution that should be protected by the auctioneer's signature, **checkMetacallGasLimit=true** should be preferred.

Team response

Acknowledged, Fastlane runs bundler and auctioneer so trust assumptions issue here is invalid.

Mitigation review

Official [answer stated bundler is much less trusted](#), so it can be acknowledged but technically valid.

Additional recommendations

TRST-R-1: Heartbeat grace period is not applied to custom heartbeat values

The **HEARTBEAT_GRACE_PERIOD** constant is defined as an extra buffer on top of the usual heartbeat interval in case of transaction congestion delaying an update:

```
/// @dev Extra time added to top end Oracle feed heartbeat incase of  
/// transaction congestion delaying an update.  
uint256 constant HEARTBEAT_GRACE_PERIOD = 60;
```

This grace period is correctly added to the **DEFAULT_HEARTBEAT** in all oracle adaptors:

```
uint256 public constant DEFAULT_HEARTBEAT =  
1 days + HEARTBEAT_GRACE_PERIOD;
```

However, when custom heartbeat values are provided via *addAsset()* functions across *ChainlinkAdaptor*, *RedstoneClassicAdaptor* and *RedstoneCoreAdaptor*, the grace period is not added. For example, in *ChainlinkAdaptor*:

```
config.heartbeat = uint24(heartbeat != 0 ? heartbeat : DEFAULT_HEARTBEAT);
```

The intention is that the input heartbeat parameter represents the pure heartbeat period from oracle documentation (e.g., 60 minutes for a typical Chainlink heartbeat), and **HEARTBEAT_GRACE_PERIOD** should be added on-chain. The buffer should not depend on the specific feed but on network congestion, which affects all feeds equally.

It is recommended to add **HEARTBEAT_GRACE_PERIOD** to custom heartbeat values.

TRST-R-2: In Zapper Interface, expectedShares can be ineffective and is not applied consistently

BaseZapper's *_enterCurvance()* function includes an **expectedShares** parameter that provides slippage protection when depositing assets into Curvance markets. However, this protection is absent from *BasePositionManager.onBorrow()*, which performs similar deposit operations.

While the share-to-asset exchange rate should remain within safe bounds under normal operation, and bad debt realization would increase the shares received (making the slippage check less effective), the inconsistency between these two entry points is unnecessary. It creates an asymmetry in how deposits are handled across different entry points.

It's recommended to add an **expectedShares** parameter to *BasePositionManager.onBorrow()* with a corresponding validation check, consistent with the implementation in *BaseZapper._enterCurvance()*. This ensures uniform protection across all deposit paths, even if the practical impact is limited.

TRST-R-3: Collateral can be withdrawn with regular allowance

When **userA** grants **userB** a cToken allowance, **userB** is able to withdraw **userA**'s collateral through functions that set **forceRedeemCollateral=true**. This escalates the trust assumptions beyond standard ERC20 allowance behavior.

Standard cToken ERC20 transfers (inherited from ERC20) always use **forceRedeemCollateral=false**, meaning they transfer non-collateralized shares first. This is the expected behavior when a user grants an ERC20 allowance. It should be assessed whether the semantics of an ERC20 allowance are such that the approved **userB** can choose whether to transfer/withdraw uncollateralized or collateralized shares.

TRST-R-4: Flashloans enabled on uninitialized markets allow exchange rate manipulation

When a new *BorrowableCToken* is created, there is a window of time between deployment and when *MarketManagerIsolated.listTokens()* is called where the market is not yet fully initialized. During this period, an attacker can call *flashLoan()* on the *BorrowableCToken* to manipulate **totalAssets** through flash loan fees. When *_initializeDeposits()* is later called, it sets **_totalAssets** to the pre-defined value of 77777, but any historical flash loan activity has already occurred.

Allowing flash loans on markets with zero shares creates an unnecessary attack surface during the sensitive initialization period, even though there is no identified impact. Before *listTokens()*, the token can never be used by users (everything blocked except flashloan). After *listTokens()*, the assets and shares are both hardcoded to 77777, making historic flashloans equivalent to donations at that point.

It's recommended to disable flash loans when **totalSupply() == 0** to prevent any potential manipulation during market initialization. This ensures flash loan functionality is only available after the market has been properly initialized and tokens listed.

TRST-R-5: Slippage check prevents deleveraging for accounts in bad debt

In *BasePositionManager.checkSlippage()*, which is executed before every *leverage()/deleverage()* operation, debt is subtracted from collateral. When an account is in a state where **collateral < debt**, this subtraction causes an underflow that prevents the slippage check from passing, blocking both leveraging and deleveraging operations.

While leveraging should reasonably be blocked for accounts in bad debt, deleveraging could actually restore the account to a healthy state. However, the underflow in *checkSlippage()* prevents this recovery path. Consider a scenario where **collateral = \$99** and **debt = \$100** - an oracle pricing error of just 1% could make it possible to repay all debt and restore the account to health, but the slippage check blocks this operation.

This is an edge case scenario since **collateral < debt** indicates the account is already in bad debt, which means multiple layers of protection (liquidation mechanisms, bad debt prevention) have already failed. In this scenario, the account starts with negative **valueIn** and would end with non-negative **valueOut**, resulting in a negative slippage calculation.

It's recommended to set **valueIn** to **max(valueIn, 0)** in the slippage calculation to prevent underflow. While supporting accounts in bad debt is low priority given that it assumes all bad debt protections have failed and the user wasn't liquidated, allowing deleveraging in this scenario could provide a recovery path without introducing additional risk.

TRST-R-6: Fee validation is bypassed when swap validation returns early

In *BasePositionManager*, fees are applied to **borrowAssets** and **collateralAssets**. The swap validation typically acts as an implicit check against the fee since it validates the input amount. However, when swap validation encounters early returns, this fee validation is bypassed.

The fee is set in *CentralRegistry* and is capped at **2%**. The overall calculation is only dependent on a prior user-provided input (amount of assets **borrowed/redeemed**) and the global fee, which should very rarely change. It can be considered a feature that a change in the fee causes transactions to revert (when there is no early return), effectively requiring users to have a specific fee in mind when submitting transactions.

The general pattern of performing a *mulDiv()* calculation and then assuming the result matches a user-provided value is error prone, as it could unintentionally revert. However, in this case the lack of validation when early returns occur means users don't get the same implicit protection against unexpected fee changes.

It's recommended to add explicit fee validation even when swap validation early returns, or document that fee changes may affect transaction success differently depending on execution path. This ensures consistent behavior across all code paths.

TRST-R-7: Inconsistent use of `_getTotalAssets()`

CTokens have a `_getTotalAssets()` function and a **_totalAssets** variable. For **cTokens** with yield, the function includes the vested assets in its calculation. Whenever interest is accrued through `_accrueIfNeeded()`, the **_totalAssets** variable is updated to include assets to vest. After accrual, **_totalAssets** can be used directly without needing to call `_getTotalAssets()`.

However, there is no consistent pattern in the codebase for when to use **_totalAssets** versus `_getTotalAssets()`. For example, `_mint()` uses **_totalAssets** directly while `_deposit()` uses `_getTotalAssets()`. This inconsistency could lead to subtle bugs if the wrong variant is used in a context where accrual has or hasn't occurred.

It's recommended to adopt a consistent policy for using **_totalAssets** and `_getTotalAssets()` throughout the codebase.

TRST-R-8: OracleManager doesn't round according to getLower

Prices in *OracleManager* are queried with a **getLower** flag that signals the rounding direction that should be preferred. However, all price calculations just round down, ignoring this flag. Based on the protocol's assumptions about accepted price ranges, the maximum price in USD is **1e25** in WAD precision and the minimum is **5e14**. Converting these USD assumptions into native token prices yields value ranges for all possible price units. The maximum price in native (with **native = \$5e-4** and **token = \$1e7**) is **2e28**, while the minimum price in native (with **native = \$1e7** and **token = \$5e-4**) is **5e7**. When rounding, **1 wei** is at most **1 / 5e7** of the whole price, making the impact of incorrect rounding minimal.

However, there are two reasons why the rounding should still be improved. First, *OracleManager* is designed as a generalized oracle layer, so having code that works correctly in general with in-depth defense against rounding exploits is valuable.

Second, the information is already available via the **getLower** flag, so utilizing it doesn't introduce additional complexity or parameters.

The affected calculations are:

```
price = (price * ICToken(cToken).exchangeRate()) / WAD;
```

```
function _convertNativeUSD(
    uint256 currentPrice,
    uint256 conversionRate,
    bool currentlyInUSD
) internal pure returns (uint256) {
    if (!currentlyInUSD) {
        // The price denomination is in native token and we want USD.
        return (currentPrice * conversionRate) / WAD;
    }

    // The price denomination is in USD and we want native token.
    return (currentPrice * WAD) / conversionRate;
}
```

It's recommended to use the **getLower** flag to determine rounding direction in the affected calculations, rounding down when **getLower** is **true** and rounding up when it is **false**.

TRST-R-9: Redstone Core Adaptor allows zero prices when price guard is enabled

It is possible to sign a zero price and write it to Redstone Core when a price guard is enabled. This occurs because **price == 0** is only validated after the call to `_adjustPrice()`, which can return **minPrice** instead of zero when a price guard is configured.

This differs from the Chainlink and Redstone Classic adaptors, which effectively validate **price == 0** two times: before `_adjustPrice()` and after `_adjustPrice()` (by calling `_verifyData()`).

The Redstone documentation does not explicitly mention that **price == 0** should be checked, and their example contracts do not perform this validation. However, checking for zero prices serves as a sanity check since signing zero data may indicate a common backend mistake.

TRST-R-10: AuctionManager ETH transfer reentrancy

In *AuctionManager._distributeRevenue()*, revenue destinations receive a callback from the ETH transfer:

```
if (fastlaneSplit > 0) {
    SafeTransferLib.safeTransferETH(fastlaneRevenueDestination, fastlaneSplit);
}
if (curvanceSplit > 0) {
    SafeTransferLib.safeTransferETH(curvanceRevenueDestination, curvanceSplit);
}
```

Consider the following scenario:

- Governor calls *setFastlaneSplit()*
- Malicious Fastlane revenue destination (which is also a bundler) calls *Atlas.metacall()* during the ETH transfer callback
- The metacall triggers *accumulateRevenue()* before **accumulatedRevenue** has been reset to zero
- When everything returns, **accumulatedRevenue** is incorrectly set to zero and the new revenue from the metacall is lost

While this is a contrived scenario requiring the Fastlane destination to also be a bundler, it demonstrates a violation of the Checks-Effects-Interactions (CEI) pattern. It's recommended to follow the CEI pattern by moving **delete accumulatedRevenue** before the ETH transfers in *_distributeRevenue()*.

TRST-R-11: DAOTimelock duplicated code

In *DAOTimelock.updateRoles()*, when the DAO address changes, the new DAO address is granted the **PROPOSER_ROLE**, **EXECUTOR_ROLE**, and **CANCELLER_ROLE**, and the **_DAO_ADDRESS** state variable is updated. However, this logic is duplicated:

```
function updateRoles() external {
    address registryDaoAddress = centralRegistry.daoAddress();
    address timelockDaoAddress = _DAO_ADDRESS;

    if (registryDaoAddress != timelockDaoAddress) {
        _revokeRole(PROPOSER_ROLE, timelockDaoAddress);
        _revokeRole(EXECUTOR_ROLE, timelockDaoAddress);
        _revokeRole(CANCELLER_ROLE, timelockDaoAddress);

        _grantRole(PROPOSER_ROLE, registryDaoAddress);
        _grantRole(EXECUTOR_ROLE, registryDaoAddress);
        _grantRole(CANCELLER_ROLE, registryDaoAddress);
    }
}
```

```
_DAO_ADDRESS = registryDaoAddress;  
_grantRole(PROPOSER_ROLE, registryDaoAddress);  
_grantRole(EXECUTOR_ROLE, registryDaoAddress);  
_grantRole(CANCELLER_ROLE, registryDaoAddress);  
_DAO_ADDRESS = registryDaoAddress;  
}
```

The four lines that grant the roles and update **_DAO_ADDRESS** are repeated, making them no-ops on the second execution.

TRST-R-12: BaseZapper should inherit Multicall

BaseZapper does not inherit *Multicall*, preventing users from batching multiple operations together. This differs from other protocol contracts that support multicall functionality, which allows users to combine multiple actions (such as price updates and zaps) in a single transaction for better gas efficiency and atomicity.

However, implementing this recommendation requires careful consideration since zapper logic reads **msg.value**. In a multicall context where multiple calls are executed in sequence, **msg.value** remains constant across all delegatecalls, which can break the zapper logic.

It's recommended to add *Multicall* inheritance to *BaseZapper* to enable batched operations but ensure that the implementation properly handles **msg.value** in the multicall context.

TRST-R-13: Misleading naming of delegation check

The *_checkDelegate()* function does not call *CentralRegistry.checkDelegationDisabled()*. The naming "delegations disabled" suggests that all delegations are blocked, but the actual behavior only prevents new delegations from being created while allowing existing delegations to continue functioning. It's recommended to rename *checkDelegationDisabled()* to something more indicative of its behavior.

TRST-R-14: VaultAggregator should cast to ERC4626 Interface

VaultAggregator uses the *ICToken* interface to call *convertToAssets()* on external vaults:

```
result = ICToken(vault).convertToAssets(WAD);
```

This is misleading since the contract does not expect **vault** to be a **cToken**. The *ICToken* interface is being used purely for convenience because it happens to have a *convertToAssets()* function signature that matches the ERC4626 standard.

It's recommended to create a dedicated ERC4626 interface that accurately represents the expected contract interface.

TRST-R-15: Price guard configuration sanity checks

In `setGuardedPriceConfig()` several validation checks are missing. First, it should check that **basePrice > 0** such that this function can only be used to enable price guards. To disable them, `disableGuardedPriceConfig()` must be called explicitly. This enforces a clear separation between enabling and disabling price guards. Second, it should be enforced that **ips == 0** implies **timestampStart == 0** since the **timestampStart** is only used when **ips > 0**. This prevents misconfiguration where a timestamp is set but never utilized. Third, the function should check whether the price has returned an error before using it to validate **guardedMinPrice**:

```
PricingResult memory result = this.getPrice(asset, inUSD, true);

// Having a minimum price above the current price does not make sense,
// implying that asset price behaves differently than our PriceGuard
// assumes.
// This will simply return `minPrice` if ips == 0 so can use internal
// function to handle both cases.
uint256 guardedMinPrice =
    _guardedPrice(block.timestamp - timestampStart, ips, minPrice);

if (guardedMinPrice > result.price) {
    revert BaseOracleAdaptor__MinPriceAboveCurrentPrice();
}
```

Additionally, a sanity check for **guardedBasePrice > higher price** may be added to ensure the base price is above the current market price.

Mitigation review

By commit [f10667c](#), an issue has been identified when updating from **ips > 0** to **ips = 0** due to a revert when checking **pg.timestampStart > timestampStart**. This has been mitigated by commit [6ea8779](#). Furthermore, it is also recommended to establish the invariant that **timestampStart==0 ⇔ ips==0** by implementing the change below. Without this invariant, the new [timestampStart > 0 condition](#) allows to decrease the timestamp when **ips > 0**.

```
if (ips == 0) {
    if (timestampStart != 0) {
        revert BaseOracleAdaptor__InvalidTimestamp();
    }
} else {
    if (
        timestampStart > block.timestamp ||
-        block.timestamp - timestampStart < _MINIMUM_TIMESTAMP_BUFFER
+        block.timestamp - timestampStart < _MINIMUM_TIMESTAMP_BUFFER ||
+        timestampStart == 0
    ) {
        revert BaseOracleAdaptor__InvalidTimestamp();
    }
}
```


TRST-R-16: Inconsistent use of `fullMulDiv()` in `BaseOracleAdaptor`

In `_guardedPrice()` price calculations are performed using standard multiplication and division:

```
r = FixedPointMathLib.mulDiv(price, ((timePassed * ips) + WAD), WAD);
```

However, `_adjustPrice()` uses `fullMulDiv()` for its price calculations to prevent overflow and ensure precision. This inconsistency means that `_guardedPrice()` could overflow with large price values, while similar calculations elsewhere in the oracle system are protected against this issue.

It is recommended to update `_guardedPrice()` to use `fullMulDiv()` for its calculations, matching the pattern established in `_adjustPrice()`.

TRST-R-17: Vault aggregators are incompatible with `RedstoneClassicAdaptor`

The current `VaultAggregator` and `WstETHAggregator` contracts are not compatible with `RedstoneClassicAdaptor` since they don't implement `getDataFeedId()`, which is called by `RedstoneClassicAdaptor` during the asset addition process. This limits the flexibility of using these wrapped aggregators with Redstone Classic price feeds, forcing deployments to use only Chainlink or other oracle providers that don't require this interface.

`RedstoneClassicAdaptor` validates the data feed ID during asset configuration to ensure the correct feed is being used:

```
if (Bytes32Helper.toBytes32(id) != IRedstone(feedProxy).getDataFeedId()) {  
    revert BaseOracleAdaptor__InvalidConfig();  
}
```

Since wrapped aggregators transform prices from underlying feeds, they should be able to pass through the data feed ID from their underlying Redstone aggregator when one is used. It is recommended to implement separate child wrapped aggregators that inherit from the existing `VaultAggregator` and `WstETHAggregator` contracts and pass through the data feed ID from the underlying aggregator.

TRST-R-18: `VaultZapper` and `NativeVaultZapper` don't override `redeemSwapAndDeposit()`

The vault zappers override `swapAndDeposit()` to handle deposits into ERC4626 collateral assets efficiently. However, they do not override `redeemSwapAndDeposit()` even though, like `swapAndDeposit()`, this function also makes a deposit into the collateral asset. In the case of `redeemSwapAndDeposit()`, the deposit is also used to "build deposits up", so the override is missing. This creates a usability issue where users cannot efficiently use the `redeemSwapAndDeposit()` flow with ERC4626 vault collateral assets. It is recommended to add overrides for `redeemSwapAndDeposit()` in vault and native vault zappers to provide consistent handling of ERC4626 deposits across all zapper entry points.

TRST-R-19: MAX_LIQUIDATION_INCENTIVE should be applied to liqIncHard

In *updateTokenConfig()*, **MAX_LIQUIDATION_INCENTIVE** is documented as capping the liquidation incentive. However, it only caps the **liqIncMax** and does not cap **liqIncHard**.

It is recommended to either add a check that **liqIncHard** $\leq$ **MAX_LIQUIDATION_INCENTIVE** in *updateTokenConfig()* for consistency, or update the documentation to clarify that **MAX_LIQUIDATION_INCENTIVE** only applies to auction-based liquidations.

TRST-R-20: Insufficient range for minPrice in BaseOracleAdaptor

In *BaseOracleAdaptor*, **minPrice** is stored as **uint80** with a maximum value of approximately **1.2e24**. In USD terms with WAD precision, this would represent around **\$1.2 million**, which is below the protocol's assumed price range for high-value assets like WBTC that can reach **\$1e7** according to the token risk and assumptions.

Additionally, *BaseOracleAdaptor* operates before prices are normalized to USD, meaning prices at this stage can be denominated in native tokens rather than USD. If the native token trades below **\$1**, the **uint80** type would be insufficient to represent high-value asset prices at even lower USD token prices. For example, with a native token at **\$0.50**, the maximum representable price would be effectively halved in dollar terms.

It is recommended to increase the size of **minPrice** to accommodate the full range of expected asset prices in both USD and native token denominations, or to document the limitation.

TRST-R-21: Auction buffer unexpectedly applies to cHard

The documentation states that only **cSoft** is multiplied by the **auctionBuffer**:

```
/// @param auctionBuffer The current buffer that `cSoft` is multiplied
/// against, 10 bps, or 0 if not an auction-based
/// liquidation.
```

However, the implementation also applies **auctionBuffer** to **cHard**:

```
result.cSoft = _mulDiv(result.cSoft, AUCTION_BUFFER, BPS);
result.cHard = _mulDiv(result.cHard, AUCTION_BUFFER, BPS);
```

While the impact of 10 BPS is minor, the inconsistency between documentation and implementation suggests either a documentation error or an unnecessary application of the buffer to the hard liquidation threshold.

It is recommended to either remove the **auctionBuffer** multiplication from **cHard** to match the documented behavior or update the documentation to clarify that both collateral factors are adjusted by the auction buffer.

TRST-R-22: Approval not correctly reset in BasePositionManager

In two instances in *BasePositionManager*, approval is not correctly reset after use. For the first one, the approval reset uses the wrong token and spender:

```
SwapperLib._removeApprovalIfNeeded(  
    debtAsset,  
    address(borrowableCToken)  
);
```

For the second, the approval is not reset at all:

```
SwapperLib._approveIfNeeded(collateralAsset, address(cToken), assets);
```

It must be noted that these are not the only instances, and the remaining instances have been discussed in earlier findings. It is recommended to ensure all approvals are correctly reset to zero after use, with the correct token and spender parameters.

TRST-R-23: Chain removal should be protected by elevated permissions

Cross-chain logic is out of scope, but in general, removing a chain can have unintended consequences due to messages that are still in-flight or state getting stuck on the removed chain. Thus the *removeChain()* function in *CentralRegistry* should be protected by elevated permissions, rather than the lower privileged DAO permissions.

```
function removeChain(  
    uint256 chainId,  
    address expectedMessagingHub,  
    address expectedVotingHub  
) external {  
    // Lower permissioning on removing chains as it will reduce risk to  
    // the system.  
    _checkDaoPermissions();
```

TRST-R-24: Validation improvements

Several validation checks are missing throughout the codebase:

- There are no validations on **closeFactorMin** and **closeFactorMax** in *updateTokenConfig()*. Also, there's no check that **liqIncMin > 0**, if desired.
- **action.slippage** is never validated in *SwapperLib*:

```
if (slippage > action.slippage || slippage > cr.slippageLimit()) {
```

It's recommended to add a sanity check to revert when **action.slippage > BPS**.

- If **closeFactorMin=0** and an *AuctionManager* sets its **closeFactor** to zero, this selects regular liquidation parameters:

```
if (aData.closeFactor == 0 || aData.liqInc == 0) {
```

This creates an edge case where auctions with tight parameter limits can abuse the zero-setting to break out of their limits. It's recommended to prevent **closeFactorMin** from being set to zero or handle the zero case explicitly.

- In *MarketManagerIsolated*, the check for **c.liqIncHard > MAX_LIQUIDATION_INCENTIVE** is missing:

```
c.liqIncMax > MAX_LIQUIDATION_INCENTIVE
```

It's recommended to add validation that **c.liqIncHard <= MAX_LIQUIDATION_INCENTIVE**.

- In *CommonLib*, both **0xEeeeeEeeeEeEeeEeEeEEEEEEEEEEEEEEEE** and **0x0** are accepted for native ETH, but Odos router only considers **0x0** as native. When checking Odos calldata, input and output tokens should not be **0xEeeeeEeeeEeEeeEeEeEEEEEEEEEEEEEEEE**, as Curvance contracts would expect to work with native, but Odos will treat it as a normal token.
- In *addChain()*, **messagingToGETHChainId[config.messagingChainId]** is set to the **chainId** without checking if **messagingChainId** is already used by another chain. It's recommended to require **messagingToGETHChainId[config.messagingChainId] == 0**. As a second check, **chainId != 0** should be added to prevent confusion between non-existent mappings and zero mappings.
- In *MarketManager.canSeize()*, it is never checked that the inputs are not in reversed order. For integrations, the impact is unknown. It's recommended to validate input ordering.

Mitigation review

By commit [f10667c](#), the Odos calldata checker checks that the input and output token are not equal to a constant **0xEe...eE**. All other validations have either been implemented or acknowledged. For the Odos calldata checker, it is recommended to not check a constant value but to set the value in the constructor to account for different native token representations on different chains. This has been implemented by commit [6ea8779](#).

TRST-R-25: Redundant liquidation pause flags in MarketManagerIsolated

The **liquidationPaused** and **seizePaused** flags are redundant together since both are only and always checked in *BorrowableCToken._liquidate()* (downstream in *canLiquidate()* / *seize()*).

There is no code path where one can be triggered without the other, making the separation unnecessary.

```
// Both flags checked in same execution path
if (liquidationPaused) revert();
// ...later in same flow...
if (seizePaused) revert();
```

It is recommended to keep **liquidationPaused** and remove **seizePaused**, as liquidation adequately describes the entire operation including the seize step.

TRST-R-26: Set `_useVirtualShares()` to false

The ERC4626 `decimals()` implementation uses **useVirtualShares=true** by default, which adds `_decimalsOffset()`:

```
function decimals() public view virtual override(ERC20) returns (uint8) {
    if (!_useVirtualShares()) return _underlyingDecimals();
    return _underlyingDecimals() + _decimalsOffset();
}
```

Since `_decimalsOffset()` is **0** by default in *BaseCToken* and not overridden, the code takes the longer execution path for no benefit. This affects not just `decimals()` but all conversion flows like `convertToShares()` which also branch on **useVirtualShares**. Therefore, it is recommended to set **useVirtualShares=false** in *BaseCToken*.

TRST-R-27: Incomplete price validation in token configuration checks

MarketManagerIsolated.updateTokenConfig() checks that the cToken price is not returned with **error code 2** but only validates one set of input parameters. Due to `getPrice()` accepting multiple inputs (**getLower** and **inUSD**), validating one set is insufficient. It is theoretically possible that there's an error with **getLower=false** even though **getLower=true** succeeds.

The same issue exists in *OracleManager._addFeed()*, where it's more significant since native and USD feeds can hit entirely different aggregators (e.g., in Chainlink). Checking just one feed can give a false positive when the other feed configuration is invalid.

It is recommended to check **errorCode** for both **getLower=true** and **getLower=false** to ensure price feeds work correctly in all configurations or document the loophole if only one direction is intentionally validated. The **inUSD=false** case can be skipped as it is never used in core markets.

Mitigation review

By commit [f10667c](#), the mitigation is missing from *BaseOracleAdaptor.setGuardedPriceConfig()*. It has been addressed by commit [6ea8779](#).

TRST-R-28: `RedstoneCoreAdaptor.getPrice()` should return price even if error is true

OracleManager won't use **result.price** if **result.hadError = true**, so there is no impact to the observation that *RedstoneCoreAdaptor._getPrice()* does not set **result.price** when an error is flagged. However, this is inconsistent with *RedstoneClassicAdaptor* and *ChainlinkAdaptor*. Both set **result.price** before flagging errors.

It is recommended to either adopt a consistent policy across all adaptors (always set price before error checks or always leave it as **0** on error) or document the rationale for the different approaches. For *RedstoneCoreAdaptor*, the price assignment (**result.price = config.price**) could simply be moved up to where **result.inUSD** is set for consistency with other adaptors.

TRST-R-29: *SwapperLib* should round up slippage

The slippage calculation in *SwapperLib* rounds down:

```
uint256 slippage = FixedPointMathLib.mulDiv(
    valueIn - valueOut,
    BPS,
    valueIn
);
```

Slippage represents value loss and hence should be overestimated for safety. It is recommended to round up the slippage calculation using *mulDivUp()*.

TRST-R-30: *SwapperLib* underestimates `outAmount` when `inputToken` equals `outputToken`

If **inputToken == outputToken**, the calculated **outAmount** is underestimated:

```
outAmount = CommonLib._balanceOf(outputToken) - balanceBefore;
```

While *SwapperLib* is not currently used in contexts where **inputToken == outputToken**, the logic in *SwapperLib* itself breaks under this condition. It is recommended to revert on **inputToken == outputToken** since *SwapperLib* should never be used with such inputs.

TRST-R-31: Unnecessary swap action required when `debtAsset` equals `collateralAsset`

SimplePositionManager._swapCollateralAssetToDebtAsset() requires exactly one swap action even though an arbitrary length could be supported:

```
if (swapActions.length != 1) {
    revert BasePositionManager__InvalidParam();
}
```

In the case where **debtAsset == collateralAsset**, requiring a swap action is unnecessary and forces users to pass unnecessary swap data. It is more efficient to require **length = 0** instead when no actual swap is needed.

TRST-R-32: Document that slippage check in BasePositionManager is sanity check, not security guarantee

The *checkSlippage()* function measures **slippage** by comparing the change in **collateralValue - debtValue**:

```
modifier checkSlippage(address account, uint256 slippage) {
    // Scoping to avoid stack too deep.
    {
        address[] memory assets = marketManager.assetsOf(account);
        uint256 numAssets = assets.length;
        IBorrowableCToken asset;

        for (uint256 i; i < numAssets; ++i) {
            asset = IBorrowableCToken(assets[i]);
            if (asset.isBorrowable()) {
                asset accrueIfNeeded();
            }
        }
    }

    (uint256 collateralBefore, , uint256 debtBefore) = marketManager
        .statusOf(account);
    uint256 valueIn = collateralBefore - debtBefore;

    _;

    (uint256 collateralAfter, , uint256 debtAfter) = marketManager
        .statusOf(account);
    uint256 valueOut = collateralAfter - debtAfter;

    // If there was slippage, make sure its within slippage tolerance.
    if (valueIn > valueOut) {
        if ((valueIn - valueOut) > _mulDiv(valueIn, slippage, WAD)) {
            revert BasePositionManager__InvalidSlippage();
        }
    }
}
```

Even if slippage check implementation issues are resolved, deviation thresholds introduce an error into the slippage calculation (see TRST-H-1). Therefore, it should be documented that this slippage check is primarily a sanity check rather than a security guarantee that users should rely on for protection.

TRST-R-33: Cooldown reduction side effects should be documented

When a user lowers their cooldown via *setCooldown()*, transfers and delegations become disabled for the duration of the old cooldown period. A user might expect that if they already

have delegations or transfers enabled, then lowering the cooldown won't have an effect - after all, these permissions are already active and so a phishing attack vector already exists. However, the system intentionally disables these permissions during the transition period.

This behavior could be counterintuitive for users who don't understand that reducing their cooldown will temporarily disable active permissions. It is recommended to document this behavior clearly in both the code comments and user-facing documentation.

TRST-R-34: Unnecessary WAD multiplication and division in leverage calculation

In the leverage calculation the following calculation takes place:

```
result = _mulDiv(  
  _mulDiv(maxLeverage, WAD, price),  
  10 ** IERC20(debtAsset).decimals(),  
  WAD  
);
```

WAD cancels out between the inner and outer multiplication/division operations. It is recommended to simplify the calculation by removing the redundant **WAD** operations as below:

```
result = _mulDiv(maxLeverage, 10 ** IERC20(debtAsset).decimals(), price)
```

TRST-R-35: Unify protocol interest fee config

CentralRegistry.setProtocolInterestFee() sets per-market interest fees, but this value is only read once in *BorrowableCToken*'s constructor. After deployment, **cTokens** can independently update their interest fee via *setInterestFee()*.

It is recommended that interest fee configuration is entirely maintained within the central registry, and to avoid this duplication of config / setters.

TRST-R-36: Inconsistent msg.value validation and unnecessary receive() in BaseZapper

Some Zapper functions are **payable** and the path where **msg.value** is utilized checks that **msg.value** matches the expected **inputAmount**:

```
if (inputAmount != msg.value) {  
  revert BaseZapper__ExecutionError();  
}
```

However, for paths where **msg.value** is not utilized, only *NativeVaultZapper.swapAndDeposit()* has a check, and this is only for the **inputToken == wrappedNative** case:

```
if (msg.value > 0) {
```



```
    revert BaseZapper__ExecutionError();  
}
```

Other functions that don't use **msg.value** lack this validation.

It is recommended that for any execution path that does not utilize **msg.value**, it is checked that **msg.value == 0** to prevent accidental ETH loss.

Additionally, since only *NativeVaultZapper* is expected to receive native tokens from swaps, the *receive()* function should be implemented at the level of *NativeVaultZapper*, not at the level of *BaseZapper* as it currently is.

TRST-R-37: Dead code for swapPermit2 in OdosCalldataChecker

The code for *swapPermit2()* in *OdosCalldataChecker* is dead code since the Odos router checks that the signature is signed by **msg.sender**. Curvance contracts don't implement ERC1271, so they can't sign signatures that would pass Odos router validation.

It is recommended to remove the *swapPermit2()* validation code from *OdosCalldataChecker* since it cannot be used by Curvance contracts.

TRST-R-38: Introduce lower max for multiplier

DynamicIRM._MAXIMUM_VERTEX_MULTIPLIER_MAX is set to **type(uint96).max (~7.9e28)**, which far exceeds any realistic interest rate scenario. While numerically safe within the IRM's arithmetic, this extreme ceiling allows misconfiguration and thus interest rates beyond safe bounds in the core market.

It's recommended to reduce **_MAXIMUM_VERTEX_MULTIPLIER_MAX** to **50e18** to reflect reasonable governance bounds and prevent accidental misconfigurations that push interest multipliers into extreme ranges.

TRST-R-39: Insufficient precision in DynamicIRM threshold derivation

The parameters **increaseThresholdStart** and **decreaseThresholdEnd** are derived from **vertexStart** (in BPS) but stored in **uint16** BPS. A 1 BPS change in **vertexStart** changes **thresholdLength** by **5e13**, but dividing by **1e14** loses this precision, the minimum representable change is **1e14**. This causes a rounding error up to **~2.5% (1/50)**.

It's recommended to add more precision to **increaseThresholdStart** and **decreaseThresholdEnd**. Combining this recommendation with TRST-R-38, **vertexMultiplierMax** may be stored in **uint80**, and the type for vertex thresholds may be increased to **uint24**.

TRST-R-40: LiquidityManagerIsolated `_collateralValue()` should round like `_assetValue()`

In `_collateralValue()` it always rounds down by using `_mulDiv()` when applying **collRatio**, but the function is called with both **increasesCollateral=true** and **increasesCollateral=false**. While conditional rounding is correctly applied inside `_assetValue()`, the outer multiplication by **collRatio** always rounds down regardless of direction.

It's recommended to apply conditional rounding in `_collateralValue()` when multiplying by **collRatio** and use `mulDivUp()` when **increasesCollateral=false** to match the rounding discipline in `_assetValue()`.

TRST-R-41: BasePositionManager `leverage()` and `deleverage()` should check **cToken** is listed before interaction

In `_leverage()` there is a call to `BorrowableCToken.borrowForPositionManager()` and `maxRemainingLeverageOf()` without first verifying that **borrowableCToken** is listed in the market manager. Similarly, `_deleverage()` calls `BaseCToken.withdrawByPositionManager()` without checking if **cToken** is listed. This allows interactions with arbitrary untrusted contracts.

It's recommended to validate that **borrowableCToken** and **cToken** are listed via `marketManager.isListed()` before any external calls in `_leverage()` and `_deleverage()`.

TRST-R-42: **MAX_DEBT_CAP** is inconsistent with `uint136` **totalSupply** assumption

The protocol assumes **totalSupply** fits into `uint136` based on token price and market cap constraints (\$43T market cap to overflow). However, **_MAX_DEBT_CAP** is set to `type(uint160).max`, which is inconsistent with this assumption and allows debt caps that exceed the assumed safe **totalSupply** bounds.

it's recommended to change **_MAX_DEBT_CAP** to `type(uint136).max` to align with the **totalSupply** assumptions.

TRST-R-43: Use custom error instead of underflow revert when removing too much collateral

`BaseCToken.withdrawCollateral()` can underflow when the withdrawal amount exceeds posted collateral. When **forceRedeemCollateral=true**, `canRedeemWithCollateralRemoval()` sets **collateralRedeemed = shares** without checking if **shares > collateralPosted[owner]**. The subsequent `_removeCollateral()` call then underflow when subtracting **collateralRedeemed** from the account's balance.

it's recommended to add an explicit check in `_canRedeemWithCollateralRemoval()` that **collateralRedeemed** `<= collateralPosted` and **revert** with a descriptive error instead of relying on underflow.

TRST-R-44: Inconsistent rounding in protocol fee share calculation

The **protocolFee** is rounded up using `mulDivUp()`, but **protocolFeeShares** is rounded **down** using `_mulDiv()`. This is inconsistent, and if the protocol fee rounds up to capture more value, the shares calculation should also round up to ensure the protocol receives shares corresponding to the full rounded-up fee.

```
uint256 protocolFee = FixedPointMathLib.mulDivUp(
    assetsToVest,
    interestFee,
    BPS
);
// If theres fees we need to mint new shares for the protocol.
if (protocolFee > 0) {
    // We can calculate how many shares the protocol should receive
    // from its fee on assetsToVest by using the formula:
    // (fee * totalSupply) / (totalAssets + assetsToVest - fee).
    // This means that that shares minted will result in an exchange
    // rate matching the amount of vested assets lenders should
    // benefit from.
    uint256 protocolFeeShares = _mulDiv(
        protocolFee,
        totalSupply(),
        cachedTa + assetsToVest - protocolFee
    );
    // Cache `daoAddress` then mint shares to dao operator address.
    address daoAddress = centralRegistry.daoAddress();
    _mint(daoAddress, protocolFeeShares);
    _afterDepositAction(protocolFeeShares, daoAddress);
}
```

it's recommended to use `mulDivUp()` for the **protocolFeeShares** calculation to match the rounding direction of **protocolFee**.

TRST-R-45: OracleManager should not be called with address(0) asset

The protocol treats native tokens as both **address(0)** and **0xEeeeeEeeeEeEeeEeEeEEEEEEEEEEEEEEEEEE** via `CommonLib._isNative()`, but `OracleManager` only defines **native** = **0xEeeeeEeeeEeEeeEeEeEEEEEEEEEEEEEEEEEE** and configures price feeds for this address.

External protocols like `OdosRouterV2` use **address(0)** to represent native tokens, creating a mismatch where **address(0)** could be passed to `OracleManager.getPrice()` without explicit handling, potentially returning incorrect or uninitialized price data.

It's recommended to normalize **address(0)** to **0xEeeeeEeeeEeEe...EeeeeEeeeeeeeeEEeE** in *SwapperLib._getValue()* before calling *OracleManager.getPrice()* and to add an explicit revert in *OracleManager.getPrice()* if **asset == address(0)** to prevent mispricing.

TRST-R-46: BaseCToken should support flashLoan() as a source of revenue

Flash loan functionality is only implemented in *BorrowableCToken*, but flash loans are a revenue source that benefits shareholders across all cTokens, not just borrowable ones. It's recommended to implement a virtual *flashLoan()* function in *BaseCToken* to enable flash loans on all cTokens, with *BorrowableCToken* overriding it where needed for borrowable-specific logic.

TRST-R-47: Redundant overflow check for revenue accumulation in AuctionManager

The sum of **pendingRevenue + msg.value** is checked for overflow as **type(uint208).max** but then cast to **uint192**. Since revenue is denominated in native tokens and the protocol assumes token amounts fit into **uint136**, a **uint192** overflow (~6e57) is unrealistic, it would require a market cap far beyond protocol assumptions.

It's recommended to remove the **uint208** overflow check and document that revenue is assumed to remain below **type(uint192).max**, consistent with the protocol's **uint136** token amount assumptions.

TRST-R-48: CentralRegistry should support locking markets

CentralRegistry.unlockAuctionForMarket() unlocks a market for auctions, but there is no corresponding function to relock it. Once unlocked, a market remains accessible to the *AuctionManager* for the transaction duration with no cleanup mechanism. While not currently problematic, this limits flexibility for future *AuctionManagerV2* implementations that may require explicit lock/unlock cycles or support for multiple auction managers.

It's recommended to add a function to allow explicit cleanup and support future multi-auction-manager scenarios.

TRST-R-49: AuctionManager UserOp can call unintended functions

When **userOp.data** does not match **AuctionManager.update selector**, no validation occurs.

```
// Else if UserOp does not contain a RedStone update, continue as no-op UserOp  
// This case is for liquidations triggered by interest accrual, not the oracle.
```

The comment claims this handles "no-op" cases for interest-accrual-triggered liquidations, but **userOp.data** can contain any call the execution environment is authorized to make (e.g.,

initiateAuction(), *preSolverSetup()*, *accumulateRevenue()*, *update()*). While the **userOp** signer is trusted (same as auctioneer EOA per README), the lack of validation contradicts the stated "no-op" intent.

It's recommended to add an else case in *_preOpsCall()*, validating that **userOp.data** calls *initiateAuction()* as the no-op case, aligning implementation with the comment's intent.

TRST-R-50: AuctionManager destination setters should distribute revenue

The setters *setFastlaneDestination()* and *setCurvanceDestination()* update revenue destinations without calling *_distributeRevenue()* to send accumulated revenue to the old destinations. This conflates revenue accrued under different destinations, making historical accounting unclear. While calling *_distributeRevenue()* before updating would cleanly separate periods, it introduces a tradeoff in that a malicious Fastlane destination could revert on ETH transfer and block Curvance from receiving their accumulated revenue.

If the tradeoff is accepted it's recommended to call *_distributeRevenue()* before updating destinations in *setFastlaneDestination()* and *setCurvanceDestination()*.

TRST-R-51: SwapUnsafe() in Zappers

Zappers use *swapUnsafe()* while position managers use *swapSafe()*, which includes an added slippage check. This creates an inconsistency in protection levels:

```
// In zappers
SwapperLib._swapUnsafe(centralRegistry, swapAction);

// In position managers
SwapperLib._swapSafe(centralRegistry, swapAction);
```

Zapper users are expected to be less sophisticated than position manager users since zappers contain more standard, user-friendly functions. The typical security approach is that more sophisticated users can have less safeguards and vice versa. By this logic, zappers should provide more protection for less sophisticated users, but currently it is the opposite, zappers lack the slippage check that position managers have.

It's recommended to use *swapSafe()* in zappers to provide slippage protection for less sophisticated users, bringing zappers and position managers to the same security standard.

TRST-R-52: Redundant code

Several instances of dead code, redundant operations, and unused error definitions have been identified:

- The check **adaptorToAdd != adaptorToRemove** is redundant since **adaptorToAdd** is not approved and **adaptorToRemove** is approved, and therefore they cannot be the same:

```
if (adaptorToAdd == adaptorToRemove) {
    revert OracleManager__InvalidParameter();
}
```

- The multicall provider code is unused:

```
function setMulticallProviders(
    address[] calldata providers,
    bool supported
) external {
    _checkElevatedPermissions();

    uint256 numProviders = providers.length;
    address cachedProvider;

    for (uint256 i; i < numProviders; ++i) {
        cachedProvider = providers[i];
        if (isMulticallProvider[cachedProvider] == supported) {
            revert CentralRegistry__InvalidParameter();
        }

        isMulticallProvider[cachedProvider] = supported;
        emit MulticallProviderSet(cachedProvider, supported);
    }
}
```

- The **& _BITMASK_VESTING_RATE** operation is not functional as **_vestingData** is zero-initialized:

```
_vestingData = (_vestingData & _BITMASK_VESTING_RATE) |
    (block.timestamp << _BITPOS_VEST_END) |
    (block.timestamp << _BITPOS_LAST_VEST) |
    (WAD << _BITPOS_DEBT_INDEX);
```

If **_vestingData** had non-zero bits in the first **96 bits**, the line would not clear them despite that being an important property. However, since **_vestingData** is zero-initialized storage it's recommended to set only the required subfields and ignore the first **96 bits**, removing the bitmask operation.

- In *BorrowableCToken.accrueIfNeeded()*, the code zeros out all except the lower **96 bits** in **rate**:

```
rate := and(rate, _BITMASK_VESTING_RATE)
```

This is dead code with *DynamicIRM* since a **rate > 1e25** (which still fits easily into **96 bits**) is numerically impossible. However, it's unsafe to silently cast down. It's recommended to revert with a descriptive error if **rate** is too large for **96 bits** instead of masking. If a rate that overflows **96 bits** was possible, it could DOS the protocol without being able to recover, so the restricted value range inside the interest rate model is an important security property.

- The following **else** case in the liquidation flow is dead code because it is entered when **snap.isCollateral = false && snap.debtBalance = 0**, which is not possible based on the logic in *BorrowableCToken.getSnapshot()*. It's recommended to document that this code is unreachable, yet the case can be handled to not rely on properties elsewhere in the codebase.

```

} else {
    // If there is no debt and its not a position
    // to be modified, clean up the position entry as the
    // user was liquidated (bad debt insolvency).
    if (action.cTokenModified != snap.asset) {
        positionsToClose[i] = true;
        if (result.positionClosureNeeded == 0) {
            result.positionClosureNeeded = 2;
        }
    }
}
}

```

- In *BasePositionManager.onBorrow()* the **borrowableCToken** does not have approval. It's recommended to remove this approval clearing call since *depositAsCollatera()* always consumes the entire allowance.

```

// Remove any excess approval.
SwapperLib._removeApprovalIfNeeded(
    debtAsset,
    address(borrowableCToken)
);

```

- The contract unnecessarily approves **wrappedNative** to spend its own tokens in *NativeVaultZapper*:

```

SwapperLib._approveIfNeeded(
    wrappedNative,
    wrappedNative,
    swapAction.inputAmount
);

```

It's recommended to remove this self-approval as it serves no purpose and common implementations of the *IWETH* interface don't require this approval.

- In *DynamicIRM*, the **DynamicIRM__InvalidAdjustmentRate** error is defined but it's never used:

```

error DynamicIRM__InvalidAdjustmentRate();

```

Mitigation review

By commit [f10667c](#), the redundant check for **adaptorToAdd != adaptorToRemove** still exists. It has been removed by commit [6ea8779](#). The other instances have either been fixed or determined to be intended.

TRST-R-53: Coding style improvements

Several coding style and consistency issues were identified:

- In *updateTokenConfig()*, similar variable names are used for different structs:

```
CurvanceToken storage ct = _tokenConfig[c.cToken];
```

Using **c** and **ct** variables for different structs which have the same fields is error-prone, as it's easy to type the wrong variable. It's recommended to use more distinctive names like **storedConfig** and **inputConfig**.

- Constants are defined to represent error codes:

```
/// @dev Return value indicating no price returned at all.
uint256 constant BAD_SOURCE = 2;

/// @dev Return value indicating price divergence or a missing price feed.
uint256 constant CAUTION = 1;

/// @dev Return value indicating no price error.
uint256 constant NO_ERROR = 0;
```

However, in some instances raw integers are used instead of the constants, such as in *BasePositionManager.maxRemainingLeverageOf()*:

```
if (errorCode != 0) {
    revert BasePositionManager__InvalidTokenPrice();
}
```

It's recommended to use the error code constants consistently throughout the codebase.

- Contracts should inherit their interfaces to avoid discrepancies. In particular, *CentralRegistry* does not inherit *ICentralRegistry* but claims to adhere to it in *supportsInterface()*:

```
function supportsInterface(
    bytes4 interfaced
) public view virtual override returns (bool) {
    return
        interfaced == type(ICentralRegistry).interfaced ||
        interfaced == type(IActionRegistry).interfaced ||
        super.supportsInterface(interfaced);
}
```

This is error prone as changes to the interface won't trigger compiler errors. It's recommended that contracts inherit their corresponding interfaces to ensure type safety and to prevent discrepancies.

Mitigation review

By commit [f10667c](#), error code constants are still missing in the following instances:

- [MarketManagerIsolated.sol#L534](#)
- [MarketManagerIsolated.sol#L1293](#)
- [MarketManagerIsolated.sol#L1377](#)

- [MarketManagerIsolated.sol#L1657](#)
- [LiquidityManagerIsolated.sol#L341](#)

These instances are addressed by commit [6ea8779](#).

TRST-R-54: Documentation errors

Several documentation issues were identified:

- *OracleManager* documentation is outdated:

```
/// minimum price of 1, and a maximum price of 2^240 - 1. Though,  
/// some oracle adaptors have lower maximums, which will natively  
/// restrict the maximum price returned. Such as Chainlink's uint192  
/// maximum. When using the Oracle Manager, verify what oracle adaptors
```

It's recommended to update the documentation to reflect the current price format and constraints.

- *setCollateralizationPaused()* has incorrect parameter documentation:

```
/// @param cToken The Curvance token to set minting status for.
```

- *BaseZapper* comment is incorrect:

```
// Validate `cToken` exists, otherwise transfer their tokens  
// back and return.
```

Execution reverts instead of transferring back tokens. It's recommended to update the comment to reflect the actual behavior.

- Boost multiplier documentation states incorrect threshold:

```
/// must be a positive boost i.e. > 1.01 or greater multiplier.
```

It's recommended to correct this to **> 1** in both instances in *CentralRegistry*.

- Chain ID format documentation is inconsistent:

```
/// @notice Returns an array of Chain IDs recorded in the Crosschain
```

```
/// @notice Array of Chain IDs recorded in the Crosschain Protocol's Chain  
/// ID format.
```

Chain IDs are in GETH format, not the crosschain protocol format. It's recommended to clarify this in the documentation.

- The comment referring to ERC777 should be removed from *BaseCToken._processDeposit()* as ERC777s are not part of the threat model:

```
// Need to transfer before minting or ERC777s could reenter.
SafeTransferLib.safeTransferFrom(asset(), by, address(this), assets);
```

- The `_MAX_VERTEX_ADJUSTMENT_VELOCITY` explanation is incorrect:

```
/// @notice The maximum rate at which `vertexMultiplier` is adjusted,
/// in BPS on top of base rate (1 `BPS`).
/// E.g. 1 * BPS = 200% multiplied to vertex interest rate per
/// `ADJUSTMENT_RATE` at 100% utilization.
uint256 internal constant _MAX_VERTEX_ADJUSTMENT_VELOCITY = 2000;
/// @notice The minimum rate at which `vertexMultiplier` is adjusted,
/// in BPS on top of base rate (1 `BPS`).
/// E.g. 0.01 * BPS = 101% multiplied to vertex interest rate per
/// `ADJUSTMENT_RATE` at 100% utilization.
uint256 internal constant _MIN_VERTEX_ADJUSTMENT_VELOCITY = 100;
```

It should say “120% multiplied” since `_MAX_VERTEX_ADJUSTMENT_VELOCITY` is set to **2000 = 20%**.

- The `depositAndLeverage()` documentation is incorrect since `depositAsCollateralFor()` will revert if `BasePositionManager` is not a valid delegate. It's recommended to update the note accordingly.

```
/// @dev Measures slippage through pre/post conditional slippage check
/// in `checkSlippage` modifier.
/// NOTE: The caller MUST have approved this smart contract to have
/// delegated actions inside `action.cToken` or
/// depositAsCollateralFor will only deposit and the leverage
/// operation will fail.
```

- Harvester references should be removed from `addMarketPermissions()` and `removeMarketPermissions()`:

```
//// @notice Authorizes an address to manage markets.
/// @notice Adds a Harvester contract for use in Curvance.
/// @dev Only callable on a 5-day delay or by the Emergency Council.
/// Cannot be a supported Harvester contract prior.
/// Emits a {PermissionsUpdated} event.
/// @param newAddress The address to add market permissions to
/// inside Curvance.
function addMarketPermissions(address newAddress) external {
    _checkElevatedPermissions();

    // Validate `newAddress` is not currently supported.
    if (hasMarketPermissions[newAddress]) {
        revert CentralRegistry__InvalidParameter();
    }

    hasMarketPermissions[newAddress] = true;
    emit PermissionsUpdated("Market", newAddress, true);
}

//// @notice Deauthorizes an address to manage markets.
/// @dev Only callable on a 5-day delay or by the Emergency Council.
/// Has to be a supported Harvester contract prior.
/// Emits a {PermissionsUpdated} event.
/// @param addressApproved The address to remove market permissions from
/// inside Curvance.
function removeMarketPermissions(address addressApproved) external {
```

```

    _checkElevatedPermissions();

    // Validate `addressApproved` is currently supported.
    if (!hasMarketPermissions[addressApproved]) {
        revert CentralRegistry__InvalidParameter();
    }

    delete hasMarketPermissions[addressApproved];
    emit PermissionsUpdated("Market", addressApproved, false);
}

```

- Core contract setter documentation is misleading:

```

/// @dev Only callable on a 5-day delay or by the Emergency Council.
/// Only settable once. Emits a {FeeTokenSet} event.
/// @param newFeeToken The new address of fee token.
function setFeeToken(address newFeeToken) external {
    _checkCanSetCoreContract(feeToken);
    _checkElevatedPermissions();
    feeToken = newFeeToken;
    emit FeeTokenSet(newFeeToken);
}

```

The contracts are not “only settable once”, they are just “not settable more than once after genesis”.

- *NativeVaultZapper* incorrectly refers to ERC4626:

```

/// The "Native Vault" contract is the zapper for working with
/// native-token based erc4626 tokens such as shMON or aprMON. No type

```

The **deposit()** **payable** interface defined in *IVault* is not ERC4626 compatible. Thus, it should not be referred to as ERC4626 tokens, but, for example, native vault tokens.

- The *maxRemainingLeverageOf()* documentation does not mention that it can overestimate its result due to leverage fees that are charged in *BasePositionManager.onBorrow()*. This consideration should be added to the comment referring to factors such as AMM fees and slippage as sources for deviation.
- The *AuctionManager* constructor documentation contradicts its implementation regarding **zeroSolvers**:

```

/// - zeroSolvers: false - Allows oracle updates without solvers
/// (no OEV capture).

```

The constructor passes **false** for **zeroSolvers**, but according to Atlas documentation, setting **zeroSolvers** to **false** means the *metacall()* will not proceed if there are no solver operations (at least 1 must exist). To allow oracle updates without solvers, **zeroSolvers** must be **true**. It's recommended to correct the documentation, since it has been determined that the implementation is correct.

Mitigation review

The recommended documentation changes have been implemented, except for the Chain ID format, where the comments are still wrong. This has been addressed by commit [1933505](#).

Centralization risks

TRST-CR-1: Emergency Council

The Emergency Council is assigned DAO permissions, elevated permissions, and market permissions inside *CentralRegistry*:

```
mapping(address => bool) public hasDaoPermissions;
mapping(address => bool) public hasElevatedPermissions;
mapping(address => bool) public hasMarketPermissions;
```

The Emergency Council is also assigned the canceller role inside *DAOTimelock*:

```
_grantRole(CANCELLER_ROLE, _EMERGENCY_COUNCIL);
```

In contrast to *DAOTimelock*, the Emergency Council is expected to act without a delay. The Emergency Council can configure protocol parameters, pause markets, add or remove assets from oracles, update fees, cancel timelock transactions, and perform any action requiring elevated permissions. The Emergency Council is therefore fully trusted with a trust level even higher than the *DAOTimelock*.

TRST-CR-2: DAO

The DAO address is assigned DAO permissions inside *CentralRegistry*:

```
mapping(address => bool) public hasDaoPermissions;
```

It is set as the proposer, executor, and canceller inside *DAOTimelock*:

```
_DAO_ADDRESS = cr.daoAddress();
_grantRole(PROPOSER_ROLE, _DAO_ADDRESS);
_grantRole(EXECUTOR_ROLE, _DAO_ADDRESS);
_grantRole(CANCELLER_ROLE, _DAO_ADDRESS);
```

Elevated permissions (DAO Timelock and Emergency Council) can call *CentralRegistry.transferDaoPermissions()* to change the DAO address:

```
function transferDaoPermissions(address newDaoAddress) public virtual {
    _checkElevatedPermissions();

    //
```

DAO permissions are allowed to directly call *CentralRegistry.withdrawFees()* and *BorrowableCToken.skim()* without timelock delay. The intention is that the DAO cannot perform malicious actions unless going through the **>=5 day** timelock.

TRST-CR-3: DAO Timelock

The DAO Timelock is assigned DAO permissions, elevated permissions, and market permissions inside *CentralRegistry*:

```
mapping(address => bool) public hasDaoPermissions;  
mapping(address => bool) public hasElevatedPermissions;  
mapping(address => bool) public hasMarketPermissions;
```

The timelock is managed by the DAO address with a timelock delay of at least **5 days**. The Emergency Council can call *CentralRegistry.transferTimelockPermissions()* to change the timelock address:

```
function transferTimelockPermissions(address newTimelock) external {  
    _checkEmergencyCouncilPermissions();  
    //
```

The DAO timelock can configure protocol parameters, manage markets, update oracle settings, adjust fees, and perform administrative functions. By enforcing the minimum delay of 5 days, users are provided with sufficient time for reacting to proposals. Overall, the DAO Timelock is fully trusted but operates under transparency via the timelock mechanism, this means the community can likely react to malicious operations in time.

TRST-CR-4: Risk Manager contracts with market permissions

Risk Managers are assigned market permissions inside *CentralRegistry*:

```
mapping(address => bool) public hasMarketPermissions;
```

Risk Managers are contracts with their own specific restrictions on actions. These restrictions can be enforced by the external Risk Manager contracts, for example, when calling *setGuardedPriceConfig()*, a restriction may be implemented to only adjust values by a specific percentage per week.

A Risk Manager can have any trust level depending on the Risk Manager contract implementation. For example, updating **collReqSoft** via *updateTokenConfig()* can cause instant liquidations, since positions that were previously healthy under the old configuration can suddenly become liquidatable under the new one.

TRST-CR-5: Auction permissions

AuctionManager contracts are assigned auction permissions inside *CentralRegistry*:

```
mapping(address => bool) public hasAuctionPermissions;
```

AuctionManager contracts can call *CentralRegistry.unlockAuctionForMarket()*, *CentralRegistry.resetAuctionForMarket()*,

MarketManagerIsolated.setTransientLiquidationConfig(), and *MarketManagerIsolated.resetTransientLiquidationConfig()*. They can set **incentive** and **closeFactor** within the range allowed by the market's configuration:

```
if (incentive < c.liqIncMin || incentive > c.liqIncMax) {
    _revert(_INVALID_PARAMETER_SELECTOR);
}

// Validate 'closeFactor' is within configured allowed close factor
// range. This also validates 'closeFactor' is not > the 16 bits we
// have allocated.
if (closeFactor < c.closeFactorMin || closeFactor > c.closeFactorMax) {
    _revert(_INVALID_PARAMETER_SELECTOR);
}
```

Collateral values for health checks are discounted by **AUCTION_BUFFER** when liquidated with an *AuctionManager*, granting priority to perform liquidations.

The *AuctionManager* contract itself has **bundler**, **auctioneer**, **user**, and **governor** roles. The **auctioneer** and **user** roles are held by the same Fastlane EOA. The **bundler** needs to be part of the data signed by the Fastlane EOA (Fastlane will be liberal in assigning bundlers), and the **governor** is a multisig with Curvance and Fastlane signers.

Regardless of role assignments, the principal consideration with regards to the safety of Curvance is that liquidatable accounts can be liquidated with **incentive** and **closeFactor** within the allowed range.

Systemic risks

TRST-SR-1: Token risks and assumptions

The protocol makes assumptions about token behaviors and properties that are critical for safe operation. These include assumptions that tokens do not have a fee-on-transfer (FoT), are not rebasing, do not perform callbacks, and comply with ERC20. Tokens are expected to have between 6 and 24 decimals.

The protocol trusts tokens not to change their behaviors and to stay within assumed ranges of value. The following token value and range assumptions must hold:

- Minimum \$ price per token = **\$5e-4 (\$0.0005)**, in WAD: **5e14**
- Maximum \$ price per token = **\$10e6 (\$10m)**, in WAD: **1e25**
- Maximum \$/wei: **\$1e7 / 1e8 wei = 0.1 \$ per wei**
- Minimum \$/wei: **\$5e-4 / 1e24 wei = 5e-28 \$ per wei**
- **totalSupply** fits into **uint136** (minimum market cap to reach this is **2¹³⁶ wei * 5e-28 \$ per wei = \$43T**). Market caps up to **\$43T** can safely be used within the protocol

If tokens violate these assumptions the protocol's accounting and liquidation mechanisms may break down, leading to loss of funds. Unlike systems that can remove malicious tokens reactively, many of the protocol's core operations assume these properties hold continuously. Therefore, tokens must be carefully vetted before listing, and the protocol should maintain a whitelist of supported token functionalities to prevent incompatible assets from being added.

TRST-SR-2: Oracle risks

Oracles are trusted to report correct prices, and to adhere to their stated heartbeats and deviation thresholds.

Besides PUSH-based oracles that update prices when deviation crosses a threshold (e.g., Chainlink), the protocol also uses a PULL-based Redstone oracle which does not automatically update prices at a certain deviation threshold. If prices are not manually updated frequently, and the allowed staleness period is not configured safely, the protocol may operate with prices that deviate significantly from real market values.

For wrapped aggregators, there is additional risk that *convertToAssets()* (the vault exchange rate) does not return the rate at which the free market would value the shares. Potential causes include:

- Withdrawal locking mechanisms (discount for illiquidity can be expected)
- Withdrawal fees that reduce the actual redeemable value
- Available off-chain information that is not yet represented on-chain

This introduces deviations similar to other oracle types. For example, if a vault has announced a loss but the vault shares are still valued at the old `convertToAssets()` rate, borrowers using those shares as collateral may not be properly liquidated, exposing lenders to bad debt.

TRST-SR-3: Min/max price guards can override oracle prices and block operations

The protocol's price guard mechanism returns a successful max price when the upper guard is hit, rather than reverting. And the price guard mechanism returns an invalid zero price when the lower guard is hit which bubbles up as a **BAD_SOURCE** error. This asymmetric behavior is a design choice. The lower guard acts as a circuit breaker, blocking all protocol operations that rely on the token's price. And the upper guard aims to protect against inflated prices. The risk of capping a price is that the price might genuinely deviate to the upside, which e.g., could allow borrowers to take on excessive debt that has a higher market value than the internal valuation calculated by the protocol.

TRST-SR-4: Extreme interest risk and interest assumptions

The protocol assumes interest rates are properly managed and that 5-year interest doesn't appreciate the cToken exchange rate by more than **1000x**. Longer time periods or higher interest rates have not been checked for compatibility with core markets. A margin of safety exists and there is no hard limit where something specific breaks after 5 years or more than **1000x** in 5 years.

An example of a limitation is **marketDebtIndex**, defined as **uint80** with a maximum value of approximately **1.2e24**. Since its starting value is **1e18**, a factor of at most **1.2e6** can be multiplied before an unsafe cast occurs that truncates leading bits:

```
shl(_BITPOS_DEBT_INDEX, marketDebtIndex)
```

At a rate of **1000x** in 5 years, an overflow would occur in roughly 10 years. If interest rates are mismanaged or markets operate for extended periods with high utilization, the debt index could eventually overflow, causing incorrect debt calculations and loss of funds.

TRST-SR-5: External protocol risk

Curvance integrates with external protocols including DEXs for swaps, ERC4626 vaults as cToken assets, and the Atlas framework (<https://www.atlasevm.com/>) for auction-based liquidations.

This audit has only assessed the specific integration of Odos (<https://docs.odos.xyz/>) and Atlas. Other DEX integrations and vault protocols have not been in scope. An issue in an external integration can lead to loss of funds and the protocol's security depends on the security of these external integrations.

TRST-SR-6: Dual borrowable market setup can disincentivize liquidators

Curvance markets can be set up with two *BorrowableCToken* where both **cTokens** can be borrowed using the others shares as collateral.

In such a setup, liquidators may have to discount the received liquidated shares due to multiple factors:

- Shares can't be redeemed immediately during high utilization which is an opportunity cost as well as requiring liquidators to take on credit risk of Curvance.
- Unrealized bad debt can effectively make shares less valuable than calculated in the liquidation flow.